Annotators' Instruction-Assisted Prompting (AIAP): Bridging Human Annotation and Machine Reasoning for Financial Sentiment Classification

Anonymous

Submitted for the Degree of Master of Science in

Machine Learning



Department of Computer Science
Royal Holloway University of London
Egham, Surrey TW20 0EX, UK
June 10, 2026

Declaration

This report has been prepared on the basis of my own work. Where other published and unpublished source materials have been used, these have been acknowledged.

Word Count: 14,885

Student Name: Anonymous

Date of Submission: September 1, 2026

Abstract

The rapid adoption of Large Language Models (LLMs) in the financial sector has made prompt engineering a critical factor in ensuring accurate, domain-specific text classification. This study presents a comprehensive comparative evaluation of standard prompting strategies (Zero-Shot, Few-Shot, and Self-Consistency) against a novel Annotators' Instruction-Assisted Prompting (AIAP) hybrid technique for the task of Financial Sentiment Analysis. To rigorously assess performance, 24 targeted tests were conducted across three distinct financial datasets of varying annotator agreement levels, utilizing both an 8B-parameter domain-specialized model (Fin-R1-GGUF) and a 3B-parameter general-purpose model (Llama-3.2-abliterate).

The empirical results yielded three primary findings. First, evaluating prompt efficacy revealed that while the AIAP hybrid method achieved performance parity on highly agreeable data, Self-Consistency emerged as the most robust baseline across noisier, mixed-domain datasets. Second, challenging assumptions regarding model scale and specialization, the smaller, general-purpose Llama-3.2-abliterate model outperformed the larger, domain-specific Fin-R1 model, on the datasets where both were tested, effectively matching commercial baselines. Finally, the study demonstrated a direct, proportional relationship between model accuracy and human annotator agreement, revealing that the inherent relativity of ground truth in subjective datasets acts as a fundamental cap on classification performance. Ultimately, this research provides actionable insights into optimizing local, consumer-hardware-compatible LLMs for complex financial reasoning tasks.

Contents

1. Introduction 1

1.1 Defining the Hypotheses 1

1.1.1 H1: Accuracy depends on annotator agreement. 2

1.1.2 H2: A Financially Specialized LLM is better for FSA. 2

1.1.3 H3: The annotators' instruction-assisted prompting can outperform Few-Shot. 2

1.1.4 H4: 3B-LLMs can compete with GPT-3.5-like models on FSA 3

1.2 The Significance of this study 3

1.3 Future Career Impact of this Study 4

1.3.1 Technical Skills 4

1.3.2 Finance Domain Knowledge 4

1.3.3 Career Direction 5

2. Background Research 6

2.1 Prompt Engineering and its Techniques 6

2.1.1 Prompt Engineering for Financial Tasks 6

2.1.2 Zero-Shot Prompting 6

2.1.3 Few-Shot Prompting 6

2.1.4 Chain of Thought 6

2.1.5 Self-Consistency 7

2.2 Related Studies and Experiments 7

3. Methodology 9

3.1 Aim 9

3.2 Dataset Selection 9

3.2.1 Financial PhraseBank 9

3.2.2 SEntFiN 1.0 10

3.2.3 Financial Opinion Mining and Question Answering (FiQA) 11

3.3 Large Language Model Selection 11

3.3.1 Fin-R1 11

3.3.2 Llama 3.2 12

3.4 Dataset Preprocessing 12

3.4.1 Financial PhraseBank 12

3.4.2 SEntFiN 1.0 13

3.4.3 Financial Opinion Mining and Question Answering (FiQA) 13
3.5 Templates for Prompting Techniques 13
3.5.1 Zero-Shot 14
3.5.2 Few-Shot 14
3.5.3 Self-Consistency 14
3.5.4 Chain of Thought 14
3.5.5 Annotators' Instruction (Novel Method) 14
3.6 Standard Output Format for Experiment 15
3.7 Evaluation Metrics 15
3.7.1 Precision 15
3.7.2 Recall 16
3.7.3 F1-Score 16
3.7.4 Macro F1-Score 16
3.7.5 Weighted F1-Score 16
3.7.6 Confusion Matrix 17
3.7.7 Support 17
3.7.8 Token Efficiency 17
3.7.9 McNemar's Test 18

4. The Experiment 19

4.1 Mini-Experiment 19
4.2 Improvisations for Main Experiment 19
4.2.1 Replacement for Chain of Thought 19
4.2.2 Increase in Output Token Limit 20
4.2.3 Hybridization of Novel Method 20
4.2.4 Selection of Final Datasets 21

5. Experiment Results 22

5.1 Individual Performance Results 22
5.1.1 Fin-R1-GGUF | FiQA | Few-Shot 22
5.1.2 Fin-R1-GGUF | FiQA | Hybrid AIAP 22
5.1.3 Fin-R1-GGUF | FiQA | Self-consistency 23
5.1.4 Fin-R1-GGUF | FiQA | Zero-Shot 23
5.1.5 Fin-R1-GGUF | FP-50 | Few-Shot 23
5.1.6 Fin-R1-GGUF | FP-50 | Hybrid AIAP 23

5.1.7 Fin-R1-GGUF | FP-50 | Self-Consistency 24
5.1.8 Fin-R1-GGUF | FP-50 | Zero-Shot 24
5.1.9 Fin-R1-GGUF | FP-75 | Few-Shot 24
5.1.10 Fin-R1-GGUF | FP-75 | Hybrid AIAP 25
5.1.11 Fin-R1-GGUF | FP-75 | Self-Consistency 25
5.1.12 Fin-R1-GGUF | FP-75 | Zero-Shot 25
5.1.13 Llama 3.2 - Abliterate | FiQA | Few-Shot 25
5.1.14 Llama 3.2 - Abliterate | FiQA | Hybrid AIAP 26
5.1.15 Llama 3.2 - Abliterate | FiQA | Self-Consistency 26
5.1.16 Llama 3.2 - Abliterate | FiQA | Zero-Shot 26
5.1.17 Llama 3.2 - Abliterate | FP-50 | Few-Shot 27
5.1.18 Llama 3.2 - Abliterate | FP-50 | Hybrid AIAP 27
5.1.19 Llama 3.2 - Abliterate | FP-50 | Self-Consistency 27
5.1.20 Llama 3.2 - Abliterate | FP-50 | Zero-Shot 28
5.1.21 Llama 3.2 - Abliterate | FP-66 | Few-Shot 28
5.1.22 Llama 3.2 - Abliterate | FP-66 | Hybrid AIAP 28
5.1.23 Llama 3.2 - Abliterate | FP-66 | Self-Consistency 28
5.1.24 Llama 3.2 - Abliterate | FP-66 | Zero-Shot 29
5.1.25 Llama 3.2 - Abliterate | FP-75 | Few-Shot 29
5.1.26 Llama 3.2 - Abliterate | FP-75 | Hybrid AIAP 29
5.1.27 Llama 3.2 - Abliterate | FP-75 | Self-Consistency 30
5.1.28 Llama 3.2 - Abliterate | FP-75 | Zero-Shot 30
5.1.29 Llama 3.2 - Abliterate | FP-100 | Few-Shot 30
5.1.30 Llama 3.2 - Abliterate | FP-100 | Hybrid AIAP 30
5.1.31 Llama 3.2 - Abliterate | FP-100 | Self-Consistency 31
5.1.32 Llama 3.2 - Abliterate | FP-100 | Zero-Shot 31
5.2 Prompting Technique Performance Results 31
5.2.1 Few-Shot 31
5.2.2 Hybrid AIAP 32
5.2.3 Self-Consistency 32
5.2.4 Zero-Shot 32
5.3 Dataset-Based Performance Results 33
5.3.1 FiQA 33
5.3.2 FP-50 33

5.3.3 FP-66 33
5.3.4 FP-75 33
5.3.5 FP-100 34
5.4 LLM-Based Performance Results 34
5.4.1 Fin-R1-GGUF 34
5.4.2 Llama 3.2 Abliterate....34
5.5 Overall Performance Results....35
5.6 McNemar's Test Results - Statistical Significance....35
5.7 Empirical Findings vs. Initial Hypotheses 36
5.7.1 Accuracy depends on Annotator Agreement. 36
5.7.2 A Financially Specialized LLM is better for FSA. 37
5.7.3 The annotators' instruction-assisted prompting can outperform Few-Shot. 37

6. Discussion....40
6.1 Deciphering the Findings 40
6.2 Error Analysis....45
6.3 Disparity with Resource Usage....47
6.4 Project Limitations 48

7. Conclusion & Future Scope....49
7.1 The Impact of Model Abliteration over Size and Specialization 49
7.2 Conditional Efficacy of the AIAP Hybrid Approach 49
7.3 The Relativity of Ground Truth in Subjective Tasks 50

8. Self-Assessment....51
8.1 Project Execution and Achievements....51
8.2 Challenges Overcome....51
8.3 Lessons Learned and Career Impact....51

9. References....53

10. How to Use My Project....55

11. Appendix A....57
11.1 deduplicate_phrasebank.py....57
11.2 process_fiqa.py 59
11.3 experiment_harness.py....61
11.4 evaluate_results.py....73

12. Appendix B 90

1. Introduction

Prompt engineering is the process of designing, evaluating, and refining prompts sent to large language models (LLMs) like GPT. The objective is to design the prompts in a way that enables the LLMs to provide the most relevant and precise responses. This involves studying how the AI model deciphers and responds to varying prompt structures, as well as aligning those prompts for distinct use cases. Well-structured prompts are believed to significantly improve the performance of LLMs, thereby increasing their relevance across various applications. In the context of Financial Sentiment Analysis (FSA), there are various prior studies on standard prompting techniques targeting widely used cloud-based AI models like GPT-4o, as well as more primitive, task-focused models like FinBERT, which are optimized for the task. Prompt engineering is a versatile and interdisciplinary field that requires an understanding of Natural Language Processing, Artificial Intelligence, and domain-specific applications. For this particular study, there will be a focus on how intelligent prompting can assist LLMs in achieving the task of Financial Sentiment Analysis (FSA) with better accuracy on datasets with varying clarity and annotator acceptance. As noted by Samir Lemeš[1], "As AI models become more sophisticated, the role of prompt engineering will become even more important in leveraging their full potential."

1.1 Defining the Hypotheses

The motive of this study is to test multiple theories grounded in a theoretical understanding of domain knowledge, Machine Learning, Natural Language Processing, and other aspects of understanding how Artificial Intelligence behaves. For the Finance sector, accuracy has always been a term of utmost importance; hence, questions arise such as: "Are Large Language Models reliable to interpret the financial data, and if so, to what measure?" Taking the example of stock markets, these are known to be volatile and are largely driven by greed, fear, and investors' perceptions of how other investors will react to relevant occurrences. In clearer terms, every investor looks at financial price data and local and global news, and tries to predict how most investors will react to the current situation, because that is what will ultimately drive the direction of stock prices. Expecting Artificial Intelligence-based technologies like Neural Networks and Large Language Models to naturally grasp the understanding of complex, interdependent aspects, such as those just discussed, raises doubts.

Hence, the most basic way to address these doubts is to test the accuracy of Large Language Model responses when presented with financial statements with a simple objective of analysing the statement to conclude whether the statement is financially positive, negative, or neutral. Even with this task, which will determine

the accuracy of LLMs, this study aims to confirm several hypotheses, primarily focused on concerns regarding determining the best practices and conditions that lead to the best results Artificial Intelligence can provide at this stage for the decided task. Below, I will discuss the hypotheses for which experiments have been conducted in this study.

1.1.1 H1: Accuracy depends on annotator agreement.

For the task of Financial Sentiment Analysis (FSA) to be given to LLMs, existing datasets created as part of previously existing research need to be used. Now, it is widely recognized that annotators of financial sentiment often disagree on the sentiment of corporate statements, events, local news headlines, and international events, which tend to impact an entity's stability or future, irrespective of size. Consider a recent international incident censored only to give context for obvious reasons: "One of the highest-ranking administrative officials of a powerful but non-democratic country with a questionable hostility level dies in a helicopter crash under suspicious circumstances." Given the circumstances, how will international markets react? Some annotators and investors might hope that the new officer will bring internationally favourable reforms that reduce hostility, which will drive a market spike, while others might beg to differ on the possibility that the country might consider this an attack by an enemy and move towards war or instability against another nation, which will cause a market crash. Between these two extremes lies the possibility that the situation is a neutral and unfortunate accident that will be looked into, a new official with the same intentions as the previous one will take over, and there will be no change in the situation, which will drive non-significant micro-trends in the market price. Given the divide in opinions among annotators, investors, and generally intelligent individuals, it is probably unwise to assume that artificial intelligence can decisively predict financial sentiment on statements that are not fully agreed upon by the relevant human parties, since the truthfulness of partially contested labels is relative to begin with.

1.1.2 H2: A Financially Specialized LLM is better for FSA.

For any task in the finance industry, users might have preconceived notions or prejudices that push them to turn to financially specialized models such as FinBERT, BloombergGPT, FinGPT, etc. This study aims to test this theory to determine whether LLMs specialized for finance are better suited for finance-based tasks or whether general-purpose models like GPT are better suited. And if so, what is the difference in performance between the two types of Large Language Models?

1.1.3 H3: The annotators' instruction-assisted prompting can outperform Few-Shot.

A novel prompting approach will be tested in this study to determine whether placing an LLM in the same situation as a Financial Annotator helps boost the resulting accuracy. This raises the question of how an LLM performs when instructed in a manner very similar to that of a human annotator. There is

speculation that the model's behaviour might be more predictable when placed in the exact same human environment in which human annotators performed the same task. This experiment aims to test this theory to determine whether this method has any significance initially for financial sentiment analysis. If successful, this could prove useful for more complex financial tasks.

1.1.4 H4: 3B-LLMs can compete with GPT-3.5-like models on FSA

Traditionally, there is limited trust and usage of locally run small-sized LLMs, and most of today's data, queries, and problems are handled by Large Language Models by well-known cloud AI providers, which have trusted models with high accuracy. This study aims to test this prejudice. It is worthwhile to understand the performance gap between a smaller model with 3B parameters and one with tens of billions of parameters. This would better inform our decisions on how much computing power to use, depending on the tasks.

1.2 The Significance of this study

Large Language Models have evolved the way humans interact with technology, enabling natural language conversations with systems that can understand, reason, and generate human-quality text. From customer service chatbots to automated content creation, these models are taking over entire industry segments by automating tasks that previously required human expertise. However, the power of LLMs is not automatic—it depends heavily on how they are instructed. A poorly crafted prompt can yield irrelevant or undesired outputs, while a well-designed prompt can unlock a model's full capabilities.

The financial sector, in particular, stands to benefit from LLMs. Financial analysts spend countless hours reading news, acquisition reports, and regulatory filings to assess market sentiment. An LLM that can, with enhanced accuracy, classify the sentiment of financial text could have the capacity to save time, reduce human bias, and provide real-time insights, provided we figure out the best way to utilize its power. However, financial sentiment analysis is inherently challenging. Financial language is nuanced, context-dependent, and often ambiguous—a single sentence can be interpreted as positive, negative, or neutral depending on the reader's perspective, as discussed in the above sections. This subjectivity is reflected in benchmark datasets, where human annotators agree on sentiment labels only 50-75% of the time.

Despite growing interest in prompt engineering, most research has focused on general-purpose tasks such as arithmetic reasoning or common-sense question answering. Relatively little work has systematically evaluated prompt engineering techniques for financial sentiment analysis, particularly with smaller, locally run models. This gap matters because many practitioners—such as financial analysts, risk managers, and compliance officers—do not have access to large subscription-

based cloud-based models like GPT-4 for data security reasons. They need practical, cost-effective solutions that can run on modest hardware.

This study addresses that gap by systematically evaluating prompt-engineering techniques for financial sentiment analysis tasks. By using locally run models and multiple datasets, this research provides actionable insights for practitioners who need to deploy LLMs in real-world financial applications.

1.3 Future Career Impact of this Study

This project has equipped me with a unique combination of technical and domain-specific skills that will directly benefit my career as a Machine Learning Engineer and an Integration Specialist.

1.3.1 Technical Skills

I have developed hands-on expertise in working with Large Language Models, including deploying and running models locally using Ollama. Along with that, I learned how to manage model selection for specific purposes amid the plethora of options and systematically evaluate performance using rigorous metrics such as precision, recall, F1, and statistical significance tests. These skills are directly transferable to any role involving AI integration, as organisations increasingly seek professionals who can deploy and evaluate LLMs without relying solely on cloud APIs.

I have strengthened my software engineering capabilities by building a modular, reproducible experiment pipeline. I have developed a Python script that can be easily adapted for other NLP tasks, datasets, or models. Achieving this aligns my profile and deems my work capable of meeting future DevOps-based expectations, reinforcing my ability to create scalable solutions.

I have gained experience in prompt engineering—a skill that is becoming essential in every role within the IT domain. Understanding how subtle changes in prompt design affect model outputs is now much more than simply academics and directly translates to my efficiency in any and every future role I take up.

1.3.2 Finance Domain Knowledge

This project has given me a deep understanding of financial sentiment analysis as an entry point for testing LLMs for the finance domain and its importance in fintech, algorithmic trading, and risk management. I now appreciate the challenges of working with subjective, noisy, and domain-specific data—insights that will help me communicate effectively with clients and stakeholders in future roles.

1.3.3 Career Direction

Previously, as an Associate Software Engineer building a CRM for a Mumbai-based firm, I frequently had to translate business requirements into technical implementations, with a very vague understanding of how raw coding logic translates into meaningful financial insights. This project has helped me understand the significance of having deep multi-domain knowledge and has demonstrated my ability to independently scope, design, and execute a research project from problem formulation to data analysis. It has also reinforced my interest in AI applications for business—an area I plan to pursue professionally, whether by building AI-powered tools for entities or by transitioning into AI product development roles.

Writing the dissertation itself has sharpened my ability to communicate complex technical concepts clearly and with precision—a skill that is invaluable for writing documentation, and client-facing reports.

This project has not only advanced my technical skills but also deepened my understanding of how AI can solve real-world business problems. It has confirmed my desire to pursue a career where I can apply machine learning and software engineering to create practical, impactful solutions for enterprises.

2. Background Research

The aim of this section is to discuss the background research conducted which led to the formulation of the outline of the experiment conducted in this study. The relevant literature revealed a certain gap which could be filled by this experiment and hence these will be discussed in detail in the following sections to shed light on the inferred interpretation of the previously conducted studies that led to the ideation of this project.

2.1 Prompt Engineering and its Techniques

To fully understand the significance of this experiment and its limited scope within the vast domain of Machine Learning, it is imperative to begin with Prompt Engineering and its significance in both the general and domain senses. With an understanding of the generalized concept of Prompt Engineering, the standardized techniques used to implement it and their related literature will be discussed.

2.1.1 Prompt Engineering for Financial Tasks

Prompt Engineering, as discussed above, is the art of designing and structuring prompts sent to a Large Language Model with the intention of altering the responses to be favourably relevant to the requirements that led to the creation of the prompt. Prompt engineering is rapidly evolving from a niche skill to a critical competency across various industries, including finance. (Satyadhar Joshi, 2025)[2].

2.1.2 Zero-Shot Prompting

In Zero-Shot Prompting, the model receives a task description in the prompt but lacks labelled data for training on specific input-output mappings. The model then leverages its pre-existing knowledge to generate predictions for the new task based on the given prompt (P. Sahoo et al., 2025)[6]. In simpler terms, zero-shot prompting means instructions that don't include labelled examples (Jurafsky and Martin, 2026)[5].

2.1.3 Few-Shot Prompting

Few-Shot Prompting provides models with a few input-output examples to induce an understanding of a given task. According to P. Sahoo et al. (2025), providing even a few high-quality examples has improved model performance on complex tasks compared to no demonstration [6]. Concisely, the task of prompting with examples is sometimes called few-shot prompting, as contrasted with zero-shot prompting (Jurafsky and Martin, 2026)[5].

2.1.4 Chain of Thought

This prompting technique was introduced by Wei et. Al. (2022)[3] which replicates a series of intermediate reasoning steps, significantly improving the ability of large language models to perform complex reasoning. Consider one's own

thought process when solving a complicated reasoning task, such as a multi-step math word problem. It is typical to decompose the problem into intermediate steps and solve each before giving the final answer: "After Jane gives 2 flowers to her mom, she has 10 — then after she gives 3 to her dad, she will have 7 — so the answer is 7."

2.1.5 Self-Consistency

Wang et al. (2023)[4] proposed the self-consistency decoding strategy to replace the naive greedy decoding used in chain-of-thought prompting. It first samples a diverse set of reasoning paths rather than taking only the greedy one, and then selects the most consistent answer by marginalizing out the sampled reasoning paths. Self-consistency leverages the intuition that a complex reasoning problem typically admits multiple ways of thinking, leading to its unique correct answer.

2.2 Related Studies and Experiments

A notable number of relevant studies and experiments have been conducted, the results of which led to the ideation of this study. Let's take into consideration the results of Y. Shen and P. K. Zhang (2024)[7], of their experiment on the financial sentiment analysis using the Financial PhraseBank dataset from Malo et al. (2014)[10]. They used GPT-3.5-Turbo, GPT-4o, and FinBERT models with Zero-Shot and Few-Shot techniques to achieve a highest of 78%, 86% and 88% accuracy, respectively. The GPT-3.5-Turbo model is a 150B–175B-parameter model, a highly capable and large model; the same goes for GPT-4o, whose parameter count is not officially disclosed but is likely in the trillions, according to Neoteric sp. z o.o.[8]. FinBERT is an encoder-only, non-generative, domain-specific language model with roughly 110M parameters; a version of which has been trained exclusively on the Financial PhraseBank dataset, making it exclusively suited for the task.

JayaShree Shetty et al. (2025)[9] conducted a similar experiment using the same Financial Phrasebank dataset using the ProsusAI/FinBERT model, which has been extensively trained for Financial Sentiment Analysis on this exact dataset. They drew a comparison between the divisions of the dataset based on Annotator Agreement. The Financial Phrasebank dataset (Malo et al. (2014)[10]) is divided into 4 datasets with 50%, 66%, 75%, and 100% annotator agreement. Their results report an overall accuracy of 88.22%, which is closely consistent with the experiment by Y. Shen and P. K. Zhang (2024)[7]. Their experiment also featured an agreement classification task, which they reported facing significant challenges, reporting only 53.2% accuracy. On observing their reported accuracies of 37.93%, 34.62%, 30.93%, and 64.49% for 50%, 66%, 75%, and 100% annotator agreement, respectively. In their study, they have reportedly tried to explain the disparity in accuracy results by concluding that the model struggles to capture subtle linguistic ambiguities that lead to non-unanimous annotations, and that it processes each sentence in isolation, without leveraging additional context that might aid disambiguation. Upon close examination of their study, it becomes clear that they failed to realize a particular

caveat of the dataset. They considered the data segregation to be 50% agreement (4,846 sentences), 66% agreement (4,217 sentences), 75% agreement (3,453 sentences), and 100% unanimous agreement (2,264 sentences). But on close observation of the dataset description given by Malo et al. (2014)[10], the dataset is actually provided as >=50% agreement (4,846 sentences), >=66% agreement (4,217 sentences), >=75% agreement (3,453 sentences), and 100% unanimous agreement (2,264 sentences). Meaning that the sentences JayaShree Shetty et al. (2025)[9] considered to have 50% annotator agreement actually contained sentences having 50-100% annotator agreement. This also implies that the 66% annotator agreement sentences contained 66-100% annotator agreement and so on. Only the 100% annotator agreement sentences dataset contains sentences with only a single annotator agreement classification, which explains the inflated accuracy result of 64.49% compared to other classifications, since this is the only correctly reported result. This is a notable issue that needs to be addressed when using the Financial PhraseBank dataset segregations to avoid inconsistent results caused by repeated sentences.

A very unique experiment in Financial Sentiment Analysis conducted by Georgios Fatouros et. Al. (2023)[11] is worth considering. They annotated a dataset of 2291 unique news headlines which have an impact on one of 6 forex pair, to test OpenAI's GPT-3.5-Turbo and FinBERT's ability to predict financial sentiments correctly. In their experiment they created 6 unique prompt templates to test with GPT-3.5-Turbo. The unique aspect about their experiment is that the GPT-3.5-Turbo model's results scored better than FinBERT. This forms a basis for our assumption that the FinBERT model in the previously considered experiments performed better than much more powerful models because it was trained on the Financial PhraseBank dataset and then tested on the same which led to outstanding accuracy results. With this observation it should be kept in mind that if Financial Phrasebank dataset is being used for a Financial Sentiment Analysis task, FinBERT should not likely be used to test the same since it apparently already knows the answers in a way.

On observing several other studies and experiments, it has been observed that the only financially specialized model used for Financial Sentiment Analysis is FinBERT, and it has been compared with mostly GPT large language models. Only a select few existing journal articles like Qi, R. (2025)[12] have considered Llama for Financial Sentiment Analysis, that too, where an experiment has not been conducted and only a survey based on already existing information has been presented. This provides a good basis to assume that there is a gap in the research on General-Purpose Open-Source models like Llama and Mistral, when compared with Open-Source Finance-Specialized models other than FinBERT, for the task of Financial Sentiment Analysis.

3. Methodology

After conducting background research on Financial Sentiment Analysis using Large Language Models, some clear gaps were identified where new research could shed light on some trends and results. In this section, the aims and methodology for preparation and execution of the experiment will be discussed.

3.1 Aim

The primary objective of this experiment is to test the hypotheses discussed in the above sections. In this experiment, multiple prompting techniques will be implemented on data from varying datasets to conduct a comparative study across multiple levels, to gain an understanding of the parameters that affect the accuracy of a Large Language Model for the task of Financial Sentiment Analysis.

3.2 Dataset Selection

The initial step in setting up an experiment involving Financial Sentiment Analysis is deciding whether to create and annotate our own dataset or to select an existing and reliable dataset that aligns with the experiment's needs. The following datasets will be considered for the experiment, and the ones deemed necessary to test our hypotheses will be selected.

3.2.1 Financial PhraseBank

This is the most commonly used dataset for Financial Sentiment Analysis. This dataset contains a total of 4846 financial statements, classified into 3 labels: {Positive, Negative, Neutral}. This dataset was created in the study by Malo et al. (2014)[10]. This dataset is available in four possible configurations depending on the percentage of agreement of annotators:

- Number of instances with >=50% annotator agreement: 4846
- Number of instances with >=66% annotator agreement: 4217
- Number of instances with >=75% annotator agreement: 3453
- Number of instances with 100% annotator agreement: 2264

According to Malo et al. (2014)[10], the dataset was made out of English news on all listed companies in OMX Helsinki. The news was downloaded from the LexisNexis database using an automated web scraper. Out of this news database, a random subset of 10,000 articles was selected to obtain good coverage across small and large companies, companies in different industries, as well as different news sources. All sentences that did not contain any of the lexicon entities were excluded. This reduced the overall sample to 53,400 sentences, each containing at least one recognized lexicon entity. The sentences were then classified according to the types of entity sequences detected. Finally, a random sample of ~5000 sentences was chosen to represent the overall news database.

The selected collection of phrases was annotated by 16 people with adequate background knowledge on financial markets. Three of the annotators were researchers, and the remaining 13 were master's students at Aalto University School of Business, majoring primarily in finance, accounting, and economics.

3.2.2 SEntFiN 1.0

This is a large dataset created by Ankur Sinha et al. (2023)[13], comprising 10,753 human-annotated news headlines with entity-sentiment annotations, of which 2,847 headlines contain multiple entities, often with conflicting sentiments. They augment their dataset with a database of over 1,000 financial entities and their various representations in news media, amounting to over 5,000 phrases. They exclusively state that the entity database is particularly useful as its objective is to evaluate the economic value of sentiments, which requires mapping sentiments to entities and constructing sentiment indices.

The authors of the dataset stated that, in total, 10,753 headlines were sampled from all financial and economic news published between 2011 and 2015 by the Indian business news provider The Economic Times[14]. The annotation scheme and guidelines were prepared by one of the primary authors, who also acted as the curator of the annotation process. For the annotation process, they identified three human annotators — students pursuing a master's degree in business administration and management and with a strong academic performance in courses relevant to financial markets and macroeconomics. These three human annotators independently approached the curator about their interest in the annotation process, and the curator ensured that the annotators were not informed of the other annotators' presence until the final dataset was prepared. Each of the three annotators was provided with 10,753 news headlines, in which the entities and their entity spans were identified by the curator (and validated by the other authors). The annotators were asked to assign a sentiment class — positive, negative, or neutral — to the entities identified in the news headlines.

In order to reduce ambiguity during the annotation process, a sample of 100 annotated headlines, and list of 15 typical ambiguous cases with the rationale for the final recommendation was provided to the annotators as a part of training. The cues for the sources and reasons for ambiguity were derived from the observations in (Malo et al., 2014). After each news headline was annotated by each annotator, and prior to the consensus and final decision-making process, the average inter-annotator agreements for each pair of classes were calculated. The authors of the dataset observed high agreement levels from the tests conducted.

This information has been mentioned clearly so as to take into consideration every aspect of the dataset and its origins for the ease of explaining the nature and behaviour of the results obtained by the use of this dataset.

3.2.3 Financial Opinion Mining and Question Answering (FiQA)

This dataset was created as part of an open challenge by Macedo Maia et al. (2018)[15]. This dataset was provided to participants to complete the first task of this challenge. They were given a text instance in the financial domain (a microblog message, news statement, or headline) in English to detect the target aspects mentioned in the text (from a predefined list of aspect classes) and to predict the sentiment score for each of the mentioned targets. Sentiment scores were to be defined using continuous numeric values ranging from -1 (negative) to 1 (positive).

The datasets included two types of discourse — financial news headlines and financial microblogs — with manually annotated target entities, sentiment scores, and aspects. The financial news headlines dataset contains a total of 529 annotated headline samples (436 samples for the training set and 93 samples for the test set), while the financial microblogs contained a total of 774 annotated post samples (675 samples for the training set and 99 samples for the test set).

Using the entire dataset (test, train, and validation), there are a total of 499 news headline samples with labels that are usable for our experiment after omitting the microblogs.

3.3 Large Language Model Selection

For our experiment, we will be using two types of models: general-purpose LLMs, which have not been specialized for any particular domain, and a finance-specialized LLM. This is to determine the difference in performance on financial tasks to measure the advantage gained by a particular type. For this particular experiment, I am looking to work with a relatively small-sized model with approximately 3 billion parameters that can run comfortably on compatible consumer hardware. In background research, I have learned that various experiments have been conducted on multiple versions of OpenAI's GPT models and also on FinBERT. Since, in the above section, I am considering the Financial PhraseBank dataset, and in past studies I have observed that FinBERT gives overfitted results due to its training on the same dataset, it would only be fair to use other open-source generative LLMs specialized in financial tasks. For the general-purpose LLM, the requirement could very well be an open-source model like Llama or Mistral to compare results published in other studies where larger GPT models have been used to assess the exact accuracy advantage provided by differences in model size and computing power. After careful consideration, the following models were selected for the experiments:

3.3.1 Fin-R1

The hf.co/Mungert/Fin-R1-GGUF:latest model is fine-tuned for complex financial reasoning, analysis, and decision-making tasks with the following specs:

- Base Architecture: Qwen
- Parameters: 7B
- File Size: 4.7GB
- Model Type: Generative
- Licencing: Open Source
- Modifications: Fine-tuned for financial reasoning, risk assessment, and numerical analysis

3.3.2 Llama 3.2

The huihui_ai/llama3.2-abliterate:latest model is an uncensored variant of Meta's standard Llama 3.2 Instruct model, modified using a technique called abliteration to remove built-in safety alignment vectors for minimal denials, with the following specs:

- Base Architecture: Llama 3.2 (Instruct-tuned)
- Parameters: 3.24B
- File Size: 2.2GB
- Model Type: Generative
- Licencing: Open Source
- Modifications: The safety and censorship mechanisms have been systematically bypassed, allowing the model to answer controversial or restricted prompts without refusal

3.4 Dataset Preprocessing

To conduct the experiment, the data must be in a particular format. To bring the datasets into the required format—a 2-column CSV file with columns (statement, sentiment)—I will use Python scripts to process the 3 selected datasets, the algorithms for which will be discussed in this section.

3.4.1 Financial PhraseBank

The Financial PhraseBank files were made available as 4 text files: Sentences_50Agree.txt, Sentences_66Agree.txt, Sentences_75Agree.txt, Sentences_AllAgree.txt, corresponding to the Annotator agreements in which they are available. As observed in the background research, to have a purely range-based annotator agreement, I wrote a script that accepts all 4 files with a primary objective. Firstly, convert all four text files to comma-separated value files (.csv).

Next is to check the newly formed CSV files. We check Sentences_50Agree.csv for all the rows present in Sentences_66Agree.csv and remove them. This makes the Sentences_50Agree.csv file contain only the sentences which have an Annotator Agreement ranging from 50% to 65%. We consecutively do the same process for the remaining files. The Sentences_66Agree.csv file is checked for rows that match rows in sentences_75Agree.csv, and those rows are

removed, leaving sentences with a 66 - 74% annotator agreement. The Sentences_75Agree.csv file is checked for rows that match rows in sentences_AllAgree.csv, and those rows are removed, leaving sentences with a 75-99% annotator agreement. The Sentences_AllAgree.csv file is left as is.

With this process we have 4 files with ranged annotator agreement:

- Sentences_50Agree.csv: 627 rows (50-65% Annotator Agreement)
- Sentences_66Agree.csv: 763 rows (66-74% Annotator Agreement)
- Sentences_75Agree.csv: 1189 rows (75-99% Annotator Agreement)
- Sentences_AllAgree.csv: 2264 rows (100% Annotator Agreement)

3.4.2 SEntFiN 1.0

This is a slightly complex dataset containing sentences with multiple entities that exhibit conflicting financial sentiments. To use this dataset in my experiment, I will use only those rows that feature sentiment for a single entity. This dataset was already available as a CSV file. So, I have used a Python script that accepts a CSV file and converts it to the required format. To do this, firstly, the 'Words' and 'Serial No.' columns are dropped. Using the help of a simple parser function, which checks a regex of the 'Sentiment' column, we check for those rows with sentiment decisions for multiple entities and drop them. With this, my dataset is ready for use.

3.4.3 Financial Opinion Mining and Question Answering (FiQA)

This is a slightly tricky dataset since it was available in the form of 3 files, namely Train, Test, and Valid, with a (.parquet) file extension. A Python script was used to firstly extract the contents of these files and add them to a Pandas dataframe with a conditional statement. The files were in a 3-column format. The first column contained text; the second column contained a number between -1 and 1, which was the sentiment measure. 0 being neutral, a number between 0 and 1 (1 inclusive) being positive, and -1 and 0 (-1 inclusive) being negative. There was a third column that classified the rows as headline or blog. The Python script was programmed to only accept rows classified as headlines and omit the rest. Once this was done, the third row was dropped. Post which, the Sentiment decimal numbers were converted into strings equivalent to either Negative, Positive, or Neutral based on the number present in that column previously. With this process, the FiQA dataset is now ready for use in this experiment with 498 rows of data.

3.5 Templates for Prompting Techniques

To run the experiment, a Python script will be developed that accesses the pre-processed dataset files, extracts each statement row by row, and passes it to the Language Model after inserting it into the template for a prompting technique that includes a placeholder for the dataset. Below are the templates that will be used for the selected prompting techniques.

3.5.1 Zero-Shot

“””You are a financial sentiment analyst. Classify the sentiment of the following financial news text as 'positive', 'negative', or 'neutral'.
Text: {Placeholder for Statement}
Sentiment:”””””

3.5.2 Few-Shot

“””You are a financial sentiment analyst. Classify the sentiment of the following financial news text as 'positive', 'negative', or 'neutral'.

Examples:
Text: "The company reported record profits this quarter."
Sentiment: positive

Text: "The stock market crashed due to inflation fears."
Sentiment: negative

Text: "The board announced a routine quarterly review."
Sentiment: neutral

Now classify this text:
Text: { Placeholder for Statement }
Sentiment:"""

3.5.3 Self-Consistency

This Prompting technique uses the same template as Chain of Thought.

3.5.4 Chain of Thought

“””You are a financial sentiment analyst. Think step-by-step about the sentiment of the following financial news text. Then, classify it as 'positive', 'negative', or 'neutral'.

Text: { Placeholder for Statement }

Step 1: Identify the key financial entities and actions in this text.
Step 2: Determine if the overall tone is favourable, unfavourable, or neutral.
Step 3: Classify the sentiment.

Sentiment:"""

3.5.5 Annotators’ Instruction (Novel Method)

“””You are a financial sentiment analyst. You have been given the following instructions from the dataset annotators:

Instructions for annotators:

- Sentiment is the opinion or feeling expressed in the text towards the financial entity.
- Positive: The text expresses a favourable or optimistic view.
- Negative: The text expresses an unfavourable or pessimistic view.
- Neutral: The text expresses no clear opinion or contains only factual information.

Now, carefully analyse this text and classify its sentiment as 'positive', 'negative', or 'neutral'.

Text: { Placeholder for Statement }

Sentiment:"""

3.6 Standard Output Format for Experiment

The output of the tests conducted is in the format of a CSV file with the following columns:

- Statement: Pasted directly from the dataset.
- True Sentiment: The sentiment given in the dataset.
- Predicted Sentiment: The final sentiment predicted by the LLM and parsed by the Python script from the generated raw response.
- Raw Response: The unparsed response generated by the LLM.
- Technique: The prompting technique used for that iteration.

3.7 Evaluation Metrics

To accurately understand the test results and the performance of the prompting techniques and Large Language Models on the datasets in question, we need to calculate certain means, averages and conduct certain tests which will be explained in this section.

3.7.1 Precision

Precision is basically the accuracy achieved in a test. It answers the big question: Out of all the sentences the LLM claimed were Positive, Negative, or Neutral, how many actually were?

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Where:
TP (True Positive): a correct prediction
FP (False Positive): a false alarm (incorrectly predicting a class when it isn't)

FN (False Negative): a missed detection (falsely predicting another class when it actually is)

3.7.2 Recall

This is the precision of individual classification of the data. Basically, how many positive sentiments were guessed correctly, how many negative sentiments were guessed correctly, and how many neutral sentiments were guessed correctly?

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

3.7.3 F1-Score

The F1 score is the harmonic mean of Precision and Recall. It gives an overall score that lies between 0 and 1, with 1 being the best score.

$$\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

3.7.4 Macro F1-Score

This is an evaluation metric for multi-class classification models. Unlike the Weighted F1-Score, the Macro F1-Score treats all classes with equal importance, regardless of how many examples belong to each class. It computes the unweighted arithmetic mean of the individual F1-scores across all classes:

$$\text{F1}_{\text{macro}} = \frac{1}{K} \sum_{i=1}^{K} \text{F1}_i$$

Where:
K: the total number of sentiment classes (K = 3: Positive, Negative, Neutral).
$\mathbf{F1_i}$ is the individual F1-score for class i.

Because financial sentiment datasets (such as the Financial PhraseBank) are heavily imbalanced toward the "Neutral" class, evaluating models solely on accuracy or Weighted F1 can mask poor predictive performance on the minority classes (Positive and Negative). Reporting the Macro F1-Score ensures that an LLM cannot achieve a high score simply by over-predicting the majority class; it penalizes models that fail to correctly identify sparse but critically important positive and negative market signals.

Macro F1 answers: "Is the model actually smart, or is it just lazy and guessing 'Neutral' most of the time?"

3.7.5 Weighted F1-Score

The Weighted F1-Score is an evaluation metric for multi-class classification tasks that accounts for class imbalance in the dataset. In a multi-class setting, the Weighted F1-Score computes the individual F1-Score for each distinct class ($F1_i$)

and calculates an overall average weighted by each class's support—the total number of true instances (N_i) belonging to that class relative to the total dataset size (N_{total}):

$$\mathrm{F1}_{\text{weighted}} = \sum_{i=1}^{k} \left(\frac{N_i}{N_{\text{total}}} \times \mathrm{F1}_i \right)$$

Reporting the Weighted F1-Score ensures that the performance metric reflects the model's expected accuracy under realistic class proportions. In this study, it is evaluated alongside the Macro F1-Score (which weights all classes equally) to ensure the model achieves strong predictive power on minority sentiment classes rather than relying solely on high performance within the majority class.

Weighted F1 answers: "How well does the model perform overall across realistic financial text distributions?"

3.7.6 Confusion Matrix

While metrics like the Macro and Weighted F1-Scores provide a numerical summary of overall performance, they do not reveal the specific nature of a model's errors. To address this, a Confusion Matrix is used as a visual evaluation tool.

For this study's three-class classification task, the confusion matrix is formatted as a 3 X 3 grid. The rows represent the true, human-annotated sentiment labels, while the columns represent the LLM's predicted labels. The diagonal cells contain the correct predictions (True Positives for each class), whereas all off-diagonal cells represent specific misclassifications (e.g., an actual "Negative" sentence falsely predicted as "Neutral").

Methodological Necessity: The confusion matrix visually exposes whether the model is genuinely learning to identify the minority classes or simply defaulting to the dataset's majority class when uncertain.

3.7.7 Support

This is the number of instances in the dataset for a particular class in multi-class classification models. Like, if for the FiQA dataset Support_Positive is 320, that means out of the total number of rows in the dataset, 320 statements have their labelled sentiment as positive.

3.7.8 Token Efficiency

Token efficiency, defined as the proportion of output tokens relative to total tokens. This is a measure of the model's conciseness.

$$\text{Token Efficiency (\%)} = \left(\frac{\text{Output Tokens}}{\text{Input Tokens} + \text{Output Tokens}} \right) \times 100$$

3.7.9 McNemar's Test

Only this statistical test will be conducted on the results of this experiment. McNemar's test is specifically designed for paired nominal data, making it ideal for comparing two classifiers (e.g., Hybrid vs. Few-Shot) on the same test samples. It focuses only on the disagreements between the two classifiers, ignoring cases where both agree (whether correct or incorrect).

$$\chi^2 = \frac{(|b - c| - 1)^2}{b + c}$$

Where:
b = Number of samples where Technique A is correct and Technique B is incorrect
c = Number of samples where Technique B is correct and Technique A is incorrect
The -1 is the continuity correction.
This follows a chi-square distribution with 1 degree of freedom.

Once the chi-square is translated to a p-value, the significance threshold is $p < 0.005$, i.e. $\chi^2 > 3.84$ for 1 degree of freedom.

4. The Experiment

The experiment is conducted in 2 phases. A shorter mini-experiment is conducted on a fraction of the processed data to meet our requirements for observing the performance of the standard prompting techniques and our novel Annotators' Instruction method. Based on the performance, necessary changes will be made to eliminate underperforming techniques. Measures will be taken to boost the performance of our novel method. After these changes, the full experiment will be conducted on a selection of complete datasets.

4.1 Mini-Experiment

For this stage of the experiment, the performance of all models was considerably low due to the low limit set for output tokens. This was done to observe which techniques give straight raw responses and which ones ramble. Few-Shot achieved the highest accuracy, followed by zero-shot, Annotators' Instruction, and Chain of Thought.

Based on these results, Chain of Thought was eliminated from the main experiment due to its performance. This decision is supported by the study conducted by Anmol Singhal et al. (2025)[16], in which they tested Zero-Shot, Few-Shot, Chain of Thought, and two other variants of Chain of Thought on Meta's Llama 3 model, where Chain of Thought performed the lowest among all techniques. They conducted this test on the Financial News Sentiment Prediction Dataset (FNSPID)[17], which comprises thousands of labelled news articles from 2012-2023 covering S&P 500 companies.

These experiment results also prove that there is a need to change the approach in our Annotators' Instruction novel method.

4.2 Improvisations for Main Experiment

There were many observations from the mini experiment that helped in building a stronger Main Experiment. I will discuss those changes in this section.

4.2.1 Replacement for Chain of Thought

As seen in the previous section, Chain of Thought performed poorly compared to all other prompting techniques, with even our novel method (Annotators' Instruction) achieving better accuracy. Looking at this occurrence, it has been decided to replace Chain of Thought with Self-Consistency, since it forces the model to generate several different Chain of Thought responses.

4.2.2 Increase in Output Token Limit

In the mini experiment, it was proved that the output token limit was too low for the LLM to generate its complete reasoning and reach a conclusion. After running a few tests, the output token limit was set to 1000 after observing an average count across all techniques. The limit is set to avoid models from continually rambling on a complex statement. If such an event occurs, the experiment script will interrupt the model and mark the sentiment as 'Unknown'.

4.2.3 Hybridization of Novel Method

As seen in the mini experiment, the Annotators' Agreement prompting technique did not perform at an acceptable level even though it outperformed Chain of Thought. Since Few-Shot was the winner, to improve the accuracy of this technique, I will combine the two methods into a hybrid approach to test the performance boost this move offers. The following is the updated template for the Annotators' Instruction Hybrid Technique:

"""You are a financial sentiment analyst. You have been given the following instructions from the dataset annotators:

Instructions for annotators:
- Sentiment is the opinion or feeling expressed in the text towards the financial entity.
- Positive: The text expresses a favourable or optimistic view.
- Negative: The text expresses an unfavourable or pessimistic view.
- Neutral: The text expresses no clear opinion or contains only factual information.

Examples:
Text: "The company reported record profits this quarter."
Sentiment: positive

Text: "The stock market crashed due to inflation fears."
Sentiment: negative

Text: "The board announced a routine quarterly review."
Sentiment: neutral

Now, carefully analyse this text and classify its sentiment as 'positive', 'negative', or 'neutral' using the instructions above.
Text: {text}
Sentiment:"""

4.2.4 Selection of Final Datasets

After careful consideration of the hypotheses stated in this study, it has been decided to conduct the main experiment on the following preprocessed datasets:

- Sentences_50Agree.csv: 628 rows (50-65% Annotator Agreement)
- Sentences_75Agree.csv: 1190 rows (75-99% Annotator Agreement)
- Sentences_66Agree.csv: 763 rows (66-54% Annotator Agreement)
    - To be run only on Llama-3.2-abliterate model
- Sentences_AllAgree.csv: 2264 rows (100% Annotator Agreement)
    - To be run only on Llama-3.2-abliterate model
- FiQA : 498 rows of Financial News Headlines

If the use of these datasets for the experiment results in ambiguous or findings that are questionable in quality, the remaining datasets will be used to obtain conclusive results.

5. Experiment Results

A total of 24 tests were conducted using 4 prompting techniques, 3 pre-processed datasets and 2 large language models. The tests' performance will be analysed on 5 levels:

- Individually based
- Prompting Technique based
- Dataset based
- Large Language Model based
- Overall

To analyse the raw data of these tests, please refer to the Confusion Matrices for these tests in Appendix B.

5.1 Individual Performance Results

In this section, I will analyse the test results without any grouping. Each result represents a test conducted on one LLM, using one prompting technique with 1 selected dataset.

5.1.1 Fin-R1-GGUF | FiQA | Few-Shot

- **Accuracy:** 0.673
- **Macro-F1:** 0.560
- **Weighted-F1:** 0.760
- **Token Efficiency:** 81.55

Metric/Label	Positive	Negative	Neutral
Precision	0.913	0.935	0.062
Support	320	165	13
Recall	0.656	0.703	0.692
F1	0.764	0.803	0.115

5.1.2 Fin-R1-GGUF | FiQA | Hybrid AIAP

- **Accuracy:** 0.558
- **Macro-F1:** 0.498
- **Weighted-F1:** 0.670
- **Token Efficiency:** 70.44

Metric/Label	Positive	Negative	Neutral
Precision	0.898	0.934	0.536
Support	320	165	13
Recall	0.525	0.6	0.846
F1	0.663	0.731	0.101

5.1.3 Fin-R1-GGUF | FiQA | Self-consistency

- **Accuracy:** 0.663
- **Macro-F1:** 0.568
- **Weighted-F1:** 0.755
- **Token Efficiency:** 90.77

Metric/Label	Positive	Negative	Neutral
Precision	0.926	0.959	0.755
Support	320	165	13
Recall	0.628	0.709	0.923
F1	0.749	0.815	0.140

5.1.4 Fin-R1-GGUF | FiQA | Zero-Shot

- **Accuracy:** 0.645
- **Macro-F1:** 0.575
- **Weighted-F1:** 0.764
- **Token Efficiency:** 91.04

Metric/Label	Positive	Negative	Neutral
Precision	0.908	0.942	0.08
Support	320	165	13
Recall	0.675	0.685	0.923
F1	0.774	0.793	0.157

5.1.5 Fin-R1-GGUF | FP-50 | Few-Shot

- **Accuracy:** 0.506
- **Macro-F1:** 0.544
- **Weighted-F1:** 0.498
- **Token Efficiency:** 79.68

Metric/Label	Positive	Negative	Neutral
Precision	0.402	0.667	0.602
Support	195	90	342
Recall	0.687	0.689	0.354
F1	0.508	0.678	0.446

5.1.6 Fin-R1-GGUF | FP-50 | Hybrid AIAP

- **Accuracy:** 0.517
- **Macro-F1:** 0.536
- **Weighted-F1:** 0.522
- **Token Efficiency:** 67.65

Metric/Label	Positive	Negative	Neutral
Precision	0.398	0.721	0.570
Support	195	90	342
Recall	0.518	0.544	0.509
F1	0.450	0.620	0.538

5.1.7 Fin-R1-GGUF | FP-50 | Self-Consistency

- **Accuracy:** 0.531
- **Macro-F1:** 0.580
- **Weighted-F1:** 0.524
- **Token Efficiency:** 89.63

Metric/Label	Positive	Negative	Neutral
Precision	0.410	0.747	0.646
Support	195	90	342
Recall	0.723	0.756	0.362
F1	0.523	0.751	0.464

5.1.8 Fin-R1-GGUF | FP-50 | Zero-Shot

- **Accuracy:** 0.526
- **Macro-F1:** 0.568
- **Weighted-F1:** 0.522
- **Token Efficiency:** 91.25

Metric/Label	Positive	Negative	Neutral
Precision	0.408	0.744	0.645
Support	195	90	342
Recall	0.728	0.678	0.371
F1	0.523	0.709	0.471

5.1.9 Fin-R1-GGUF | FP-75 | Few-Shot

- **Accuracy:** 0.723
- **Macro-F1:** 0.733
- **Weighted-F1:** 0.729
- **Token Efficiency:** 79.29

Metric/Label	Positive	Negative	Neutral
Precision	0.539	0.703	0.922
Support	317	117	755
Recall	0.908	0.872	0.622
F1	0.677	0.779	0.743

5.1.10 Fin-R1-GGUF | FP-75 | Hybrid AIAP

- **Accuracy:** 0.761
- **Macro-F1:** 0.746
- **Weighted-F1:** 0.764
- **Token Efficiency:** 67.80

Metric/Label	Positive	Negative	Neutral
Precision	0.617	0.752	0.841
Support	317	117	755
Recall	0.732	0.778	0.771
F1	0.670	0.765	0.804

5.1.11 Fin-R1-GGUF | FP-75 | Self-Consistency

- **Accuracy:** 0.727
- **Macro-F1:** 0.738
- **Weighted-F1:** 0.731
- **Token Efficiency:** 89.87

Metric/Label	Positive	Negative	Neutral
Precision	0.544	0.707	0.928
Support	317	117	755
Recall	0.924	0.889	0.618
F1	0.684	0.788	0.742

5.1.12 Fin-R1-GGUF | FP-75 | Zero-Shot

- **Accuracy:** 0.706
- **Macro-F1:** 0.716
- **Weighted-F1:** 0.712
- **Token Efficiency:** 89.91

Metric/Label	Positive	Negative	Neutral
Precision	0.520	0.68	0.919
Support	317	117	755
Recall	0.896	0.872	0.6
F1	0.658	0.764	0.726

5.1.13 Llama 3.2 - Abliterate | FiQA | Few-Shot

- **Accuracy:** 0.753
- **Macro-F1:** 0.597

- **Weighted-F1:** 0.800
- **Token Efficiency:** 45.29

Metric/Label	Positive	Negative	Neutral
Precision	0.940	0.804	0.095
Support	320	165	13
Recall	0.738	0.794	0.615
F1	0.826	0.799	0.165

5.1.14 Llama 3.2 - Abliterate | FiQA | Hybrid AIAP

- **Accuracy:** 0.546
- **Macro-F1:** 0.493
- **Weighted-F1:** 0.674
- **Token Efficiency:** 36.84

Metric/Label	Positive	Negative	Neutral
Precision	0.982	0.922	0.049
Support	320	165	13
Recall	0.522	0.570	0.846
F1	0.682	0.704	0.092

5.1.15 Llama 3.2 - Abliterate | FiQA | Self-Consistency

- **Accuracy:** 0.717
- **Macro-F1:** 0.586
- **Weighted-F1:** 0.769
- **Token Efficiency:** 65.87

Metric/Label	Positive	Negative	Neutral
Precision	0.950	0.781	0.099
Support	320	165	13
Recall	0.65	0.842	0.769
F1	0.772	0.810	0.175

5.1.16 Llama 3.2 - Abliterate | FiQA | Zero-Shot

- **Accuracy:** 0.715
- **Macro-F1:** 0.589
- **Weighted-F1:** 0.767
- **Token Efficiency:** 65.53

Metric/Label	Positive	Negative	Neutral
Precision	0.950	0.780	0.107
Support	320	165	13

Recall	0.647	0.836	0.846
F1	0.770	0.807	0.190

5.1.17 Llama 3.2 - Abliterate | FP-50 | Few-Shot

- **Accuracy:** 0.534
- **Macro-F1:** 0.576
- **Weighted-F1:** 0.531
- **Token Efficiency:** 46.02

Metric/Label	Positive	Negative	Neutral
Precision	0.42	0.658	0.644
Support	195	90	342
Recall	0.646	0.811	0.398
F1	0.509	0.726	0.492

5.1.18 Llama 3.2 - Abliterate | FP-50 | Hybrid AIAP

- **Accuracy:** 0.512
- **Macro-F1:** 0.499
- **Weighted-F1:** 0.512
- **Token Efficiency:** 36.72

Metric/Label	Positive	Negative	Neutral
Precision	0.4	0.706	0.551
Support	195	90	342
Recall	0.441	0.4	0.582
F1	0.420	0.511	0.566

5.1.19 Llama 3.2 - Abliterate | FP-50 | Self-Consistency

- **Accuracy:** 0.536
- **Macro-F1:** 0.568
- **Weighted-F1:** 0.534
- **Token Efficiency:** 66.08

Metric/Label	Positive	Negative	Neutral
Precision	0.421	0.612	0.623
Support	195	90	342
Recall	0.564	0.822	0.444
F1	0.482	0.701	0.519

5.1.20 Llama 3.2 - Abliterate | FP-50 | Zero-Shot

- **Accuracy:** 0.533
- **Macro-F1:** 0.565
- **Weighted-F1:** 0.531
- **Token Efficiency:** 66.09

Metric/Label	Positive	Negative	Neutral
Precision	0.421	0.615	0.620
Support	195	90	342
Recall	0.574	0.8	0.439
F1	0.486	0.696	0.514

5.1.21 Llama 3.2 - Abliterate | FP-66 | Few-Shot

- **Accuracy:** 0.638
- **Macro-F1:** 0.663
- **Weighted-F1:** 0.631
- **Token Efficiency:** 42.45

Metric/Label	Positive	Negative	Neutral
Precision	0.591	0.630	0.702
Support	281	94	388
Recall	0.726	0.926	0.505
F1	0.652	0.75	0.588

5.1.22 Llama 3.2 - Abliterate | FP-66 | Hybrid AIAP

- **Accuracy:** 0.624
- **Macro-F1:** 0.628
- **Weighted-F1:** 0.623
- **Token Efficiency:** 36.26

Metric/Label	Positive	Negative	Neutral
Precision	0.601	0.686	0.626
Support	281	94	388
Recall	0.573	0.628	0.660
F1	0.586	0.656	0.642

5.1.23 Llama 3.2 - Abliterate | FP-66 | Self-Consistency

- **Accuracy:** 0.630
- **Macro-F1:** 0.655
- **Weighted-F1:** 0.622
- **Token Efficiency:** 61.76

Metric/Label	Positive	Negative	Neutral
Precision	0.588	0.601	0.703
Support	281	94	388
Recall	0.712	0.979	0.487
F1	0.644	0.745	0.575

5.1.24 Llama 3.2 - Abliterate | FP-66 | Zero-Shot

- **Accuracy:** 0.628
- **Macro-F1:** 0.652
- **Weighted-F1:** 0.619
- **Token Efficiency:** 61.60

Metric/Label	Positive	Negative	Neutral
Precision	0.587	0.597	0.699
Support	281	94	388
Recall	0.708	0.979	0.484
F1	0.642	0.742	0.572

5.1.25 Llama 3.2 - Abliterate | FP-75 | Few-Shot

- **Accuracy:** 0.717
- **Macro-F1:** 0.712
- **Weighted-F1:** 0.724
- **Token Efficiency:** 45.48

Metric/Label	Positive	Negative	Neutral
Precision	0.559	0.602	0.887
Support	317	117	755
Recall	0.801	0.906	0.653
F1	0.659	0.724	0.752

5.1.26 Llama 3.2 - Abliterate | FP-75 | Hybrid AIAP

- **Accuracy:** 0.743
- **Macro-F1:** 0.696
- **Weighted-F1:** 0.743
- **Token Efficiency:** 36.46

Metric/Label	Positive	Negative	Neutral
Precision	0.609	0.774	0.802
Support	317	117	755
Recall	0.678	0.556	0.799
F1	0.642	0.647	0.800

5.1.27 Llama 3.2 - Abliterate | FP-75 | Self-Consistency

- **Accuracy:** 0.741
- **Macro-F1:** 0.726
- **Weighted-F1:** 0.747
- **Token Efficiency:** 65.81

Metric/Label	Positive	Negative	Neutral
Precision	0.604	0.582	0.888
Support	317	117	755
Recall	0.779	0.940	0.694
F1	0.680	0.719	0.780

5.1.28 Llama 3.2 - Abliterate | FP-75 | Zero-Shot

- **Accuracy:** 0.738
- **Macro-F1:** 0.725
- **Weighted-F1:** 0.745
- **Token Efficiency:** 65.85

Metric/Label	Positive	Negative	Neutral
Precision	0.602	0.582	0.887
Support	317	117	755
Recall	0.782	0.940	0.689
F1	0.680	0.719	0.776

5.1.29 Llama 3.2 - Abliterate | FP-100 | Few-Shot

- **Accuracy:** 0.874
- **Macro-F1:** 0.865
- **Weighted-F1:** 0.876
- **Token Efficiency:** 41.60

Metric/Label	Positive	Negative	Neutral
Precision	0.784	0.814	0.936
Support	570	303	1391
Recall	0.828	0.980	0.870
F1	0.805	0.889	0.902

5.1.30 Llama 3.2 - Abliterate | FP-100 | Hybrid AIAP

- **Accuracy:** 0.842
- **Macro-F1:** 0.814
- **Weighted-F1:** 0.837
- **Token Efficiency:** 33.76

Metric/Label	Positive	Negative	Neutral
Precision	0.834	0.891	0.836
Support	570	303	1391
Recall	0.637	0.785	0.939
F1	0.722	0.835	0.884

5.1.31 Llama 3.2 - Abliterate | FP-100 | Self-Consistency

- **Accuracy:** 0.886
- **Macro-F1:** 0.878
- **Weighted-F1:** 0.889
- **Token Efficiency:** 60.71

Metric/Label	Positive	Negative	Neutral
Precision	0.808	0.795	0.958
Support	570	303	1391
Recall	0.893	0.983	0.863
F1	0.848	0.879	0.908

5.1.32 Llama 3.2 - Abliterate | FP-100 | Zero-Shot

- **Accuracy:** 0.887
- **Macro-F1:** 0.880
- **Weighted-F1:** 0.890
- **Token Efficiency:** 60.67

Metric/Label	Positive	Negative	Neutral
Precision	0.801	0.799	0.962
Support	570	303	1391
Recall	0.904	0.983	0.860
F1	0.849	0.82	0.908

5.2 Prompting Technique Performance Results

In this section, I will analyse the test results grouped by prompting technique. Each result represents the performance of one distinct prompting technique across all individual tests in which it was used.

5.2.1 Few-Shot

- **Accuracy:** 0.724
- **Macro-F1:** 0.737
- **Weighted-F1:** 0.725
- **Token Efficiency:** 63.52

Metric/Label	Positive	Negative	Neutral
Precision	0.631	0.741	0.806
Support	2515	1141	3999
Recall	0.765	0.854	0.661
F1	0.692	0.793	0.726

5.2.2 Hybrid AIAP

- **Accuracy:** 0.701
- **Macro-F1:** 0.695
- **Weighted-F1:** 0.699
- **Token Efficiency:** 50.87

Metric/Label	Positive	Negative	Neutral
Precision	0.661	0.826	0.696
Support	2515	1141	3999
Recall	0.594	0.641	0.786
F1	0.626	0.722	0.738

5.2.3 Self-Consistency

- **Accuracy:** 0.730
- **Macro-F1:** 0.742
- **Weighted-F1:** 0.730
- **Token Efficiency:** 79.71

Metric/Label	Positive	Negative	Neutral
Precision	0.645	0.728	0.809
Support	2515	1141	3999
Recall	0.759	0.878	0.670
F1	0.697	0.796	0.733

5.2.4 Zero-Shot

- **Accuracy:** 0.727
- **Macro-F1:** 0.738
- **Weighted-F1:** 0.728
- **Token Efficiency:** 80.38

Metric/Label	Positive	Negative	Neutral
Precision	0.639	0.724	0.812
Support	2515	1141	3999
Recall	0.765	0.864	0.664
F1	0.696	0.788	0.731

5.3 Dataset-Based Performance Results

In this section, I will analyse the test results grouped by datasets. Each result represents the test performance on one distinct dataset across all individual tests in which it was used.

5.3.1 FiQA

- **Accuracy:** 0.664
- **Macro-F1:** 0.557
- **Weighted-F1:** 0.747

Metric/Label	Positive	Negative	Neutral
Precision	0.932	0.867	0.723
Support	2560	1320	104
Recall	0.630	0.717	0.808
F1	0.752	0.785	0.133

5.3.2 FP-50

- **Accuracy:** 0.524
- **Macro-F1:** 0.559
- **Weighted-F1:** 0.526

Metric/Label	Positive	Negative	Neutral
Precision	0.410	0.674	0.606
Support	1560	720	2736
Recall	0.610	0.688	0.432
F1	0.490	0.681	0.504

5.3.3 FP-66

- **Accuracy:** 0.630
- **Macro-F1:** 0.652
- **Weighted-F1:** 0.626

Metric/Label	Positive	Negative	Neutral
Precision	0.591	0.621	0.676
Support	1124	376	1552
Recall	0.680	0.878	0.534
F1	0.632	0.728	0.597

5.3.4 FP-75

- **Accuracy:** 0.732
- **Macro-F1:** 0.725

- **Weighted-F1:** 0.738

Metric/Label	Positive	Negative	Neutral
Precision	0.569	0.658	0.878
Support	2536	936	6040
Recall	0.813	0.844	0.681
F1	0.669	0.739	0.767

5.3.5 FP-100

- **Accuracy:** 0.872
- **Macro-F1:** 0.861
- **Weighted-F1:** 0.874

Metric/Label	Positive	Negative	Neutral
Precision	0.805	0.820	0.918
Support	2280	1212	5564
Recall	0.815	0.933	0.883
F1	0.810	0.873	0.900

5.4 LLM-Based Performance Results

In this section, I will analyse the test results grouped by LLM. Each result represents the test performance on one distinct Large Language Model across all individual tests in which it was used.

5.4.1 Fin-R1-GGUF

- **Accuracy:** 0.654
- **Macro-F1:** 0.677
- **Weighted-F1:** 0.654

Metric/Label	Positive	Negative	Neutral
Precision	0.581	0.792	0.685
Support	3328	1488	4440
Recall	0.724	0.728	0.577
F1	0.645	0.759	0.626

5.4.2 Llama 3.2 Abliterate

- **Accuracy:** 0.749
- **Macro-F1:** 0.752
- **Weighted-F1:** 0.750

Metric/Label	Positive	Negative	Neutral

Precision	0.679	0.731	0.805
Support	6732	3076	11556
Recall	0.719	0.848	0.740
F1	0.698	0.785	0.771

5.5 Overall Performance Results

- **Accuracy:** 0.720
- **Macro-F1:** 0.730
- **Weighted-F1:** 0.722

Metric/Label	Positive	Negative	Neutral
Precision	0.643	0.748	0.774
Support	10060	4564	15996
Recall	0.720	0.809	0.695
F1	0.680	0.777	0.732

5.6 McNemar's Test Results - Statistical Significance

This statistical test is the verdict for any wins that are observable in the above sections. In this section, I will discuss only those tests that are significant to exploring the support for or against the hypotheses.

Model	Dataset	Technique_a	Technique_b	p_value	Significance
llama3.2_abliterate	FP-75	Few_Shot	Self_Consistency	0.043	TRUE
llama3.2_abliterate	FiQA	Hybrid_AIAP	Few_Shot	$2.9*10^{-19}$	TRUE
llama3.2_abliterate	FiQA	Hybrid_AIAP	Zero_Shot	$8.3*10^{-14}$	TRUE
llama3.2_abliterate	FiQA	Hybrid_AIAP	Self_Consistency	$2.7*10^{-13}$	TRUE
llama3.2_abliterate	FiQA	Few_Shot	Self_Consistency	0.049	TRUE
Fin-R1-GGUF	FP-75	Hybrid_AIAP	Few_Shot	0.012	TRUE
Fin-R1-GGUF	FP-75	Hybrid_AIAP	Self_Consistency	0.017	TRUE
Fin-R1-GGUF	FP-75	Hybrid_AIAP	Zero_Shot	0.0002	TRUE
Fin-R1-GGUF	FiQA	Zero_Shot	Hybrid_AIAP	$3.3*10^{-7}$	TRUE
Fin-R1-GGUF	FiQA	Self_Consistency	Hybrid_AIAP	$6.7*10^{-6}$	TRUE
Fin-R1-GGUF	FiQA	Few_Shot	Hybrid_AIAP	$3.2*10^{-7}$	TRUE

Dataset	Technique	Model_a	Model_b	p_value	Significance
FP-50	Self_Consistency	Fin-R1-GGUF	Llama-3.2-abliterate	0.886	FALSE
FP-50	Zero_Shot	Fin-R1-GGUF	Llama-3.2-abliterate	0.840	FALSE
FP-50	Hybrid_AIAP	Fin-R1-GGUF	Llama-3.2-abliterate	0.888	FALSE
FP-50	Few_Shot	Fin-R1-GGUF	Llama-3.2-abliterate	0.197	FALSE
FP-75	Self_Consistency	Fin-R1-GGUF	Llama-3.2-abliterate	0.363	FALSE
FP-75	Zero_Shot	Fin-R1-GGUF	Llama-3.2-abliterate	0.034	TRUE
FP-75	Hybrid_AIAP	Fin-R1-GGUF	Llama-3.2-abliterate	0.225	FALSE

FP-75	Few_Shot	Fin-R1-GGUF	Llama-3.2-abliterate	0.730	FALSE
FiQA	Self_Consistency	Fin-R1-GGUF	Llama-3.2-abliterate	0.034	TRUE
FiQA	Zero_Shot	Fin-R1-GGUF	Llama-3.2-abliterate	0.528	FALSE
FiQA	Hybrid_AIAP	Fin-R1-GGUF	Llama-3.2-abliterate	0.248	FALSE
FiQA	Few_Shot	Fin-R1-GGUF	Llama-3.2-abliterate	0.0002	TRUE

5.7 Empirical Findings vs. Initial Hypotheses

In this section, I will analyse the test results presented in the preceding sections to determine whether this experiment rejects the null hypothesis for all four alternative hypotheses defined in the Introduction section of this study.

5.7.1 Accuracy depends on Annotator Agreement.

To determine support for this alternative hypothesis, I will reference the test results for the FP-50 and FP-75 datasets to determine whether they are statistically significant.

Overall dataset-based performance findings state the following:

- **FP-50 (Accuracy):** 0.524 ~ 52.4%
- **FP-75 (Accuracy):** 0.732 ~ 73.2%

The overall dataset findings above show an approximate 21% boost in accuracy when annotator agreement increased from 50-65% to 75-99%. The alternative hypothesis is so far well supported. To further analyse the findings, I will examine dataset-level accuracy metrics, which will show how each prompting technique performed on this dataset, regardless of the LLM type.

The findings on prompting technique performance metrics per dataset state the following:

Technique/ Dataset	**Few-Shot**	**Hybrid-AIAP**	**Self-Consistency**	**Zero-Shot**
FP-50	0.520 ~ 52%	0.514 ~ 51.4%	0.533 ~ 53.3%	0.530 ~ 53%
FP-75	0.720 ~ 72%	0.752 ~ 75.2%	0.734 ~ 73.4%	0.722 ~ 72.2%
Difference	0.200 ~ 20%	0.238 ~ 23.8%	0.201 ~ 20.1%	0.192 ~ 19.2%

The above findings completely complement the overall dataset-based performance findings. These findings confirm that for every prompting technique, there is an increase in accuracy of approximately 20% when annotator agreement increased from 50-65% to 75-99%.

Given the above findings, since I have sufficient support for the proposed alternative hypothesis, I can correctly conclude that the null hypothesis is rejected.

5.7.2 A Financially Specialized LLM is better for FSA.

To determine support for this alternative hypothesis, I will refer to the test results on the performance of both LLMs to check for any statistical significance.

Overall LLM-based performance findings state the following:

- **Fin-R1-GGUF (Accuracy):** 0.654 ~ 65.4%
- **Llama-3.2-abliterate (Accuracy):** 0.749 ~ 74.9%

The overall LLM findings above show an accuracy advantage of approximately 9.5% when using the Llama-3.2-abliterate model compared to Fin-R1-GGUF (finance-specialized). These findings completely support the null hypothesis. To further analyse the findings, I will investigate LLM-level accuracy metrics, which will state how each LLM performed with every prompting technique, irrespective of the dataset.

Technique/ Model	Few-Shot	Hybrid-AIAP	Self-Consistency	Zero-Shot
Fin-R1-GGUF	0.653 ~ 65.3%	0.651 ~ 65.1%	0.660 ~ 66%	0.652 ~ 65.2%
Llama-3.2	0.754 ~ 75.4%	0.722 ~ 72.2%	0.760 ~ 76%	0.759 ~ 75.9%
Difference	0.101 ~ 10.1%	0.071 ~ 7.1%	0.100 ~ 10%	0.107 ~ 10.7%

The above findings completely complement the overall LLM-based performance findings. These findings suggest an advantage of approximately 10% for the Llama-3.2-abliterate model over Fin-R1-GGUF when using all prompting techniques.

The above findings lack sufficient support for the proposed alternative hypothesis; hence, the findings fail to reject the null hypothesis.

5.7.3 The annotators' instruction-assisted prompting can outperform Few-Shot.

To determine support for this alternative hypothesis, I will refer to the test results on the performance of the prompting techniques to check for any statistical significance.

Overall prompting technique-based performance findings state the following:

- **Hybrid AIAP (Accuracy):** 0.701 ~ 70.1%
- **Few-Shot (Accuracy):** 0.724 ~ 72.4%

The above findings on overall prompting techniques show that Few-Shot prompting has an accuracy advantage of approximately 2.3% over the novel method Hybrid_AIAP. These findings provide support for the null hypothesis with some margin. To further analyse the findings, I will examine dataset-level accuracy

metrics, which will show how each prompting technique performed across all datasets, irrespective of LLM type.

Technique/ Dataset	Few-Shot	Hybrid-AIAP	Self-Consistency	Zero-Shot
FP-50	0.520 ~ 52%	0.514 ~ 51.4%	0.533 ~ 53.3%	0.530 ~ 53%
FP-75	0.720 ~ 72%	0.752 ~ 75.2%	0.734 ~ 73.4%	0.722 ~ 72.2%
FiQA	0.713 ~ 71.3%	0.552 ~ 55.2%	0.690 ~ 69%	0.700 ~ 70%

The findings above complement the overall dataset findings. These findings suggest that, on the FP-50 dataset, the Few-Shot prompting technique has an accuracy advantage of 0.6% over the Hybrid-AIAP prompting technique. On the FP-75 dataset, the Hybrid-AIAP prompting technique achieves a 3.2% accuracy advantage over the Few-Shot prompting technique. On the FiQA dataset, Few-Shot prompting has an accuracy advantage of 16.1% over Hybrid-AIAP prompting.

Considering the above findings, and with insufficient support for the proposed alternative hypothesis, the findings fail to reject the null hypothesis.

5.7.4 3B-LLMs can compete with GPT-3.5-like models on FSA

To assess support for this alternative hypothesis, I will refer to studies that used GPT-based LLMs on the same or similar datasets and compare their performance results with those of our Llama-3.2-abliterate model.

In the background research, we discussed the study conducted by Y. Shen and P. K. Zhang (2024)[7], which tested Zero-Shot and Few-Shot prompting techniques using GPT-3.5-Turbo and GPT-4o models on the Financial Phrasebank dataset. To gather support for our alternative hypothesis, I have run the same tests on our Llama-3.2-abliterate general-purpose Large Language Model, which is, in terms of architecture, substantially inferior to the GPT models used by Y. Shen and P. K. Zhang (2024)[7].

The comparative analysis for the results of both our studies is presented below:

Technique/Model	GPT-3.5-Turbo	GPT-4o	Llama-3.2-abliterate
Zero-Shot	0.78 ~ 78%	0.85 ~ 85%	0.764 ~ 76.4%
Few-Shot	0.77 ~ 77%	0.86 ~ 86%	0.754 ~ 75.4%

The findings above show that using GPT-4o yields an accuracy advantage of almost 10% over Llama-3.2-abliterate. When comparing the performance of GPT-3.5-Turbo with that of Llama-3.2-abliterate, GPT-3.5-Turbo merely shows an accuracy advantage of 1.6% for both Zero-Shot and Few-Shot prompting techniques.

It was earlier established that GPT-3.5-Turbo is a huge LLM with an estimated 175B parameters and can only run via Cloud APIs, while Llama-3.2-

abliterate is a 3B-parameter model that can run on consumer hardware. Given the difference in the use of computational power, it garners immense support for the alternative hypothesis.

Considering the above findings, and with sufficient support for the proposed alternative hypothesis, I can correctly conclude that the null hypothesis is rejected.

6. Discussion

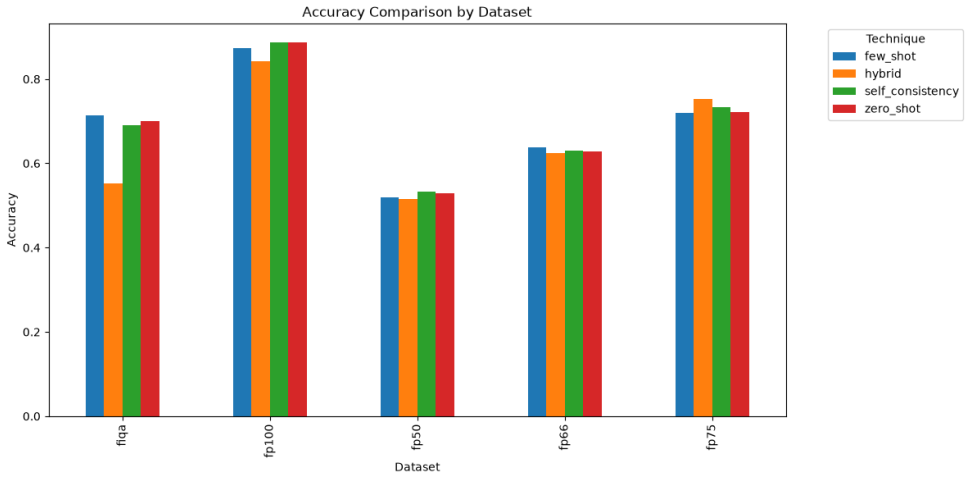
This project set out to evaluate and compare standard prompt engineering strategies against my novel, Annotator Instruction-guided approach for Financial Sentiment Classification. As shown in the experimental results, Self-Consistency prompting consistently outperformed the Human-Guided method across the evaluated datasets and models.

This section interprets these findings to understand the underlying reasons behind the models' behaviour. It begins by explaining why the models responded better to generating multiple reasoning paths and majority voting rather than complex annotator rules. This section will also present an extensive error analysis using the confusion matrices and evaluate the computational trade-offs of each method, concluding with an acknowledgement of the limitations of the current experimental setup.

6.1 Deciphering the Findings

In the introduction section of this study, I defined the hypotheses this study was meant to explore. This study has achieved much more than that. Let's discuss, one by one, the outcomes and behaviours observed while exploring the validity of each hypothesis.

The first hypothesis stated that "**Accuracy depends on Annotator Agreement**." In the results section, I used only the results of FP-50 and FP-75 to prove a clear pattern. To gather support for the fourth hypothesis, tests were also conducted on FP-66 and FP-100. Looking at all the combined results, the trend is clear. I would like to present it visually below.



Looking at the above graph, a clear pattern is visible. The accuracy for FP-50 sits at the lowest while increasing in an ascending manner with Annotator Agreement. It was seemingly logical to assume this behaviour, but it had to be put to the test. FP-50 dataset contains sentences with only 50-65% annotator agreement.

This means that out of the 16 annotators of this dataset, only 8-10 agree on the sentences in this dataset. Also, as mentioned by Malo et al. (2014)[10], the annotators had adequate background knowledge on financial markets; this only implies that the sentences in the data might have separate reasoning paths leading to different outcomes. To reduce speculation, let's analyse examples from the FP-50 and FP-100 datasets. There are these 2 examples in **FP-50**: -

"Compared with the FTSE 100 index, which rose 36.7 points (or 0.6 %) on the day, this was a relative price change of -0.2 %."

&

"Compared with the FTSE 100 index, which rose 51.5 points (or 0.9 %) on the day, this was a relative price change of -0.6 %."

Now, looking at both of these examples, we can see that in both sentences, the value of the asset in question increased. Although it underperformed relative to the increase in the overall market value. The only difference is in the first example: the market rose by 0.6%, but the asset's performance was 0.2% lower, which means the asset's actual return rose by 0.4%. Whereas in the second example, the market rose by 0.9%, but the asset's performance was 0.6% lower, so the asset's actual return rose by 0.3%.

Now there are three ways of looking at these examples. The simplest way, being that differences are extremely minute and are observable in the charts of countless assets on the market on a daily basis, hence the sentences could be labelled "neutral". Then there is the more critical view: albeit in both the examples there is a rise in the asset's value, there is more emphasis on the underperformance; hence, the sentences could be labelled "negative". A more reasonable and realistic annotator could beg to differ. In both sentences, the asset's return ultimately rose and hence could be labelled "positive", although this view might be a stretch since the sentences show clear emphasis on the underperformance.

Coming to the actual labels, the **first example was labelled "negative"** and the **second one was labelled "neutral"**. Hence, here comes the question of opinionated clarity and relative terms: "Where do we get to draw the line between negative and neutral?" Sentence one was labelled "negative" because the underperformance was 0.4%, while sentence two was labelled "neutral" since the underperformance was **barely** 0.3%. It is a matter of annotator opinion where we draw the line below which we can use the term **"barely".**

Now let us look at an example from **FP-100**: -

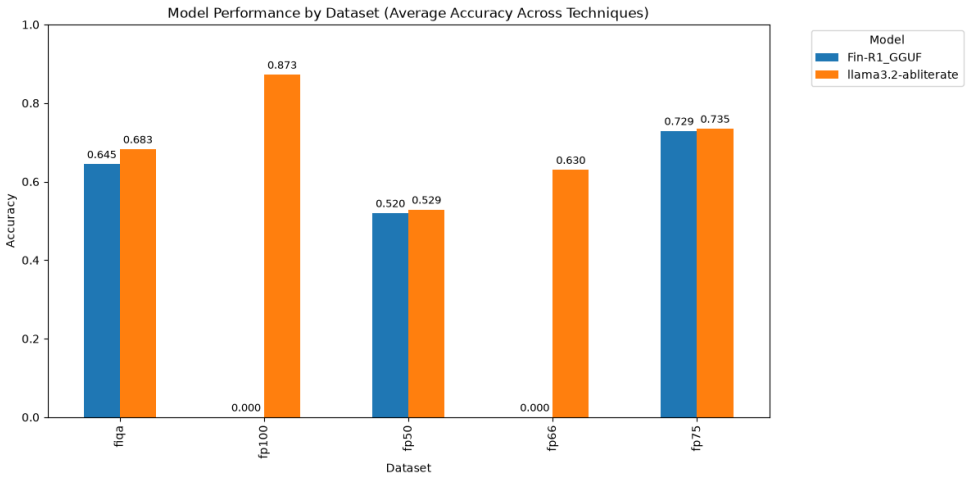
"Finnish financial group Aktia reports operating profit of EUR 44.4 mn in January-September 2009, up from EUR 37.3 mn in the corresponding period in 2008."

To be honest, I took the liberty of looking for a sentimentally convoluted sentence to make a fair comparison with the example I considered from FP-50. Looking at this sentence, we focus on two phrases: "reports operating profit" and "up by EUR 37.3 mn," and, as expected, any layman, finance professional, and LLM with basic NLP capabilities would choose "positive" for the sentiment in this example. And that is exactly what happened. This sentence is labelled as "positive" with 100% annotator agreement.

Hence, I believe the hypothesis that "**Accuracy depends on Annotator Agreement**" is well reasoned along with enough statistical support.

The second hypothesis stated that "**A financially specialized LLM is better for FSA**." This is a premise that was driven logically but needed to be tested. To a Layman, if there is a financial task, they would naturally go to a person with expertise or experience in financial matters, such as a Chartered Accountant, an MBA graduate, or someone with similar credentials and background.

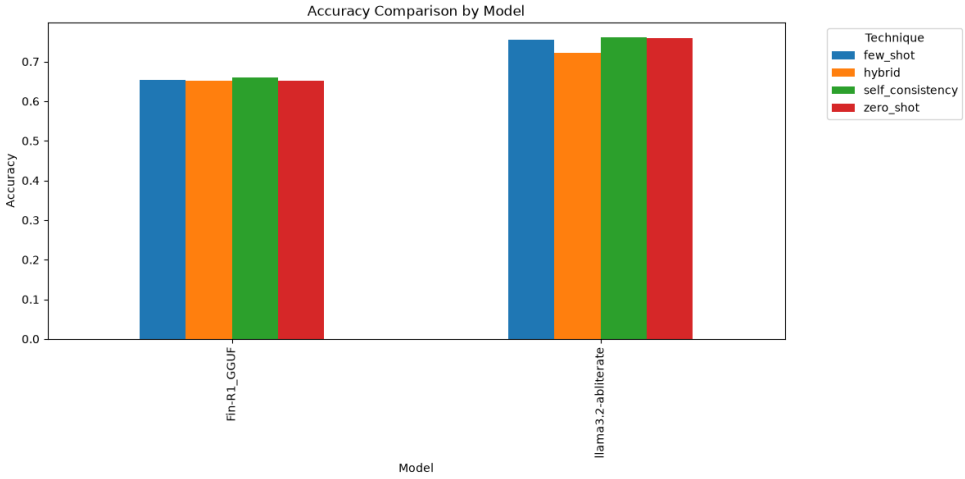
To test this hypothesis, I compared the overall accuracy of both models. The Fin-R1-GGUF model (fine-tuned for finance-related tasks) and the Llama-3.2-abliterate model. The Finance model was given an advantage to test this hypothesis. Fin-R1-GGUF is an 8B-parameter model, while Llama-3.2 abliterate is a 3.4B-parameter model. The results for this test were seemingly interesting. The Llama-3.2-abliterate model has outperformed the Fin-R1-GGUF on almost every dataset it was tested on, but there is a pattern to observe, considering the visual below.



Fin-R1-GGUF trails behind Llama-3.2-abliterate by merely >1% accuracy in the Financial PhraseBank dataset, but for the FiQA dataset, the difference is a substantial ~4%. Now, looking at the difference, the FiQA dataset contains financial news headlines with a slightly broader domain spread than the financial PhraseBank, which contains strictly financial statements. It seems like the Fin-R1-GGUF model is only suited for heavily finance-focused tasks and context. Where financial news headlines come into play, this becomes a job where Llama-3.2's more

generalized training gives it an advantage of increased capability in understanding broader context.

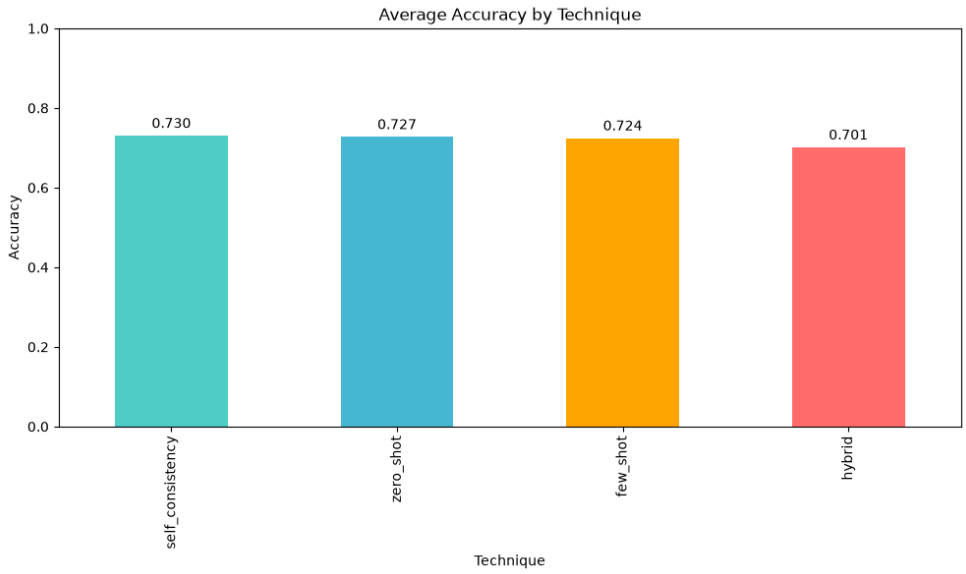
Looking at the model performance comparison by technique, there was not much to observe. Every technique uniformly outperformed in Llama-3.2 with no exceptions.



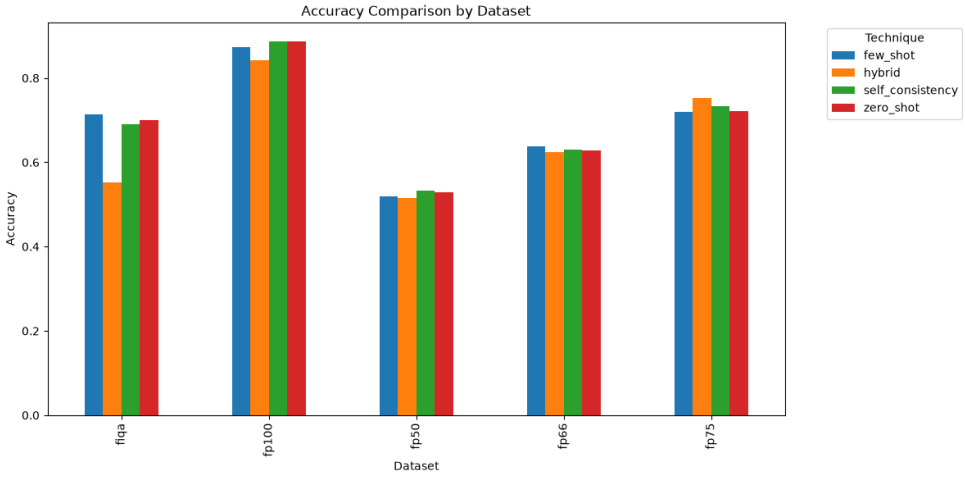
Hence, it is probably safe to assume that model performance depends on the type of content in the dataset, and the Fin-R1-GGUF model performs better in a much more constrained segment of finance than across a wider range of its relative domain boundaries.

The third hypothesis stated that "**The annotators' instruction-assisted prompting can outperform Few-Shot.**" The idea behind this novel method was that if the model is given the same instructions as the human annotator, then its performance should also be comparable to that of a human annotator. This idea did not hold solely even in the mini experiment. Hence, I hybridized this method with the best-performing method in that experiment, which was Few-Shot.

This hybridized AIAP method then performed much better than its original form, able to cope with the remaining methods. In the mini experiment, Chain-of-Thought performed the worst; hence, it was replaced by Self-Consistency, which is also an enhanced form of Chain-of-Thought. Surprisingly, this method performed best. It might be safe to assume that, since the mini experiment was conducted on a small subset of the entire Financial PhraseBank dataset, the randomly chosen subset might have been skewed toward sentences with multiple reasoning paths; hence, leading to the failure of Chain-of-Thought, which follows a greedy decoding strategy. Looking at the overall prompting technique performance visuals, we can see that Self-Consistency does not have a significant advantage over other techniques.

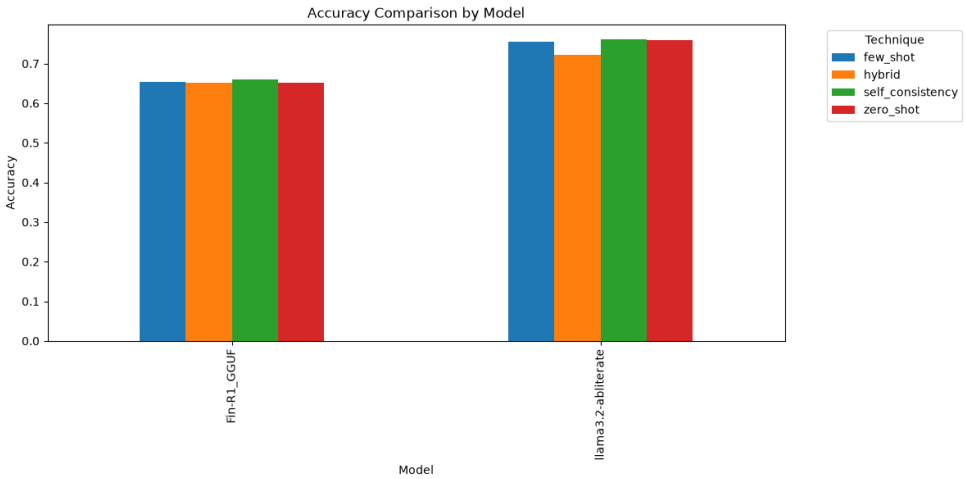


Hybrid-AIAP is barely 3% behind Self-Consistency, which is an acceptable margin of error in my opinion. Although, if we look at the prompting technique accuracy comparisons across datasets, there is an observable pattern.



Looking at this visual, we can observe two clear trends. Firstly, Hybrid-AIAP's accuracy drastically increased from FP-50 to FP-75. Secondly, the accuracy fell in FP-100. In FP-66 and FP-100, tests were conducted only with the Llama-3.2-abliterate model, which might suggest that the Hybrid-AIAP technique is better suited for the Fin-R1-GGUF model. From my interpretation, it also looks like, since the prompt gives instructions as well as examples to the model, it becomes too much for it to handle convoluted reasoning, hence showing the drastic underperformance on the FiQA dataset and the slight underperformance on FP-50.

Looking at the model-based technique performance, this trend looks quite clear.

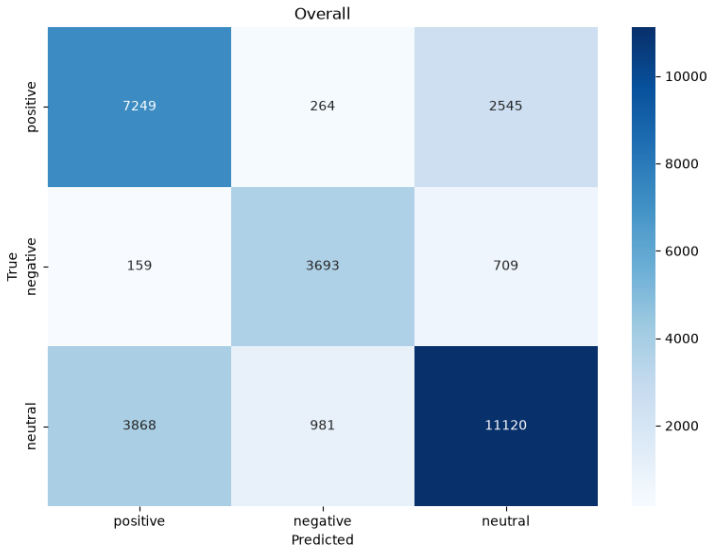


Hybrid-AIAP performs much more closely to the other prompting techniques on the Fin-R1-GGUF model, while it notably underperforms with Llama-3.2-abliterate. This means that this prompting technique is better suited for finance-specialized models.

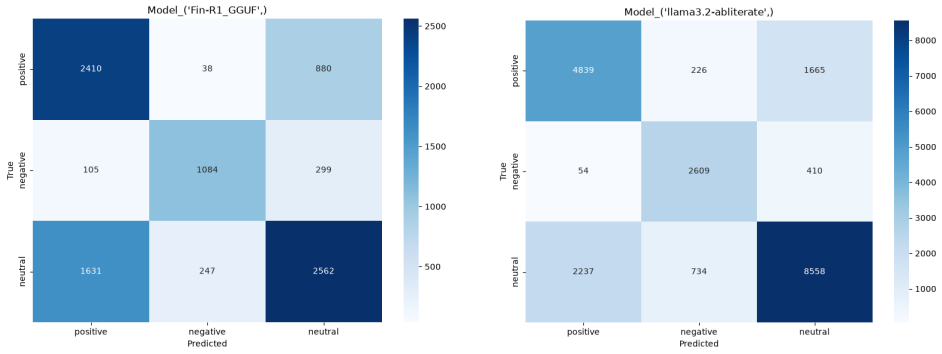
The fourth hypothesis stated that "**3B-LLMs can compete with GPT-3.5-like models on FSA.**" To explore this hypothesis, we considered the results of Few-Shot and Zero-Shot with the Llama-3.2-abliterate model on the entire Financial Phrasebank dataset. This was done to compare our study's results with those of Y. Shen and P. K. Zhang (2024)[7]. As we saw in the results section, even though GPT-4o outperformed Llama-3.2 by 10%, which is in line with what was vastly expected, this was not the case with GPT-3.5-Turbo. Llama-3.2-abliterate, a 3B-parameter model capable of running on consumer-grade laptops, should have shown significant disadvantages compared to GPT-3.5-Turbo, but the performance difference was negligible relative to the computing power GPT-3.5-Turbo provides. I believe that this may be partially attributed to the use of Llama-3.2's "abliterate," which has all safety filters removed. This model is basically tuned to focus on the given task without looking for restrictions. It is hypothesized that the removal of safety filters reduces the amount of computing power it spends, freeing up more parameters for the actual task. To completely understand and verify this theory, this same task needs to be performed on the Llama-3.2 model without abliteration.

6.2 Error Analysis

In this section, I aim to analyse the patterns created by incorrect predictions in this experiment. This helps us understand whether the results were skewed toward the majority class or whether the models failed to classify a particular class in a dataset. Let's start by looking at the overall confusion matrix for the experiment.



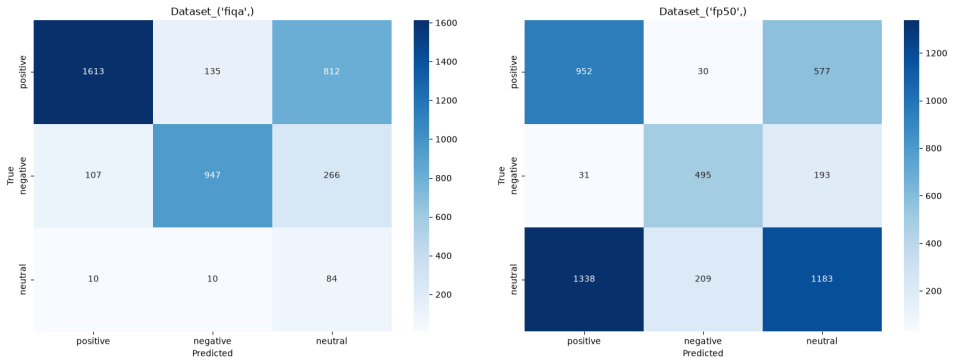
Observing the values in this matrix, we can clearly observe that there was some amount of confusion between the extremes and the neutral. Out of 10,058 positive-labelled texts, 2545 were predicted "neutral"; out of 4561 neutral-labelled texts, 709 were predicted "neutral," and out of 15,969 neutral-labelled statements, 3868 were wrongly predicted "positive." This means; that although in total only 19% of negative statements were wrongly predicted, roughly 25% of positive-labelled statements were labelled "neutral" and vice versa. This means the models are having trouble distinguishing "positive" and "neutral" sentiments. Let's consider model-level confusion matrices to see if we can pinpoint where a majority of errors are occurring.



Comparing the two confusion matrices for Fin-R1-GGUF and Llama-3.2-abliterate, I have a very interesting observation. Looking for the previously found pattern, Fin-R1-GGUF wrongly labelled roughly 25% of positive-labelled statements as "neutral," but roughly 37% neutral-labelled statements were labelled "positive." While Llama-3.2-abliterate wrongly labelled approximately 19% of "neutral" statements as "positive" and almost 25% of "positive" statements as "neutral." We

know for a fact that Llama-3.2 outperformed Fin-R1-GGUF. But I believe that the performance of Llama-3.2 looks better due to additional tests conducted on FP-100 using it, which yielded a high accuracy of around 80% across all prompting techniques.

For a fairer assessment of errors, let's check the confusion matrices for FiQA and FP-50 since these datasets performed the worst.



As clearly visible in the confusion matrix for the FiQA dataset, the experiment performed poorly with positive labels, predicting 31% of them wrongly as "neutral." We can also observe a microtrend where 20% of negative labels are incorrectly predicted as "neutral." With the FP-50 dataset, we have a major finding: roughly 40% of neutral labels are wrongly predicted as "positive," and 37% of positive labels are wrongly predicted as "neutral."

From the above assessment, we now have clarity that the models can only very vaguely estimate the class boundaries. And this also relates to the clarity and classification of the labels. The FiQA dataset, which contains financial news headlines, gave the models difficulty classifying between neutral labels, while the FP-50 dataset is known to have poor annotator support. With these findings, we can pinpoint exactly where the annotators disagreed with the labels, which accounts for the equivalent number of wrong predictions in our experimental results.

6.3 Disparity with Resource Usage

As mentioned in earlier sections, I used the Fin-R1-GGUF model (finance-specialized model) in comparison with Llama-3.2-abliterate. Fin-R1-GGUF is an 8B-parameter model with a size of 4.7 GB and uses substantially more computational resources than Llama-3.2-abliterate, which is a 3.4B-parameter model with a size of 2.2 GB. The Fin-R1 takes almost 4 times as long as Llama-3.2 to complete a single test. With these kinds of resources being spent, it is naturally expected of Fin-R1 to perform better, but that was not the case. Llama-3.2 outperformed Fin-R1 on all counts, making this experiment look like it spent a disproportionate amount of resources on an underperforming model for the task.

The second, very interesting observation concerned token usage. The Zero-Shot prompting technique and Self-Consistency were the top performers in this experiment, and when we compare the token efficiency calculated during the experiment, we see interesting trends. Both winning techniques had an 80% token efficiency, meaning they were quite verbose. While the question asked of the model was to simply classify the given financial phrases into "positive," "negative," and "neutral," a concise answer was expected. Surprisingly, the responses that had the highest token efficiency ratio were the right ones. Hybrid-AIAP, the losing technique, had 50% token efficiency and had the lowest accuracy. This trend probably means that the models that generate an appropriate amount of reasoning for the prompt generally give accurate answers.

6.4 Project Limitations

There are a few limitations that I would like to elaborate on in this section.

This project was conducted on a consumer MacBook, which has limited resources to run Large Language Models. The hardware can comfortably run 3B models but performs poorly with 8B models.

The Financial PhraseBank dataset is heavily skewed towards the "neutral" class. Out of 4846 sentences, 2876 have been labelled "neutral," which is roughly 60% of the entire dataset. This skew most likely affects the results as well, with positive and negative labels being classified as "neutral."

This skew is completely opposite for the FiQA dataset, where neutral labels are barely 13 out of roughly 498. That's less than 1% representation. Due to these skews, there possibly are unverifiable accuracy issues with the results of this experiment.

Another issue that came to light while building the raw response parser for this experiment is the existence of bogus sentences in the datasets. For Example, this sentence that exists in the FP-75 dataset:

"A CUT ABOVE Bring the outdoors in with these birch-branch coasters."

This sentence and a few others do not make any financial sense to me or either of the 2 models. The models simply refused to comment on their sentiment, a decision I can logically back. Such sentences reduce the accuracy of the results and create an inconsistency between the reported support totals and the confusion matrices' values.

7. Conclusion & Future Scope

This study evaluated the efficacy of various prompt engineering strategies for financial sentiment analysis, yielding three key findings that simultaneously answer the initial research hypotheses and open new avenues for future investigation.

7.1 The Impact of Model Abliteration over Size and Specialization

Unexpectedly, the general-purpose Llama-3.2-abliterate (3B) outperformed the larger, domain-specific Fin-R1-GGUF (8B) model, matching the performance of the commercial GPT-3.5-Turbo. One possible explanation for this performance may be that removing safety alignment mechanisms (abliteration) probably allows smaller models to allocate their full computational capacity toward task execution, challenging traditional assumptions about model size and domain specialization. But this remains to be verified.

- **Future Scope:** As the exact mechanisms driving this performance gap remain unverified, future research should evaluate the non-abliterated Llama-3.2 model on the same tasks to isolate whether the performance gains stem solely from the removal of safety alignments or other architectural traits.

7.2 Conditional Efficacy of the AIAP Hybrid Approach

The Annotators' Instruction-Assisted Prompting (AIAP) method, combined with Few-Shot examples, matched the performance of standard baselines exclusively on the cleanest dataset (FP-75) using the domain-specialized model. However, it underperformed on noisier (FP-50) and mixed-domain (FiQA) datasets, proving its effectiveness is highly conditional on data quality.

- **Future Scope:** Because Self-Consistency emerged as the most robust baseline technique in this study, but not significantly compared to the Zero-Shot which ranked second for this study. Looking at this, future iterations of the AIAP framework should attempt hybridization with Self-Consistency, since the mini experiment failed to achieve an acceptable accuracy with annotators' instructions only (e.g., generating multiple rule-guided reasoning paths for majority voting) to improve resilience against noisy data.

7.3 The Relativity of Ground Truth in Subjective Tasks

Finally, this study demonstrated that model accuracy is directly proportional to human annotator agreement. Across all tested variables, performance on the FP-75 dataset was approximately 20% higher than on the FP-50 dataset. This highlights a fundamental limitation in sentiment classification: when human annotators lack a unanimous consensus, the model's "incorrect" prediction may simply mirror the reasoning path of a dissenting human annotator.

- **Future Scope:** To validate this trend, future studies should test the Fin-R1-GGUF model strictly on the FP-100 dataset (100% human agreement), where the AIAP hybrid method is projected to achieve its maximum potential.

8. Self-Assessment

8.1 Project Execution and Achievements

This study set out to systematically evaluate prompt engineering techniques for Financial Sentiment Analysis using small, locally run Large Language Models. I successfully designed and executed a comparative study of four prompting techniques (Zero-Shot, Few-Shot, Self-Consistency, and a novel hybrid Annotators' Instruction-Assisted Prompting method) across multiple datasets and two models (Fin-R1-GGUF and Llama-3.2-abliterate). The project was completed within the allocated timeframe, with all experiments efficiently run locally without incurring any API costs.

Developing a reproducible experiment pipeline using Ollama and Python allowed me to test multiple variables with minimal code changes. Performance was systematically evaluated using robust metrics (precision, recall, F1, macro F1, weighted F1) and validated via McNemar's test, ensuring that observed differences were statistically meaningful. I am particularly proud of the transparent reporting of the novel AIAP technique, acknowledging its limitations while highlighting its statistically significant improvement (76.1%) on the cleanest dataset (FP-75) using the financial-specialized model (Fin-R1).

8.2 Challenges Overcome

A significant challenge arose early in the project when a simplistic parser misclassified model response, leading to inflated "positive" predictions. This required a complete overhaul of the parsing logic, teaching me that LLM outputs are inherently unpredictable and require robust, pattern-based extraction. Furthermore, I had to mitigate an unanticipated positive bias exhibited by the Fin-R1 model—which initially predicted "positive" for nearly all samples—by refining the parser and experimenting with different prompt formulations.

Data preparation and visualization also presented distinct hurdles. The SEntFiN dataset required complex preprocessing to extract single-entity sentiment labels, demanding more time than initially scheduled. Additionally, to ensure unbiased results for the primary hypotheses, I had to isolate and exclude 8 specific tests (conducted on FP-66 and FP-100 using Llama-3.2-abliterate) from the broader evaluation metrics. Engineering the logic to exclude these specific tests while generating global graphs, charts, and plots proved to be a demanding but rewarding programming exercise.

8.3 Lessons Learned and Career Impact

Building the experiment harness incrementally proved invaluable, allowing me to catch bugs early and iterate quickly. As a former Associate Software Engineer,

I already understood the importance of reproducible infrastructure, but I learned firsthand that this is exceptionally critical in machine learning research, where minor changes in model versions and library dependencies can drastically alter results.

Ultimately, this project has significantly shaped my career trajectory. Gaining hands-on experience with LLM deployment, evaluation, and prompt engineering has equipped me with skills highly sought after in the AI industry. I have developed a much deeper understanding of rigorous experimental design and financial NLP, and I plan to pursue a career where I can bridge machine learning and software engineering to create impactful enterprise solutions.

9. References

[1] Lemeš, Samir. (2024). Prompt Engineering. 10.5644/PI2024.215.08.

[2] Joshi, Satyadhar, Review of Prompt Engineering Techniques in Finance: An Evaluation of Chain-of-Thought, Tree-of-Thought, and Graph-of-Thought Approaches (January 31, 2025). Available at SSRN: https://ssrn.com/abstract=5339795 or http://dx.doi.org/10.2139/ssrn.5339795

[3] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chainof-thought prompting elicits reasoning in large language models. Advances in Neural Information Processing Systems, 35:24824–24837, 2022.

[4] Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A. and Zhou, D., 2022. Self-consistency improves chain of thought reasoning in language models. arXiv preprint arXiv:2203.11171.

[5] Daniel Jurafsky and James H. Martin. 2026. Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models, 3rd edition.

[6] Sahoo, P., Singh, A.K., Saha, S., Jain, V., Mondal, S. and Chadha, A., 2024. A systematic survey of prompt engineering in large language models: Techniques and applications. *arXiv preprint arXiv:2402.07927*.

[7] Y. Shen and P. K. Zhang, "Financial Sentiment Analysis on News and Reports Using Large Language Models and FinBERT," 2024 IEEE 6th International Conference on Power, Intelligent Computing and Systems (ICPICS), Shenyang, China, 2024, pp. 717-721, doi: 10.1109/ICPICS62053.2024.10796670.

[8] https://neoteric.eu/blog/what-is-gpt-4o/#core-specs-of-gpt-4-o

[9] J. Shetty, O. Guha, V. H. Reddy, D. A. Almeida and N. P. Shetty, "Contrastive Learning for Financial Sentiment Analysis and Annotator Agreement Prediction," 2025 1st International Conference on Data Science and Intelligent Network Computing (ICDSINC), Raipur, India, 2025, pp. 303-314, doi: 10.1109/ICDSINC66221.2025.11448137.

[10] Malo, P., Sinha, A., Korhonen, P., Wallenius, J. and Takala, P., 2014. Good debt or bad debt: Detecting semantic orientations in economic texts. Journal of the Association for Information Science and Technology, 65(4), pp.782-796.

[11] Fatouros, G., Soldatos, J., Kouroumali, K., Makridis, G. and Kyriazis, D., 2023. Transforming sentiment analysis in the financial domain with ChatGPT. Machine Learning with Applications, 14, p.100508.

[12] Qi, R. (2025) "Financial News Sentiment Analysis and Market Sentiment Prediction Based on Large Language Models", Journal of Computer, Signal, and System Research, 2(6), pp. 22–31. doi:10.71222/y0vdp428.

[13] Sinha, A., Kedas, S., Kumar, R. and Malo, P., 2022. SEntFiN 1.0: Entity-aware sentiment analysis for financial news. Journal of the Association for Information Science and Technology, 73(9), pp.1314-1335.

[14] https://economictimes.indiatimes.com/

[15] Maia, Macedo, Handschuh, Siegfried, Freitas, André, Davis, Brian, McDermott, Ross, Zarrouk, Manel, & Balahur, Alexandra. (2018). WWW'18 open

challenge: financial opinion mining and question answering. Paper presented at the The Web Conference 2018, Lyon, France, 23-27 April.

[16] A. Singhal and N. Singhal, "Analogy-Driven Financial Chain-of-Thought (AD-FCoT): A Prompting Approach for Financial Sentiment Analysis," 2025 Artificial Intelligence for Business (AIxB), Laguna Hills, CA, USA, 2025, pp. 41-46, doi: 10.1109/AIxB65684.2025.00014.

[17] Dong, Z., Fan, X., and Peng, Z., “FNSPID: A comprehensive financial news dataset in time series,” arXiv : 2402.06698, 2024.

10. How to Use My Project

In this section I will provide instructions to reproduce this experiment. Please follow these instructions to the letter as the results depend heavily on the origin of the data sources. The instructions will include the webpages from where to obtain the datasets to how to set up the working Python virtual environment with all the correct dependency versions. Please follow all instructions consecutively instead of concurrently to avoid issues.

Note: System Level Commands given in this section are tested on Linux and Unix systems, if these do not work on Command Prompt or Power Shell, please use alternate commands.

Please Follow these instructions to reproduce this experiment:

- Download Financial PhraseBank dataset from the given webpage.
  - https://huggingface.co/datasets/takala/financial_phrasebank/tree/main/data
- Download the SEntFIN 1.0 dataset from the given webpage.
  - https://github.com/pyRis/SEntFiN/blob/main/SEntFiN.csv
- Download FiQA dataset in the form of test, train and validation (.parquet) files from the given webpage
  - https://huggingface.co/datasets/vibrantlabsai/fiqa/tree/main/data/main
- Create a new Directory in your system for conducting this experiment and navigate to it in the terminal.
- Move the above downloaded dataset files in the created project Directory.
- Create a new python virtual environment
  - python3 -m venv venv
- Activate this virtual environment
  - source venv/bin/activate
- Install the dependencies from **requirements.txt**
  - pip install -r requirements.txt
- Extract the zip file containing Financial PhraseBank datasets into a sub-directory
- Run **deduplicate_phrasebank.py** in the above directory.
- Move the resulting csv files to a separate directory within the project directory. (This directory will now be referred to as the **Preprocessed_Datasets** directory.)
- Run **process_sentfin.py** in the directory containing the downloaded sentfin dataset file.
- Move the resulting csv file to the **Preprocessed_Datasets** directory.
- Run **process_fiqa.py** in the directory containing the downloaded fiqa parquet files.
- Move the resulting csv file to the **Preprocessed_Datasets** directory.
- Now you can either choose to delete the downloaded dataset files or can store them for reference in a separate directory.
- Install ollama on terminal (MacOS)

    - brew install ollama/tap/ollama
    - ollama –-version #(To verify installation)
- Download the required LLMs
    - ollama pull huihui_ai/llama3.2-abliterate:latest
    - ollama pull hf.co/Mungert/Fin-R1-GGUF:latest
- Copy preprocessed dataset files from **Preprocessed_datasets** directory into project directory.
- Run **experiment_harness.py** in **project** directory**.** This experiment can be run all at once or also in parts. Use the below arguments to run the test in parts
    - –-model or -m (to choose model.) Accepted param values:
        - huihui_ai/llama3.2-abliterate:latest
        - hf.co/Mungert/Fin-R1-GGUF:latest
    - –-dataset or -d (to choose dataset.) Accepted param values:
        - complete dataset filename
        - example: fiqa_sentiment.csv
    - –-output or -o (Output file prefix)
    - –-techniques or -t (to choose prompting techniques.) Accepted param values:
        - zero_shot
        - few_shot
        - self_consistency
        - hybrid
- Create 2 new directories within project directory **Final_results** and **Evaluation_Metrics**
- Copy all the outputted result files to **Final_results** directory. Let the Summary and Final .csv files remain in project directory or they can be deleted. They are only for reference.
- Run the **evaluate_results.py** with parameters. The following params are accepted:
    - --input_dir (directory path to result csv files)
        - Please pass: ./Final_results
    - –-output_dir (Output directory to save result files to)
        - Please pass: ./Evaluation_Metrics

The evaluation results will be saved at ./Evaluation_Metrics/ for analysis and evaluation of performance and verification of the results published in this study.

11. Appendix A

This section contains all the scripts, raw code and requirement files essential to reproduce the experiment conducted in this study. Following the instructions in the above section to the best of your ability is imperative to successfully reproduce this experiment.

11.1 deduplicate_phrasebank.py

```
import pandas as pd
import os

def parse_financial_phrasebank(filename):
    """Parse the Financial PhraseBank text file into a DataFrame."""
    data = []
    encodings = ['utf-8', 'latin-1', 'cp1252', 'iso-8859-1']

    for encoding in encodings:
        try:
            with open(filename, 'r', encoding=encoding) as f:
                for line in f:
                    line = line.strip()
                    if not line:
                        continue
                    if '@' in line:
                        parts = line.rsplit('@', 1)
                        if len(parts) == 2:
                            text = parts[0].strip()
                            label = parts[1].strip()
                            if label in ['negative', 'neutral', 'positive']:
                                data.append({
                                    'text': text,
                                    'sentiment': label
                                })
            print(f"  Parsed {filename} with {encoding}")
            break
        except UnicodeDecodeError:
            continue

    return pd.DataFrame(data)

def deduplicate_datasets():
    """Create unique versions of each agreement level."""
```

```
print("="*60)
print("Deduplicating Financial PhraseBank")
print("="*60)

# Files in increasing agreement order
files = [
    "Sentences_50Agree.txt",
    "Sentences_66Agree.txt",
    "Sentences_75Agree.txt",
    "Sentences_AllAgree.txt"
]

# Parse all files
dfs = {}
for file in files:
    print(f"\nParsing {file}...")
    dfs[file] = parse_financial_phrasebank(file)
    print(f"  Loaded {len(dfs[file])} samples")

# Create unique versions
print("\n" + "="*60)
print("Creating Unique Datasets")
print("="*60)

# 1. AllAgree: keep everything
df_all = dfs["Sentences_AllAgree.txt"].copy()
df_all.to_csv("FinancialPhraseBank_AllAgree.csv", index=False)
print(f"\n  FinancialPhraseBank_AllAgree.csv: {len(df_all)} samples
(100% agreement)")

# 2. 75Agree: remove sentences that are in AllAgree
df_75 = dfs["Sentences_75Agree.txt"].copy()
# Create a set of texts from AllAgree for fast lookup
all_texts = set(df_all['text'])
df_75_unique = df_75[~df_75['text'].isin(all_texts)]
df_75_unique.to_csv("FinancialPhraseBank_75Agree.csv", index=False)
print(f"  FinancialPhraseBank_75Agree.csv: {len(df_75_unique)} samples
(75% agreement, not in 100%)")

# 3. 66Agree: remove sentences that are in 75Agree
df_66 = dfs["Sentences_66Agree.txt"].copy()
# Create a set of texts from 75Agree (including its unique ones)
# We need the original 75Agree dataset, not just the unique one
df_75_original = dfs["Sentences_75Agree.txt"]
seventyfive_texts = set(df_75_original['text'])
df_66_unique = df_66[~df_66['text'].isin(seventyfive_texts)]
df_66_unique.to_csv("FinancialPhraseBank_66Agree.csv", index=False)
```

```
        print(f" FinancialPhraseBank_66Agree.csv: {len(df_66_unique)} samples
(66% agreement, not in 75%)")

        # 4. 50Agree: remove sentences that are in 66Agree
        df_50 = dfs["Sentences_50Agree.txt"].copy()
        # Use the original 66Agree dataset
        df_66_original = dfs["Sentences_66Agree.txt"]
        sixtysix_texts = set(df_66_original['text'])
        df_50_unique = df_50[~df_50['text'].isin(sixtysix_texts)]
        df_50_unique.to_csv("FinancialPhraseBank_50Agree.csv", index=False)
        print(f" FinancialPhraseBank_50Agree.csv: {len(df_50_unique)} samples
(50% agreement, not in 66%)")

        # Summary
        print("\n" + "="*60)
        print("SUMMARY")
        print("="*60)

        summary = [
            ("50Agree", df_50_unique),
            ("66Agree", df_66_unique),
            ("75Agree", df_75_unique),
            ("AllAgree", df_all)
        ]

        for name, df in summary:
            print(f"\n{name}:")
            print(f"  Total samples: {len(df)}")
            print(f"  Class distribution:")
            print(df['sentiment'].value_counts())
            print(f"  Class percentages:")

print(df['sentiment'].value_counts(normalize=True).mul(100).round(1))

        print("\n" + "="*60)
        print(" All files saved successfully!")

    if __name__ == "__main__":
        deduplicate_datasets()
```

11.2 process_fiqa.py

```
import pandas as pd
import os

# Define the mapping from the numeric score to sentiment labels
```

```
def map_score_to_sentiment(score):
    """
    Map a numeric sentiment score to our three-class system.
    Scores range from -1 (very negative) to 1 (very positive).
    """
    if pd.isna(score):
        return "neutral"
    if score > 0:
        return "positive"
    elif score < 0:
        return "negative"
    else:
        return "neutral"

def process_fiqa_file(file_path):
    """
    Read a FiQA parquet file, filter for headlines, and convert req format.
    """
    print(f"  Processing: {os.path.basename(file_path)}")

    # Read the parquet file
    df = pd.read_parquet(file_path)
    print(f"    Original rows: {len(df)}")

    # Filter for headlines only
    df_headlines = df[df['type'] == 'headline'].copy()
    print(f"    Headlines only: {len(df_headlines)}")

    if len(df_headlines) == 0:
        return pd.DataFrame(columns=['text', 'sentiment'])

    # Rename 'sentence' to 'text'
    df_headlines = df_headlines.rename(columns={'sentence': 'text'})

    # Apply sentiment mapping
    df_headlines['sentiment']                                                    =
df_headlines['score'].apply(map_score_to_sentiment)

    # Keep only the columns we need
    df_final = df_headlines[['text', 'sentiment']]

    return df_final

# --- Main Execution ---
print("="*60)
print("Processing FiQA Sentiment Dataset")
print("="*60)
```

```
files = [
    "train.parquet",
    "valid.parquet",
    "test.parquet"
]

all_dfs = []
for file in files:
    if os.path.exists(file):
        df_processed = process_fiqa_file(file)
        all_dfs.append(df_processed)
    else:
        print(f"   Warning: File not found - {file}")

# Combine all DataFrames
if all_dfs:
    final_df = pd.concat(all_dfs, ignore_index=True)

    print("\n" + "="*60)
    print("Final Dataset Summary")
    print("="*60)
    print(f"Total samples: {len(final_df)}")
    print("\nClass distribution:")
    print(final_df['sentiment'].value_counts())
    print("\nPercentages:")

print(final_df['sentiment'].value_counts(normalize=True).mul(100).round(1))

    # Save to CSV
    final_df.to_csv('fiqa_sentiment.csv', index=False)
    print("\n Dataset saved to 'fiqa_sentiment.csv'")
else:
    print("\n No data processed. Please ensure the .parquet files are in the same directory.")
```

11.3 experiment_harness.py

```
import ollama
import pandas as pd
import time
from datetime import datetime
import sys
import argparse
import os
```

```
import re

# Force unbuffered output
sys.stdout.reconfigure(line_buffering=True)

# ============================================================
# CONFIGURATION
# ============================================================
# default parameters
CONFIG = {
    "model_name": "hf.co/Mungert/Fin-R1-GGUF:latest",
    "dataset_file": "financial_phrasebank_small.csv",
    "output_prefix": "experiment_results",
    "max_samples": None,  # Set to e.g., 100 to limit samples
    "techniques": ["zero_shot", "few_shot", "self_consistency", "hybrid"],    # Default
}

# ============================================================
# PROMPT TEMPLATES
# ============================================================
PROMPT_TEMPLATES = {
    "zero_shot": """
    You are a financial sentiment analyst. Classify the sentiment of the following financial news text as 'positive', 'negative', or 'neutral'.

    Text: {text}

    Sentiment:""",

    "few_shot": """
    You are a financial sentiment analyst. Classify the sentiment of the following financial news text as 'positive', 'negative', or 'neutral'.

    Examples:
    Text: "The company reported record profits this quarter."
    Sentiment: positive

    Text: "The stock market crashed due to inflation fears."
    Sentiment: negative

    Text: "The board announced a routine quarterly review."
    Sentiment: neutral

    Now classify this text:
    Text: {text}
    Sentiment:""",
```

"self_consistency": """
You are a financial sentiment analyst. Think step-by-step about the sentiment of the following financial news text. Classify the sentiment of the following financial news text as 'positive', 'negative', or 'neutral'.

Text: {text}

Sentiment:""",

"hybrid": """
You are a financial sentiment analyst. You have been given the following instructions from the dataset annotators:

Instructions for annotators:
- Sentiment is the opinion or feeling expressed in the text towards the financial entity.
- Positive: The text expresses a favorable or optimistic view.
- Negative: The text expresses an unfavorable or pessimistic view.
- Neutral: The text expresses no clear opinion or contains only factual information.

Examples:
Text: "The company reported record profits this quarter."
Sentiment: positive

Text: "The stock market crashed due to inflation fears."
Sentiment: negative

Text: "The board announced a routine quarterly review."
Sentiment: neutral

Now, carefully analyze this text and classify its sentiment as 'positive', 'negative', or 'neutral' using the instructions above.

Text: {text}

Sentiment:"""
}

```
# ============================================================
# HELPER FUNCTIONS
# ============================================================

def call_llm(prompt, model, temperature=0, max_retries=2):
    """Send prompt to Ollama with specified temperature"""
    for attempt in range(max_retries):
```

```
            try:
                response = ollama.chat(
                    model=model,
                    messages=[{"role": "user", "content": prompt}],
                    options={
                        "temperature": temperature,
                        "num_predict": 1000
                    }
                )
                return response['message']['content'].strip()
            except Exception as e:
                print(f"    ERROR (attempt {attempt+1}): {e}")
                if attempt < max_retries - 1:
                    print(f"    Retrying in 5 seconds...")
                    time.sleep(5)
                else:
                    return None

        def parse_response(response):
            """Extract sentiment from LLM response - robust version with markdown
support"""
            if response is None or pd.isna(response):
                return "unknown"

            response_str = str(response)
            response_lower = response_str.lower()

            # Handle the case where the response is just the sentiment word
            if response_lower.strip() in ["positive", "negative", "neutral"]:
                return response_lower.strip()

            # Look for patterns like "Sentiment: positive" or "Sentiment = positive"
(with markdown)
            patterns = [
                r'sentiment\s*[:=]\s*\*{0,2}(positive|negative|neutral)\*{0,2}',
                r'final sentiment\s*[:=]\s*\*{0,2}(positive|negative|neutral)\*{0,2}',
                r'overall sentiment\s*[:=]\s*\*{0,2}(positive|negative|neutral)\*{0,2}',
                r'sentiment\s+is\s+\*{0,2}(positive|negative|neutral)\*{0,2}',
                r'classified\s+as\s+\*{0,2}(positive|negative|neutral)\*{0,2}',
                r'classification\s*[:=]\s*\*{0,2}(positive|negative|neutral)\*{0,2}',
                r'final answer\s*[:=]\s*\*{0,2}(positive|negative|neutral)\*{0,2}',
                r'answer\s*[:=]\s*\*{0,2}(positive|negative|neutral)\*{0,2}',
                r'\\boxed\s*\{\s*\*{0,2}(positive|negative|neutral)\*{0,2}\s*\}',
                r'boxed\s*\{\s*\*{0,2}(positive|negative|neutral)\*{0,2}\s*\}',

r'(?:^|\n)\s*(?:final|overall)?\s*sentiment\s*(?:is|:)?\s*\*{0,2}(positive|negative|
neutral)\*{0,2}\s*[.!]?$',
```

```
                # NEW PATTERNS FOR QUOTED SENTIMENTS:
                # "I would classify the sentiment of this financial news text as ""Neutral""."
                r'(?:classify|classifying|classify the sentiment of this financial news text as)\s+["\']?(positive|negative|neutral)["\']?',
                r'(?:classify|classification)\s+["\']?(positive|negative|neutral)["\']?',
                r'sentiment\s+as\s+["\']?(positive|negative|neutral)["\']?',
                r'sentiment\s+is\s+["\']?(positive|negative|neutral)["\']?',
                r'would classify this as\s+["\']?(positive|negative|neutral)["\']?',
                r'would classify the sentiment as\s+["\']?(positive|negative|neutral)["\']?',
                r'would be classified as\s+["\']?(positive|negative|neutral)["\']?',
                r'^["\']?(positive|negative|neutral)["\']?$',

                # Catch patterns like: "The sentiment is: positive"
                r'sentiment\s*[:=]\s*(positive|negative|neutral)',
                r'sentiment\s+is\s+(positive|negative|neutral)',

                # NEW PATTERNS FOR fp100:

r'(?:classify|classifying)\s+the\s+sentiment\s+of\s+this\s+financial\s+news\s+text\s+as\s+["\']?(positive|negative|neutral)["\']?',

r'(?:classify|classifying)\s+the\s+sentiment\s+as\s+["\']?(positive|negative|neutral)["\']?',

r'would\s+classify\s+this\s+as\s+["\']?(positive|negative|neutral)["\']?',

r'would\s+be\s+classified\s+as\s+["\']?(positive|negative|neutral)["\']?',

r'would\s+classify\s+the\s+sentiment\s+as\s+["\']?(positive|negative|neutral)["\']?',
                r'^["\']?(positive|negative|neutral)["\']?',
                r'classify\s+this\s+news\s+as\s+["\']?(positive|negative|neutral)["\']?',
            ]
            match_found = False

            for pattern in patterns:
                match = re.search(pattern, response_lower, re.IGNORECASE)
                if match:
                    match_found = True
                    return match.group(1).lower()

            # ============================================================
            # NEW BLOCK: Only runs if patterns above found nothing
            # ============================================================
```

```
            if not match_found:
                classification_phrases = [
                    "would classify",
                    "would be classified",
                    "classify the sentiment",
                    "classification"
                ]

                if any(phrase in response_lower for phrase in classification_phrases):
                    # Look for "as 'positive'"
                    quote_match =
re.search(r"as\s+['\"]?(positive|negative|neutral)['\"]?", response_lower)
                    if quote_match:
                        return quote_match.group(1).lower()

                    # Fallback: look for the sentiment word in the classification sentence
                    sentences = re.split(r'[.!?]+', response_lower)
                    for sentence in sentences:
                        if any(phrase in sentence for phrase in classification_phrases):
                            for word in ["positive", "negative", "neutral"]:
                                if word in sentence:
                                    return word

            # Find the last occurrence of any sentiment word near the end
            sentences = re.split(r'[.!?]+', response_lower)
            for sentence in reversed(sentences[-3:]):
                sentiment_match =
re.search(r'(?:sentiment|classification|answer)\s+is\s+\*{0,2}(positive|negative|n
eutral)\*{0,2}', sentence)
                if sentiment_match:
                    return sentiment_match.group(1)
                for word in ["positive", "negative", "neutral"]:
                    if word in sentence and ("sentiment" in sentence or "classification" in
sentence):
                        return word

            # Count occurrences
            pos_count = response_lower.count("positive")
            neg_count = response_lower.count("negative")
            neu_count = response_lower.count("neutral")

            if pos_count >= 2 and pos_count >= neg_count and pos_count >=
neu_count:
                return "positive"
            elif neg_count >= 2 and neg_count >= pos_count and neg_count >=
neu_count:
                return "negative"
```

```
        elif neu_count >= 2 and neu_count >= pos_count and neu_count >= neg_count:
            return "neutral"

        return "unknown"

    def run_experiment_single(df, technique, prompt_template, model_name, temperature=0):
        """Run one experiment with a specific technique (single pass)"""
        results = []
        print(f"\n=== Running {technique} ===")

        for idx, row in df.iterrows():
            text = row['text']
            true_label = row['sentiment']

            print(f"  Sample {idx+1}/{len(df)}: ", end="", flush=True)

            prompt = prompt_template.format(text=text)
            response = call_llm(prompt, model_name, temperature=temperature)
            predicted = parse_response(response)

            results.append({
                'statement': text,
                'true_sentiment': true_label,
                'predicted_sentiment': predicted,
                'raw_response': response,
                'technique': technique
            })

            status = "✅" if predicted == true_label else "❌"
            print(f"{status} True={true_label}, Predicted={predicted}", flush=True)
            time.sleep(0.5)

        return pd.DataFrame(results)

    def run_experiment_self_consistency(df, technique, prompt_template, model_name, num_runs=3):
        """Run self-consistency: multiple passes with temperature > 0, then majority vote"""
        print(f"\n=== Running {technique} (with Self-Consistency, {num_runs} runs) ===")

        all_results = []

        for run in range(num_runs):
            print(f"  Run {run+1}/{num_runs}:")
```

```
            temperature = 0.1 + (run * 0.1)
            results = []

            for idx, row in df.iterrows():
                text = row['text']
                true_label = row['sentiment']

                # Show progress immediately
                print(f"  Sample {idx+1}/{len(df)}: ", end="", flush=True)

                prompt = prompt_template.format(text=text)
                response = call_llm(prompt, model_name,
temperature=temperature)
                predicted = parse_response(response)

                # Show result immediately
                status = "✅" if predicted == true_label else "❌"
                print(f"{status} True={true_label}, Predicted={predicted}",
flush=True)

                results.append({
                    'id': idx,
                    'true_sentiment': true_label,
                    'predicted_sentiment': predicted,
                    'run': run,
                    'raw_response': response  # ADD THIS - store the raw response
                })
                time.sleep(0.5)

            all_results.append(pd.DataFrame(results))
            print(f"  Completed run {run+1}")

        combined = pd.concat(all_results, ignore_index=True)
        final_results = []

        for idx in df.index:
            sample_predictions = combined[combined['id'] == idx]
            true_label = sample_predictions['true_sentiment'].iloc[0]

            vote_counts =
sample_predictions['predicted_sentiment'].value_counts()
            if len(vote_counts) > 0:
                predicted = vote_counts.index[0]
            else:
                predicted = "unknown"

            # Get the first raw response from this sample (from run 0)
```

```
            first_response = combined[(combined['id'] == idx) & (combined['run'] == 0)]['raw_response'].iloc[0] if len(combined[(combined['id'] == idx) & (combined['run'] == 0)]) > 0 else f"Self-consistency vote from {num_runs} runs"

            final_results.append({
                'statement': df.iloc[idx]['text'],
                'true_sentiment': true_label,
                'predicted_sentiment': predicted,
                'raw_response': first_response,
                'technique': technique
            })

        return pd.DataFrame(final_results)

    # ============================================================
    # MAIN FUNCTION
    # ============================================================

    def run_experiment(model_name=None, dataset_file=None, output_prefix=None,
                       max_samples=None, techniques=None):
        """
        Run the full experiment with the given configuration.
        """

        # Use config defaults if not provided
        if model_name is None:
            model_name = CONFIG["model_name"]
        if dataset_file is None:
            dataset_file = CONFIG["dataset_file"]
        if output_prefix is None:
            output_prefix = CONFIG["output_prefix"]
        if max_samples is None:
            max_samples = CONFIG["max_samples"]
        if techniques is None:
            techniques = CONFIG["techniques"]

        # Validate techniques
        valid_techniques = list(PROMPT_TEMPLATES.keys())
        for tech in techniques:
            if tech not in valid_techniques:
                print(f"❌ Invalid technique: {tech}")
                print(f"   Available: {valid_techniques}")
                return None

        # ============================================================
        # LOAD DATASET
```

```
# ============================================================

print("="*60)
print(f"EXPERIMENT STARTING")
print("="*60)
print(f"Model: {model_name}")
print(f"Dataset: {dataset_file}")
print(f"Techniques: {techniques}")

if not os.path.exists(dataset_file):
    print(f"❌ Dataset file not found: {dataset_file}")
    return None

df = pd.read_csv(dataset_file)

# Handle column names
if 'text' not in df.columns and 'sentence' in df.columns:
    df = df.rename(columns={'sentence': 'text'})

if 'sentiment' not in df.columns and 'label' in df.columns:
    if df['label'].dtype in ['int64', 'float64']:
        label_map = {0: "negative", 1: "neutral", 2: "positive"}
        df['sentiment'] = df['label'].map(label_map)
    else:
        df = df.rename(columns={'label': 'sentiment'})

if 'text' not in df.columns or 'sentiment' not in df.columns:
    print("❌ Dataset must have 'text' and 'sentiment' columns")
    print(f"   Available columns: {df.columns.tolist()}")
    return None

if max_samples and len(df) > max_samples:
    df = df.sample(max_samples, random_state=42)
    print(f"Using random subset of {max_samples} samples")

print(f"Loaded {len(df)} samples")
print(f"Classes: {df['sentiment'].unique()}")
print(f"Class distribution:\n{df['sentiment'].value_counts()}")

# ============================================================
# TEST OLLAMA CONNECTION
# ============================================================

print("\nTesting Ollama connection...")
try:
    test_response = ollama.chat(
        model=model_name,
```

```
            messages=[{"role": "user", "content": "Hello"}],
            options={"temperature": 0}
        )
        print(f"✅ Ollama is running with {model_name}!")
    except Exception as e:
        print(f"❌ Ollama error: {e}")
        print("Please make sure Ollama is running with: ollama serve")
        return None

    # ============================================================
    # RUN EXPERIMENTS (Only for specified techniques)
    # ============================================================

    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    dataset_name = os.path.splitext(os.path.basename(dataset_file))[0]

    all_results = []

    for technique in techniques:
        if technique == "self_consistency":
            results = run_experiment_self_consistency(
                df, technique, PROMPT_TEMPLATES[technique], model_name
            )
        else:
            results = run_experiment_single(
                df, technique, PROMPT_TEMPLATES[technique], model_name
            )

        all_results.append(results)

        tech_file                                                    =
f"{output_prefix}_{timestamp}_{dataset_name}_{technique}.csv"
        results.to_csv(tech_file, index=False)
        print(f"   Saved to {tech_file}")

    # ============================================================
    # COMBINE AND SAVE FINAL RESULTS
    # ============================================================

    final_df = pd.concat(all_results, ignore_index=True)
    column_order = ['statement', 'true_sentiment', 'predicted_sentiment',
'technique', 'raw_response']
    final_df = final_df[column_order]

    final_file = f"{output_prefix}_{timestamp}_{dataset_name}_final.csv"
    final_df.to_csv(final_file, index=False)
```

```
        print(f"\n All experiments complete!")
        print(f" Final results saved to {final_file}")
        print(f"Total runs: {len(final_df)}")

        # ============================================================
        # SUMMARY
        # ============================================================

        print("\n" + "="*60)
        print("QUICK SUMMARY - Accuracy per technique:")
        print("="*60)

        summary_data = []
        for technique in techniques:
            subset = final_df[final_df['technique'] == technique]
            correct = (subset['true_sentiment'] ==
subset['predicted_sentiment']).sum()
            total = len(subset)
            accuracy = correct / total * 100
            print(f"{technique}: {accuracy:.1f}% ({correct}/{total})")
            summary_data.append({
                'technique': technique,
                'correct': correct,
                'total': total,
                'accuracy': accuracy
            })

        summary_df = pd.DataFrame(summary_data)
        summary_file =
f"{output_prefix}_{timestamp}_{dataset_name}_summary.csv"
        summary_df.to_csv(summary_file, index=False)
        print(f"\n Summary saved to {summary_file}")

        return final_df

    # ============================================================
    # COMMAND LINE INTERFACE
    # ============================================================

    if __name__ == "__main__":
        parser = argparse.ArgumentParser(description="Run LLM prompt
engineering experiments")
        parser.add_argument("--model", "-m", help="Ollama model name")
        parser.add_argument("--dataset", "-d", help="Dataset CSV file path")
        parser.add_argument("--output", "-o", help="Output file prefix")
        parser.add_argument("--max", "-n", type=int, help="Maximum number of
samples to use")
```

```
    parser.add_argument("--techniques", "-t", nargs="+",
                        choices=["zero_shot", "few_shot", "self_consistency",
"hybrid"],
                        help="Which techniques to run (space separated)")

    args = parser.parse_args()

    # Override config with command line arguments
    model_name = args.model if args.model else CONFIG["model_name"]
    dataset_file = args.dataset if args.dataset else CONFIG["dataset_file"]
    output_prefix = args.output if args.output else CONFIG["output_prefix"]
    max_samples = args.max if args.max is not None else
CONFIG["max_samples"]
    techniques = args.techniques if args.techniques else
CONFIG["techniques"]

    run_experiment(
      model_name=model_name,
      dataset_file=dataset_file,
      output_prefix=output_prefix,
      max_samples=max_samples,
      techniques=techniques
    )
```

11.4 evaluate_results.py

```
import pandas as pd
import numpy as np
import os
import re
from sklearn.metrics import (
  precision_recall_fscore_support,
  confusion_matrix,
  accuracy_score
)
from statsmodels.stats.contingency_tables import mcnemar
import matplotlib.pyplot as plt
import seaborn as sns
from itertools import combinations
import argparse
from datetime import datetime

# ============================================================
# CONFIGURATION
# ============================================================
```

```
PROMPT_TEMPLATES = {
   "zero_shot": """
You are a financial sentiment analyst. Classify the sentiment of the following financial news text as 'positive', 'negative', or 'neutral'.

Text: {text}

Sentiment:""",

   "few_shot": """
You are a financial sentiment analyst. Classify the sentiment of the following financial news text as 'positive', 'negative', or 'neutral'.

Examples:
Text: "The company reported record profits this quarter."
Sentiment: positive

Text: "The stock market crashed due to inflation fears."
Sentiment: negative

Text: "The board announced a routine quarterly review."
Sentiment: neutral

Now classify this text:
Text: {text}
Sentiment:""",

   "self_consistency": """
You are a financial sentiment analyst. Classify the sentiment of the following financial news text as 'positive', 'negative', or 'neutral'.

Text: {text}

Sentiment:""",

   "hybrid": """
You are a financial sentiment analyst. You have been given the following instructions from the dataset annotators:

Instructions for annotators:
- Sentiment is the opinion or feeling expressed in the text towards the financial entity.
- Positive: The text expresses a favorable or optimistic view.
- Negative: The text expresses an unfavorable or pessimistic view.
- Neutral: The text expresses no clear opinion or contains only factual information.
```

```
Examples:
Text: "The company reported record profits this quarter."
Sentiment: positive

Text: "The stock market crashed due to inflation fears."
Sentiment: negative

Text: "The board announced a routine quarterly review."
Sentiment: neutral

Now, carefully analyze this text and classify its sentiment as 'positive', 'negative', or 'neutral' using the instructions above.

Text: {text}

Sentiment:"""
}

DATASET_MAP = {
    'fp50': 'FinancialPhraseBank_50Agree',
    'fp66': 'FinancialPhraseBank_66Agree',
    'fp75': 'FinancialPhraseBank_75Agree',
    'fp100': 'FinancialPhraseBank_100Agree',
    'fiqa': 'FiQA'
}

# ============================================================
# HELPER FUNCTIONS
# ============================================================

def parse_filename(filename):
    """Parse filename to extract model, dataset, technique."""
    base = os.path.splitext(filename)[0]
    parts = base.split('_')
    techniques = ['zero_shot', 'few_shot', 'self_consistency', 'hybrid']
    tech = None
    for t in techniques:
        if base.endswith('_' + t):
            tech = t
            base = base[:-len(t)-1]
            break
    if tech is None:
        raise ValueError(f"Could not parse technique from {filename}")

    datasets = ['fp50', 'fp66', 'fp75', 'fp100', 'fiqa']
    ds = None
    for d in datasets:
```

```
            if base.endswith('_' + d):
                ds = d
                base = base[:-len(d)-1]
                break
        if ds is None:
            raise ValueError(f"Could not parse dataset from {filename}")

        model = base
        return model, ds, tech

    def load_all_results(directory):
        """Load all CSV files from directory and combine into a single DataFrame."""
        all_files = [f for f in os.listdir(directory) if f.endswith('.csv')]
        data = []
        for f in all_files:
            try:
                model, dataset, technique = parse_filename(f)
            except Exception as e:
                print(f"Skipping {f}: {e}")
                continue
            df = pd.read_csv(os.path.join(directory, f))
            df['model'] = model
            df['dataset'] = dataset
            df['technique'] = technique
            data.append(df)
        if not data:
            raise ValueError("No valid result files found.")
        return pd.concat(data, ignore_index=True)

    def compute_metrics(df):
        """Compute precision, recall, f1 per class, macro, weighted, accuracy, confusion matrix."""
        labels = ['positive', 'negative', 'neutral']
        y_true = df['true_sentiment']
        y_pred = df['predicted_sentiment']

        precision, recall, f1, support = precision_recall_fscore_support(
            y_true, y_pred, labels=labels, average=None, zero_division=0
        )
        macro_f1 = precision_recall_fscore_support(
            y_true, y_pred, labels=labels, average='macro', zero_division=0
        )[2]
        weighted_f1 = precision_recall_fscore_support(
            y_true, y_pred, labels=labels, average='weighted', zero_division=0
        )[2]
        accuracy = accuracy_score(y_true, y_pred)
```

```
    cm = confusion_matrix(y_true, y_pred, labels=labels)

    metrics = {
        'accuracy': accuracy,
        'precision': dict(zip(labels, precision)),
        'recall': dict(zip(labels, recall)),
        'f1': dict(zip(labels, f1)),
        'support': dict(zip(labels, support)),
        'macro_f1': macro_f1,
        'weighted_f1': weighted_f1,
        'confusion_matrix': cm,
        'labels': labels
    }
    return metrics

def format_compliance(df):
    """This section is redundant and to be removed."""
    def is_compliant(text):
        if pd.isna(text):
            return False
        t = str(text).strip().lower()
        if t in ['positive', 'negative', 'neutral']:
            return True
        if re.match(r'^(positive|negative|neutral)[.!]?$', t):
            return True
        words = t.split()
        if len(words) <= 10:
            if 'positive' in t or 'negative' in t or 'neutral' in t:
                return True
        return False

    df['compliant'] = df['raw_response'].apply(is_compliant)
    return df['compliant'].mean() * 100

def token_efficiency(technique):
    """Compute average prompt length for a given technique."""
    template = PROMPT_TEMPLATES.get(technique)
    if template is None:
        return np.nan
    sample_text = "The company reported record profits."
    prompt = template.format(text=sample_text)
    return len(prompt) / 4

def count_tokens(text):
    """Approximate token count using 4 characters per token."""
    if text is None or pd.isna(text):
```

```
            return 0
        return len(str(text)) / 4

    def aggregate_by(df, group_cols):
        """Aggregate metrics by given columns."""
        results = []
        for name, group in df.groupby(group_cols):
            if not isinstance(name, tuple):
                name = (name,)
            metrics = compute_metrics(group)
            compliant = format_compliance(group)

            # Get technique for token efficiency if available
            tech = name[-1] if 'technique' in group_cols else None
            if tech is not None:
                # Input tokens (from prompt template)
                input_tokens = token_efficiency(tech)

                # Output tokens (from raw responses)
                avg_output_tokens =
group['raw_response'].apply(count_tokens).mean()

                # Token efficiency ratio: output / (input + output) * 100
                if input_tokens + avg_output_tokens > 0:
                    token_eff_ratio = avg_output_tokens / (input_tokens +
avg_output_tokens) * 100
                else:
                    token_eff_ratio = 0
            else:
                input_tokens = np.nan
                avg_output_tokens = np.nan
                token_eff_ratio = np.nan

            row = {
                'group': ' | '.join(str(n) for n in name),
                **{k: name[i] for i, k in enumerate(group_cols)},
                'accuracy': metrics['accuracy'],
                'precision_positive': metrics['precision']['positive'],
                'precision_negative': metrics['precision']['negative'],
                'precision_neutral': metrics['precision']['neutral'],
                'recall_positive': metrics['recall']['positive'],
                'recall_negative': metrics['recall']['negative'],
                'recall_neutral': metrics['recall']['neutral'],
                'f1_positive': metrics['f1']['positive'],
                'f1_negative': metrics['f1']['negative'],
                'f1_neutral': metrics['f1']['neutral'],
                'macro_f1': metrics['macro_f1'],
```

```
                'weighted_f1': metrics['weighted_f1'],
                'format_compliance': compliant,
                'input_tokens': input_tokens,
                'avg_output_tokens': avg_output_tokens,
                'token_efficiency_ratio': token_eff_ratio,
                'support_positive': metrics['support']['positive'],
                'support_negative': metrics['support']['negative'],
                'support_neutral': metrics['support']['neutral'],
                'confusion_matrix': metrics['confusion_matrix'],
                'labels': metrics['labels']
            }
            results.append(row)
        return pd.DataFrame(results)

    def statistical_tests(df, model, dataset):
        """Perform McNemar tests for techniques within a model-dataset."""

        # Skip McNemar for fp66 and fp100
        if dataset in ['fp66', 'fp100']:
            return []

        sub = df[(df['model'] == model) & (df['dataset'] == dataset)]
        techniques = sub['technique'].unique()
        if len(techniques) < 2:
            return []

        pivot = sub.pivot_table(index='statement', columns='technique',
values='predicted_sentiment', aggfunc='first')
        true_vals = sub.groupby('statement')['true_sentiment'].first()

        correct_matrix = pd.DataFrame(index=pivot.index)
        for tech in techniques:
            correct_matrix[tech] = (pivot[tech] == true_vals).astype(int)

        pairwise_results = []
        for tech_a, tech_b in combinations(techniques, 2):
            a = ((correct_matrix[tech_a] == 1) & (correct_matrix[tech_b] == 1)).sum()
            b = ((correct_matrix[tech_a] == 1) & (correct_matrix[tech_b] == 0)).sum()
            c = ((correct_matrix[tech_a] == 0) & (correct_matrix[tech_b] == 1)).sum()
            d = ((correct_matrix[tech_a] == 0) & (correct_matrix[tech_b] == 0)).sum()
            table = np.array([[a, b], [c, d]])
            try:
                result = mcnemar(table, exact=False, correction=True)
                p = result.pvalue
            except:
                p = np.nan
            pairwise_results.append({
```

```
                'technique_a': tech_a,
                'technique_b': tech_b,
                'p_value': p,
                'significant': p < 0.05 if not np.isnan(p) else False
            })
        return pairwise_results

    def model_comparison_tests(df, dataset, technique):
        """
        Perform McNemar's test comparing Fin-R1 vs Llama for a specific technique on a specific dataset.
        """

        # Skip model comparison for fp66 and fp100 (only Llama data available)
        if dataset in ['fp66', 'fp100']:
            return None

        # Filter for the specific dataset and technique
        sub = df[(df['dataset'] == dataset) & (df['technique'] == technique)]

        # Get predictions for each model
        fin_r1 = sub[sub['model'] == 'Fin-R1_GGUF']
        llama = sub[sub['model'] == 'llama3.2-abliterate']

        # Ensure we have data for both models
        if fin_r1.empty or llama.empty:
            return None

        # Merge on the statement to align paired observations
        merged = pd.merge(
            fin_r1[['statement', 'true_sentiment', 'predicted_sentiment']],
            llama[['statement', 'predicted_sentiment']],
            on='statement',
            suffixes=('_fin', '_llama')
        )

        # Calculate correctness
        fin_correct = (merged['predicted_sentiment_fin'] == merged['true_sentiment'])
        llama_correct = (merged['predicted_sentiment_llama'] == merged['true_sentiment'])

        # Build contingency table for McNemar
        # a: both correct, b: fin correct, llama wrong
        # c: fin wrong, llama correct, d: both wrong
        a = ((fin_correct == True) & (llama_correct == True)).sum()
        b = ((fin_correct == True) & (llama_correct == False)).sum()
```

```
            c = ((fin_correct == False) & (llama_correct == True)).sum()
            d = ((fin_correct == False) & (llama_correct == False)).sum()

            table = np.array([[a, b], [c, d]])

            try:
                result = mcnemar(table, exact=False, correction=True)
                return {
                    'dataset': dataset,
                    'technique': technique,
                    'fin_correct': fin_correct.sum(),
                    'llama_correct': llama_correct.sum(),
                    'a_both_correct': a,
                    'b_fin_only': b,
                    'c_llama_only': c,
                    'd_both_wrong': d,
                    'p_value': result.pvalue,
                    'significant': result.pvalue < 0.05
                }
            except:
                return None

        def plot_confusion_matrix(cm, labels, title, save_path):
            plt.figure(figsize=(8, 6))
            sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                        xticklabels=labels, yticklabels=labels)
            plt.title(title)
            plt.ylabel('True')
            plt.xlabel('Predicted')
            plt.tight_layout()
            plt.savefig(save_path)
            plt.close()

        def plot_accuracy_comparison(agg_df, group_by_col, output_dir):
            """Plot accuracy comparison bar charts."""

            # For technique level
            if group_by_col == 'technique':
                # For technique_agg, we already have technique as the main column
                # show accuracy for each technique grouped by model/dataset
                # bar chart of accuracy per technique
                plt.figure(figsize=(10, 6))
                avg_accuracy =
agg_df.groupby('technique')['accuracy'].mean().sort_values(ascending=False)
                avg_accuracy.plot(kind='bar', color=['#4ECDC4', '#45B7D1', '#FFA502',
'#FF6B6B'])
                plt.title('Average Accuracy by Technique')
```

```
            plt.ylabel('Accuracy')
            plt.xlabel('Technique')
            plt.ylim(0, 1)
            for i, v in enumerate(avg_accuracy):
                plt.text(i, v + 0.02, f'{v:.3f}', ha='center', fontsize=10)
            plt.tight_layout()
            plt.savefig(os.path.join(output_dir, f'accuracy_{group_by_col}.png'))
            plt.close()
            return

        # For dataset and model levels
        # Aggregate by group_by_col and technique, taking mean accuracy
        agg_plot_df = agg_df.groupby([group_by_col, 'technique'])['accuracy'].mean().reset_index()

        # Pivot: rows = group_by_col values, columns = techniques
        pivot_df = agg_plot_df.pivot(index=group_by_col, columns='technique', values='accuracy')

        if pivot_df.empty:
            print(f"Warning: No data for {group_by_col} pivot")
            return

        plt.figure(figsize=(12, 6))
        pivot_df.plot(kind='bar', ax=plt.gca())
        plt.title(f'Accuracy Comparison by {group_by_col.capitalize()}')
        plt.ylabel('Accuracy')
        plt.xlabel(group_by_col.capitalize())
        plt.legend(title='Technique', bbox_to_anchor=(1.05, 1), loc='upper left')
        plt.tight_layout()
        plt.savefig(os.path.join(output_dir, f'accuracy_{group_by_col}.png'))
        plt.close()

    def plot_model_performance_per_dataset(df, output_dir):
        """
        Plot model performance (accuracy) per dataset, grouped by technique.
        """
        # Compute accuracy for each model-dataset-technique combination
        results = []

        for (model, dataset, technique), group in df.groupby(['model', 'dataset', 'technique']):
            metrics = compute_metrics(group)
            results.append({
                'model': model,
                'dataset': dataset,
                'technique': technique,
```

```
                'accuracy': metrics['accuracy']
            })

        agg_df = pd.DataFrame(results)

        # Pivot: rows = dataset, columns = model, values = accuracy (averaged over techniques)
        pivot_df = agg_df.pivot_table(index='dataset', columns='model', values='accuracy', aggfunc='mean')

        if pivot_df.empty:
            print("Warning: No data for model performance per dataset plot")
            return

        plt.figure(figsize=(12, 6))
        pivot_df.plot(kind='bar', ax=plt.gca())
        plt.title('Model Performance by Dataset (Average Accuracy Across Techniques)')
        plt.ylabel('Accuracy')
        plt.xlabel('Dataset')
        plt.legend(title='Model', bbox_to_anchor=(1.05, 1), loc='upper left')
        plt.ylim(0, 1)

        # Add value labels on top of bars
        for container in plt.gca().containers:
            plt.gca().bar_label(container, fmt='%.3f', padding=3, fontsize=9)

        plt.tight_layout()
        plt.savefig(os.path.join(output_dir, 'model_performance_per_dataset.png'))
        plt.close()

        print("  Saved model_performance_per_dataset.png")

    # ============================================================
    # MAIN
    # ============================================================

    def main():
        parser = argparse.ArgumentParser(description='Evaluate prompt engineering experiment results')
        parser.add_argument('--input_dir', required=True, help='Directory containing result CSV files')
        parser.add_argument('--output_dir', default='evaluation_output', help='Output directory for results')
        args = parser.parse_args()
```

```
    os.makedirs(args.output_dir, exist_ok=True)

    # Load all data
    print("Loading result files...")
    df = load_all_results(args.input_dir)
    print(f"Loaded {len(df)} samples from {df['model'].nunique()} models,
{df['dataset'].nunique()} datasets, {df['technique'].nunique()} techniques.")

    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")

    # ============================================================
    # LEVEL 1: Test Level
    # ============================================================
    print("Computing test-level metrics...")
    test_agg = aggregate_by(df, ['model', 'dataset', 'technique'])
    test_agg.to_csv(os.path.join(args.output_dir,
f'test_level_metrics_{timestamp}.csv'), index=False)

    # ============================================================
    # LEVEL 2: Dataset Level
    # ============================================================
    print("Computing dataset-level metrics...")
    dataset_agg = aggregate_by(df, ['dataset', 'technique'])
    dataset_agg.to_csv(os.path.join(args.output_dir,
f'dataset_level_metrics_{timestamp}.csv'), index=False)

    dataset_overall = aggregate_by(df, ['dataset'])
    dataset_overall.to_csv(os.path.join(args.output_dir,
f'dataset_overall_metrics_{timestamp}.csv'), index=False)

    # ============================================================
    # LEVEL 3: Model Level
    # ============================================================
    print("Computing model-level metrics...")

    # FILTER BEFORE AGGREGATION: Exclude fp66 and fp100
    df_model_filtered = df[~df['dataset'].isin(['fp66', 'fp100'])]

    model_agg = aggregate_by(df, ['model', 'technique'])
    model_agg.to_csv(os.path.join(args.output_dir,
f'model_level_metrics_{timestamp}.csv'), index=False)

    model_overall = aggregate_by(df, ['model'])
    model_overall.to_csv(os.path.join(args.output_dir,
f'model_overall_metrics_{timestamp}.csv'), index=False)

    # ============================================================
```

```
            # LEVEL 4: Technique Level
            # ============================================================
            print("Computing technique-level metrics...")
            technique_agg = aggregate_by(df, ['technique'])
            technique_agg.to_csv(os.path.join(args.output_dir,
f'technique_level_metrics_{timestamp}.csv'), index=False)

            # ============================================================
            # OVERALL
            # ============================================================
            print("Computing overall metrics...")
            overall_metrics = compute_metrics(df)
            overall_input_tokens = np.nan
            overall_avg_output_tokens                                    =
df['raw_response'].apply(count_tokens).mean()
            overall_token_eff_ratio = np.nan
            overall_row = {
               'accuracy': overall_metrics['accuracy'],
               'macro_f1': overall_metrics['macro_f1'],
               'weighted_f1': overall_metrics['weighted_f1'],
               'precision_positive': overall_metrics['precision']['positive'],
               'precision_negative': overall_metrics['precision']['negative'],
               'precision_neutral': overall_metrics['precision']['neutral'],
               'recall_positive': overall_metrics['recall']['positive'],
               'recall_negative': overall_metrics['recall']['negative'],
               'recall_neutral': overall_metrics['recall']['neutral'],
               'f1_positive': overall_metrics['f1']['positive'],
               'f1_negative': overall_metrics['f1']['negative'],
               'f1_neutral': overall_metrics['f1']['neutral'],
               'support_positive': overall_metrics['support']['positive'],
               'support_negative': overall_metrics['support']['negative'],
               'support_neutral': overall_metrics['support']['neutral'],
               'avg_output_tokens': overall_avg_output_tokens,
            }
            overall_df = pd.DataFrame([overall_row])
            overall_df.to_csv(os.path.join(args.output_dir,
f'overall_metrics_{timestamp}.csv'), index=False)

            # ============================================================
            # STATISTICAL TESTS: Prompting technique McNemar's test
            # ============================================================
            print("Performing statistical tests...")
            all_tests = []
            for model in df['model'].unique():
               for dataset in df['dataset'].unique():
                  tests = statistical_tests(df, model, dataset)
                  for pair in tests:
```

```
                    all_tests.append({
                        'model': model,
                        'dataset': dataset,
                        'technique_a': pair['technique_a'],
                        'technique_b': pair['technique_b'],
                        'p_value': pair['p_value'],
                        'significant': pair['significant']
                    })
            stats_df = pd.DataFrame(all_tests)
            if not stats_df.empty:
                stats_df.to_csv(os.path.join(args.output_dir,
f'statistical_tests_{timestamp}.csv'), index=False)
            else:
                print("No statistical tests performed (insufficient data).")

            # ============================================================
            # STATISTICAL TESTS: MODEL COMPARISON McNemar's test
            # ============================================================
            print("\nRunning model comparison tests (Fin-R1 vs Llama)...")
            model_test_results = []

            for dataset in df['dataset'].unique():
                for technique in df['technique'].unique():
                    result = model_comparison_tests(df, dataset, technique)
                    if result:
                        model_test_results.append(result)

            if model_test_results:
                model_test_df = pd.DataFrame(model_test_results)
                model_test_df.to_csv(os.path.join(args.output_dir,
f'model_comparison_tests_{timestamp}.csv'), index=False)
                print(f"Model comparison tests saved.")

            else:
                print("No model comparison tests performed.")

            # ============================================================
            # CONFUSION MATRICES
            # ============================================================
            print("Generating confusion matrices...")
            cm_dir = os.path.join(args.output_dir, 'confusion_matrices')
            os.makedirs(cm_dir, exist_ok=True)

            for (model, dataset, technique), group in df.groupby(['model', 'dataset',
'technique']):
                if len(group) == 0:
                    continue
```

```
                metrics = compute_metrics(group)
                cm = metrics['confusion_matrix']
                labels = metrics['labels']
                title = f"{model}_{dataset}_{technique}"
                plot_confusion_matrix(cm, labels, title, os.path.join(cm_dir,
f'cm_{title}.png'))

            for dataset, group in df.groupby(['dataset']):
                metrics = compute_metrics(group)
                cm = metrics['confusion_matrix']
                labels = metrics['labels']
                title = f"Dataset_{dataset}"
                plot_confusion_matrix(cm, labels, title, os.path.join(cm_dir,
f'cm_dataset_{dataset}.png'))

            for model, group in df.groupby(['model']):
                if model == 'llama3.2-abliterate':
                    group = group[~group['dataset'].isin(['fp66', 'fp100'])]
                if not group.empty:
                    metrics = compute_metrics(group)
                    cm = metrics['confusion_matrix']
                    labels = metrics['labels']
                    title = f"Model_{model}"
                    plot_confusion_matrix(cm, labels, title, os.path.join(cm_dir,
f'cm_model_{model}.png'))

            for technique, group in df.groupby(['technique']):
                metrics = compute_metrics(group)
                cm = metrics['confusion_matrix']
                labels = metrics['labels']
                title = f"Technique_{technique}"
                plot_confusion_matrix(cm, labels, title, os.path.join(cm_dir,
f'cm_technique_{technique}.png'))

            metrics = compute_metrics(df)
            cm = metrics['confusion_matrix']
            labels = metrics['labels']
            plot_confusion_matrix(cm, labels, "Overall", os.path.join(cm_dir,
'cm_overall.png'))

            # ============================================================
            # ACCURACY BAR CHARTS
            # ============================================================
            print("Generating accuracy bar charts...")

            # Dataset level
            plot_accuracy_comparison(dataset_agg, 'dataset', args.output_dir)
```

```
            # Model level
            plot_accuracy_comparison(model_agg, 'model', args.output_dir)

            # Technique level
            plot_accuracy_comparison(technique_agg, 'technique', args.output_dir)

            # NEW: Model performance per dataset
            print("Generating model performance per dataset graph...")
            plot_model_performance_per_dataset(df, args.output_dir)

            # ============================================================
            # GENERATE REPORT
            # ============================================================
            print("Generating report...")

            report_lines = []
            report_lines.append("="*80)
            report_lines.append("EVALUATION REPORT - Prompt Engineering
Experiments")
            report_lines.append(f"Generated: {datetime.now().strftime('%Y-%m-%d
%H:%M:%S')}")
            report_lines.append("="*80)

            report_lines.append("\nOVERALL PERFORMANCE")
            report_lines.append("-"*40)
            report_lines.append(f"Accuracy: {overall_row['accuracy']:.4f}")
            report_lines.append(f"Macro F1: {overall_row['macro_f1']:.4f}")
            report_lines.append(f"Weighted F1: {overall_row['weighted_f1']:.4f}")
            report_lines.append("\nPer-class metrics:")
            for label in ['positive', 'negative', 'neutral']:
                report_lines.append(f"  {label.capitalize()}:
Precision={overall_row[f'precision_{label}']:.4f},
Recall={overall_row[f'recall_{label}']:.4f}, F1={overall_row[f'f1_{label}']:.4f}")
                report_lines.append("\nTOKEN EFFICIENCY (Input & Output)")
                report_lines.append("-"*40)
                report_lines.append(f"Average Output Tokens (overall):
{overall_avg_output_tokens:.1f}")
                report_lines.append("\nPer-Technique Token Efficiency:")
                for tech in technique_agg['technique'].unique():
                    subset = technique_agg[technique_agg['technique'] == tech]
                    if not subset.empty:
                        row = subset.iloc[0]
                        report_lines.append(f"  {tech}: Input={row['input_tokens']:.1f}
tokens, Output={row['avg_output_tokens']:.1f} tokens,
Ratio={row['token_efficiency_ratio']:.1f}%")
```

```
        report_lines.append("\nMODEL-LEVEL PERFORMANCE")
        report_lines.append("-"*40)
        for _, row in model_overall.iterrows():
            report_lines.append(f"\nModel: {row['group']}")
            report_lines.append(f"  Accuracy: {row['accuracy']:.4f}")
            report_lines.append(f"  Macro F1: {row['macro_f1']:.4f}")

        report_lines.append("\nDATASET-LEVEL PERFORMANCE")
        report_lines.append("-"*40)
        for _, row in dataset_overall.iterrows():
            report_lines.append(f"\nDataset: {row['group']}")
            report_lines.append(f"  Accuracy: {row['accuracy']:.4f}")
            report_lines.append(f"  Macro F1: {row['macro_f1']:.4f}")

        report_lines.append("\nTECHNIQUE-LEVEL PERFORMANCE")
        report_lines.append("-"*40)
        for _, row in technique_agg.iterrows():
            report_lines.append(f"\nTechnique: {row['group']}")
            report_lines.append(f"  Accuracy: {row['accuracy']:.4f}")
            report_lines.append(f"  Macro F1: {row['macro_f1']:.4f}")

        if not stats_df.empty:
            report_lines.append("\nSTATISTICAL SIGNIFICANCE TESTS (McNemar's)")
            report_lines.append("-"*40)
            for _, row in stats_df.iterrows():
                sig = "YES" if row['significant'] else "NO"
                report_lines.append(f"{row['model']} | {row['dataset']} | {row['technique_a']} vs {row['technique_b']} | p={row['p_value']:.4f} | Significant: {sig}")

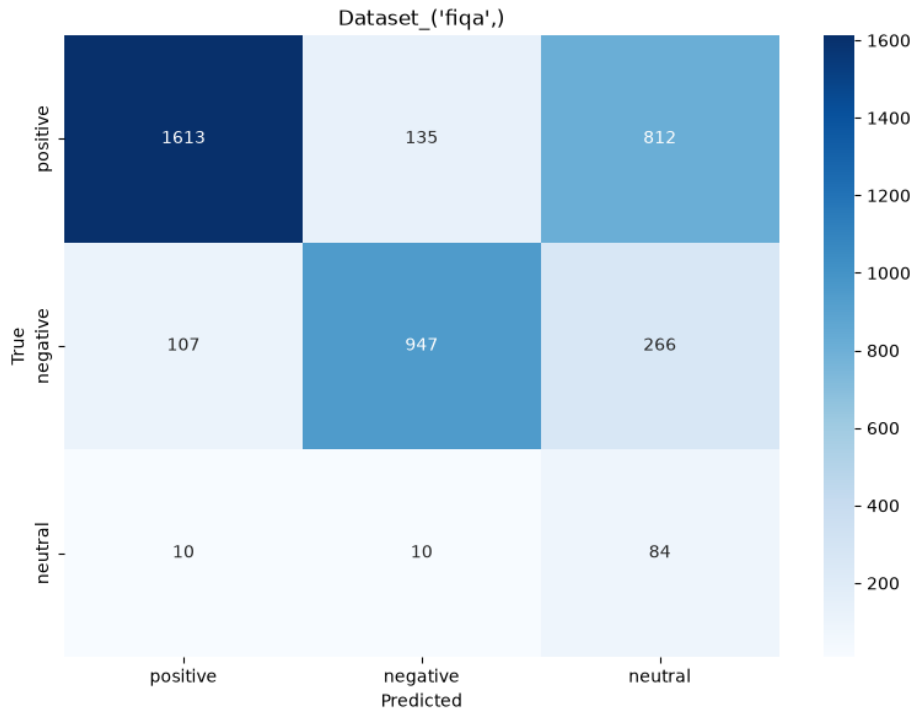
        report_path = os.path.join(args.output_dir, f'report_{timestamp}.txt')
        with open(report_path, 'w') as f:
            f.write('\n'.join(report_lines))

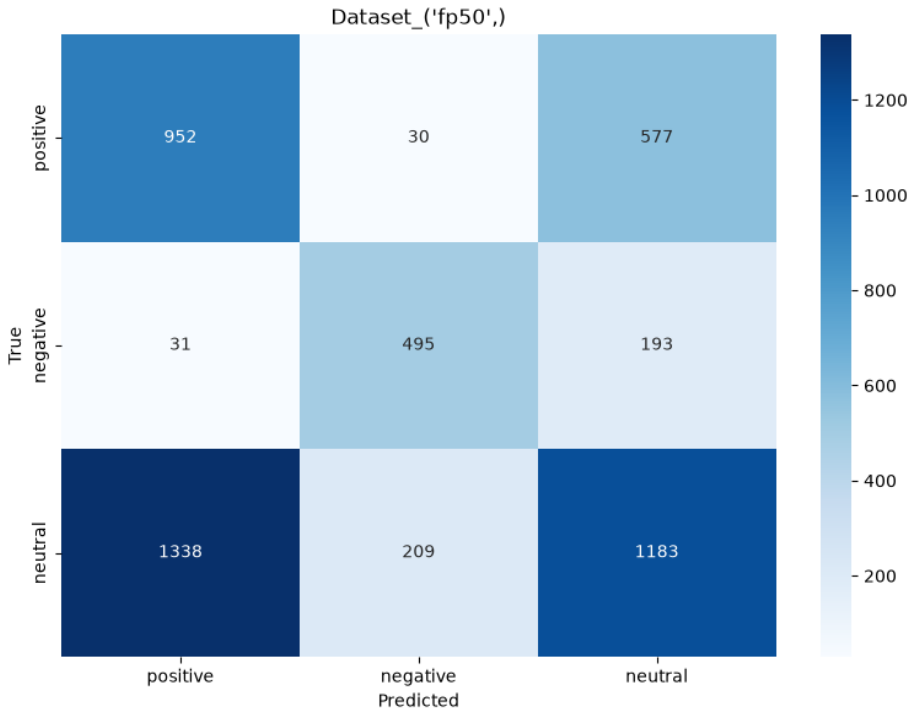
        print(f"\n Evaluation complete! Results saved to {args.output_dir}")
        print(f"Report: {report_path}")
        print("Confusion matrices and plots are also saved.")

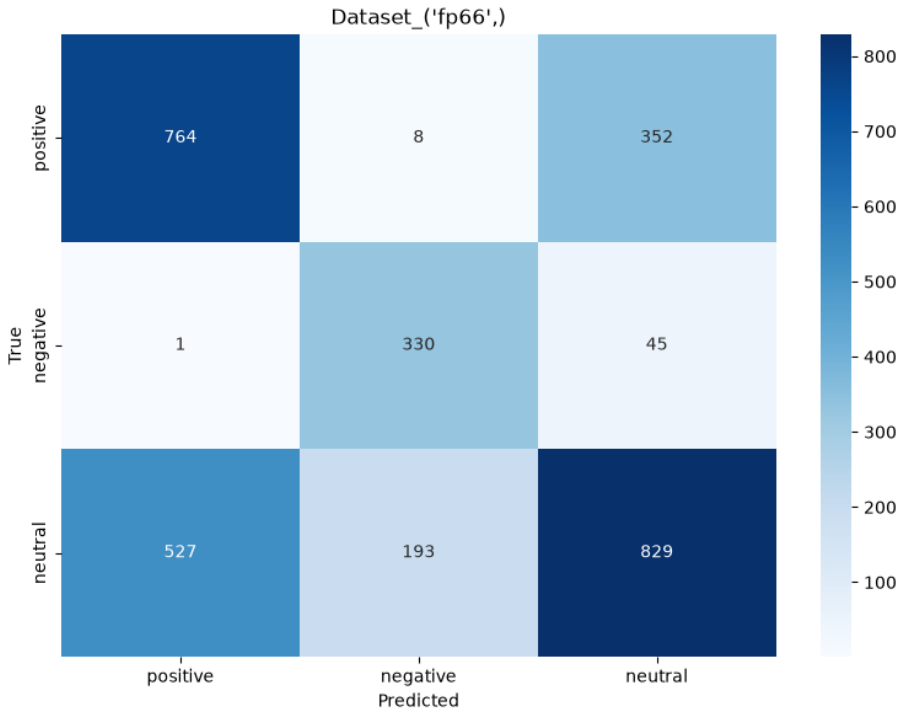
    if __name__ == "__main__":
        main()
```

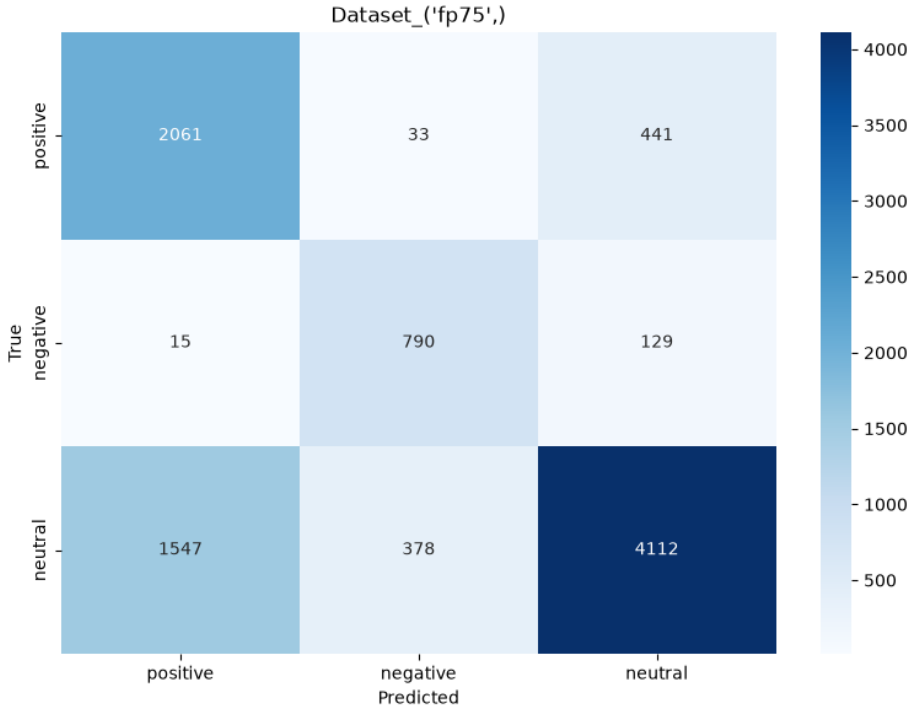
12. Appendix B

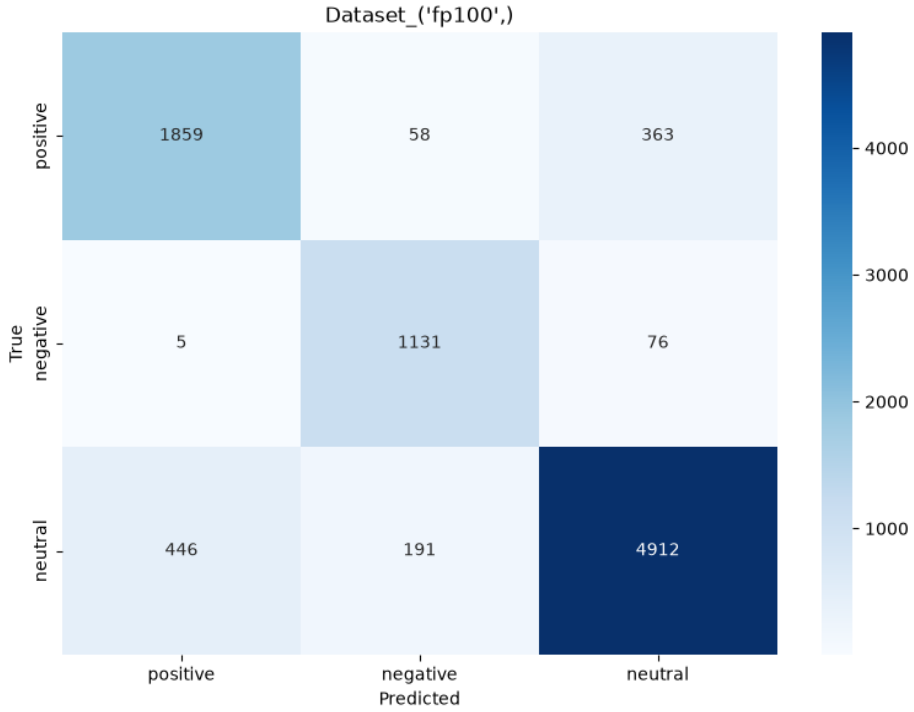
This section contains all the Confusion Matrices corresponding to all the tests performed in this study.

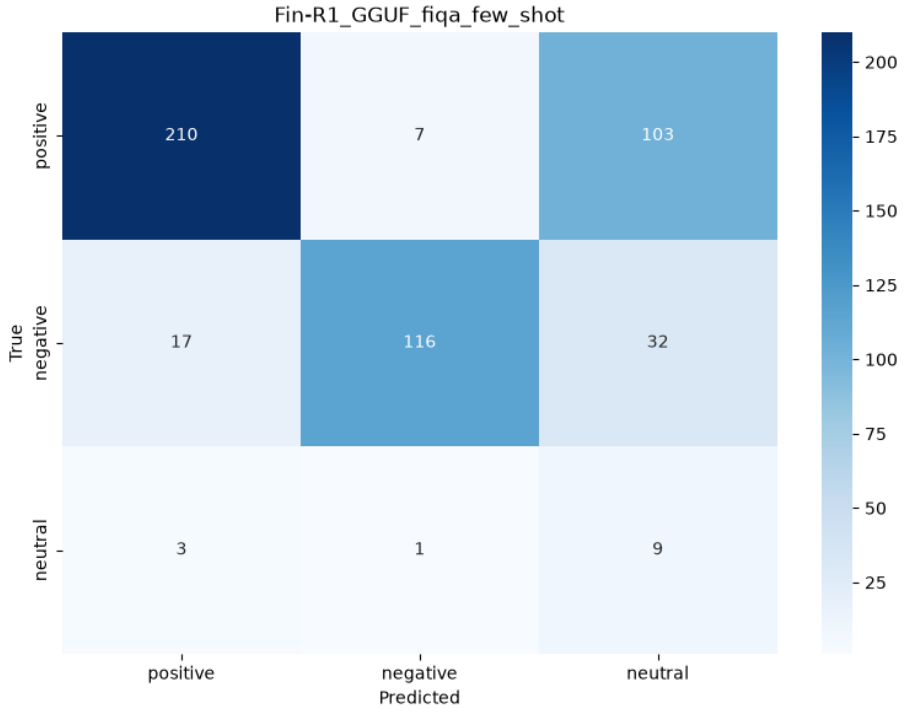


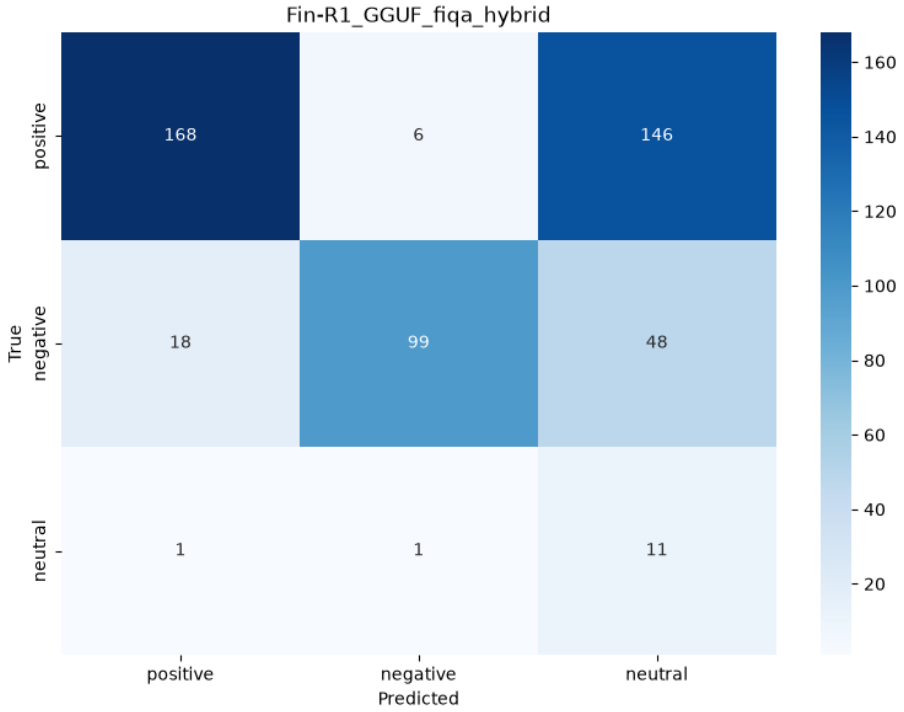

Dataset_('fp50',)
positive
negative
neutral
True
952
30
577
31
495
193
1338
209
1183
positive
negative
neutral
Predicted
1200
1000
800
600
400
200

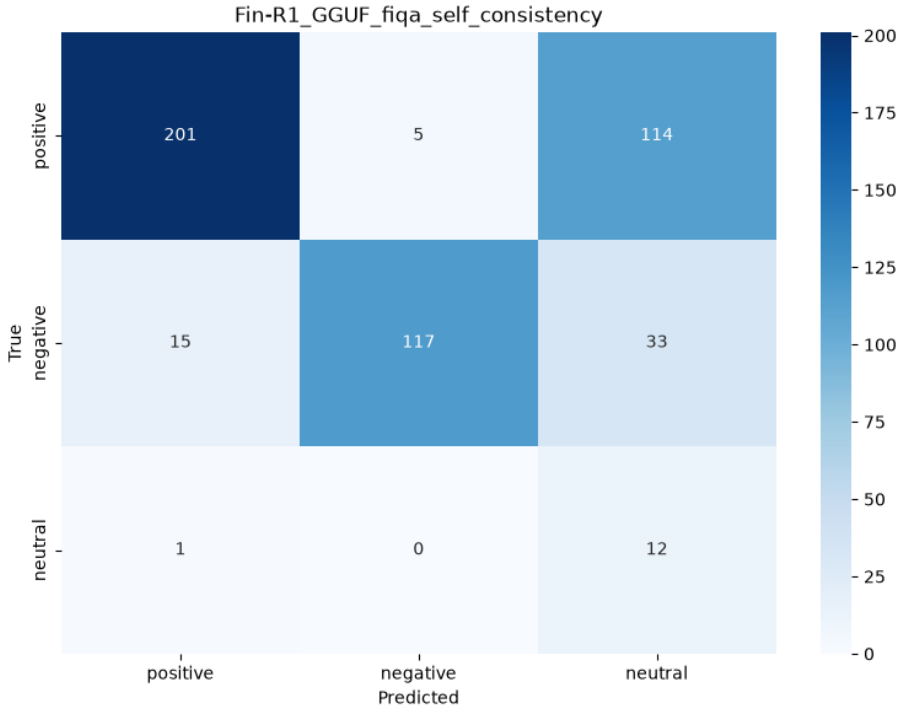

Dataset_('fp66',)
positive
negative
neutral
True
764
8
352
1
330
45
527
193
829
positive
negative
neutral
Predicted
800
700
600
500
400
300
200
100

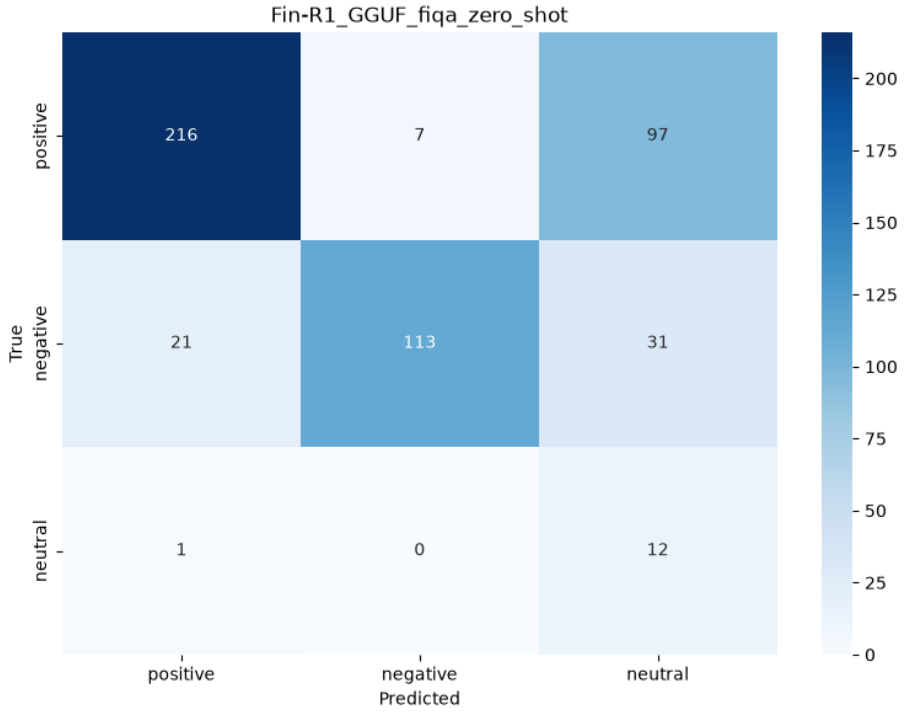

Dataset_('fp75',)
positive
negative
neutral
True
2061
33
441
15
790
129
1547
378
4112
positive
negative
neutral
Predicted
4000
3500
3000
2500
2000
1500
1000
500

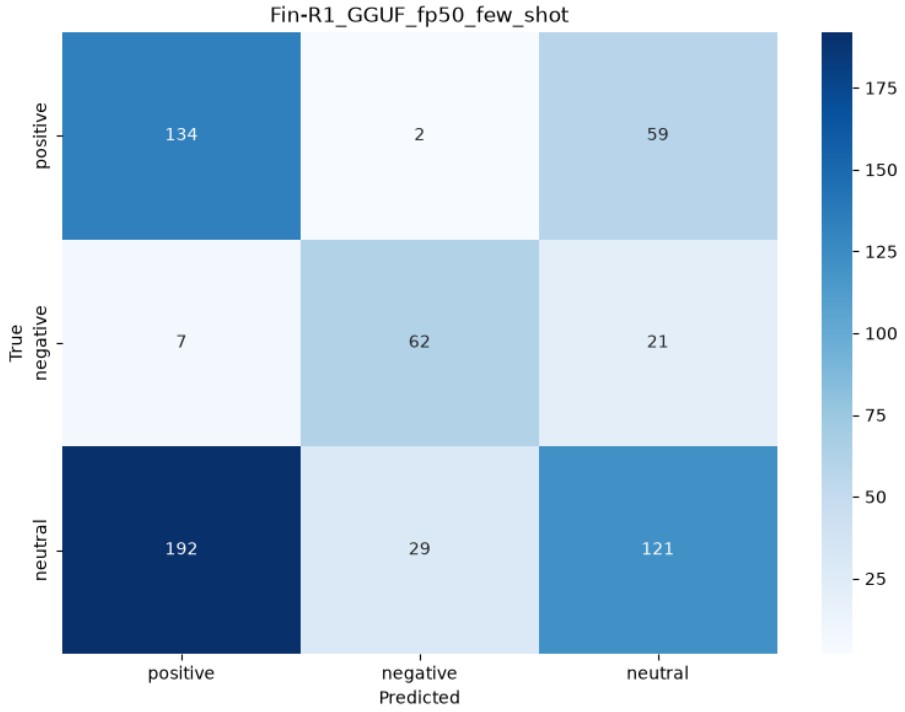

Dataset_('fp100',)
positive
negative
neutral
True
1859
58
363
5
1131
76
446
191
4912
positive
negative
neutral
Predicted
4000
3000
2000
1000

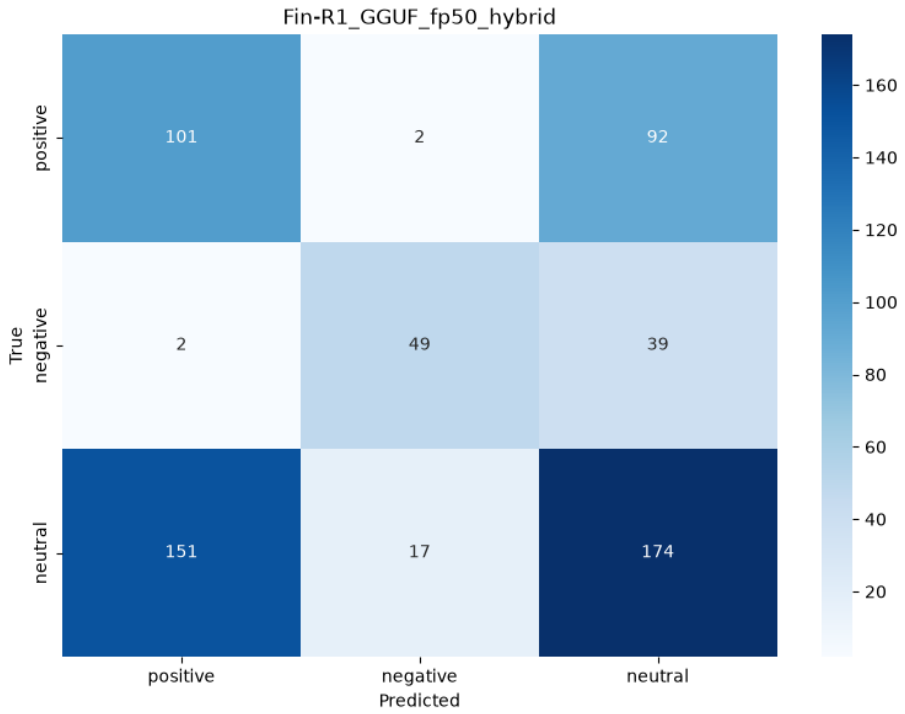

Fin-R1_GGUF_fiqa_few_shot
positive
negative
neutral
True
210
7
103
17
116
32
3
1
9
positive
negative
neutral
Predicted
200
175
150
125
100
75
50
25

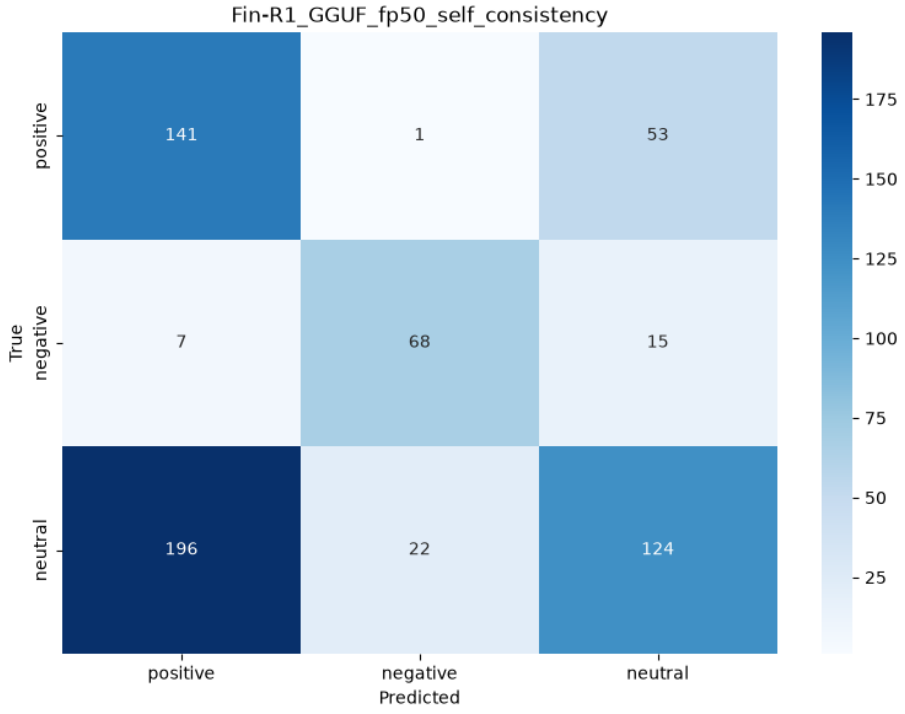

Fin-R1_GGUF_fiqa_hybrid
positive
negative
neutral
True
168
6
146
18
99
48
1
1
11
positive
negative
neutral
Predicted
160
140
120
100
80
60
40
20

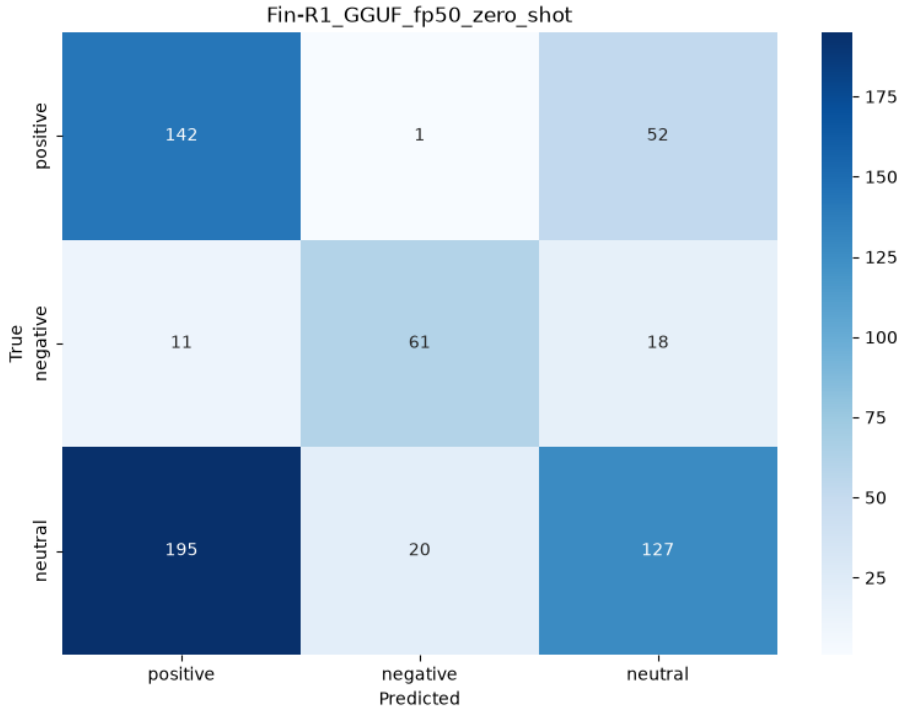

Fin-R1_GGUF_fiqa_self_consistency
positive
negative
neutral
True
201
5
114
15
117
33
1
0
12
positive
negative
neutral
Predicted
200
175
150
125
100
75
50
25
0

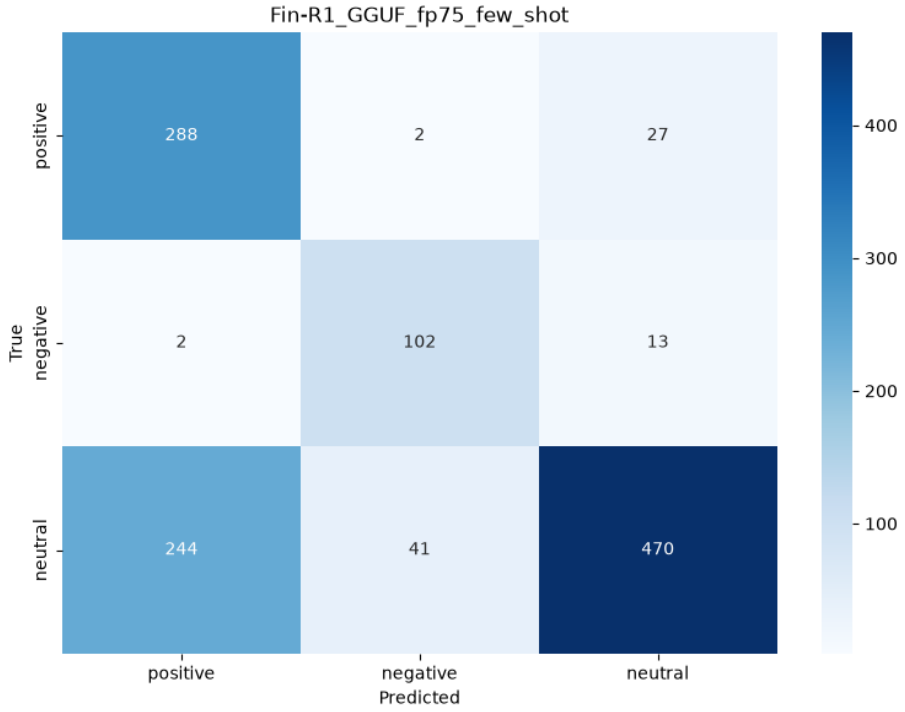

Fin-R1_GGUF_fiqa_zero_shot
positive
negative
neutral
True
216
7
97
21
113
31
1
0
12
positive
negative
neutral
Predicted
200
175
150
125
100
75
50
25
0

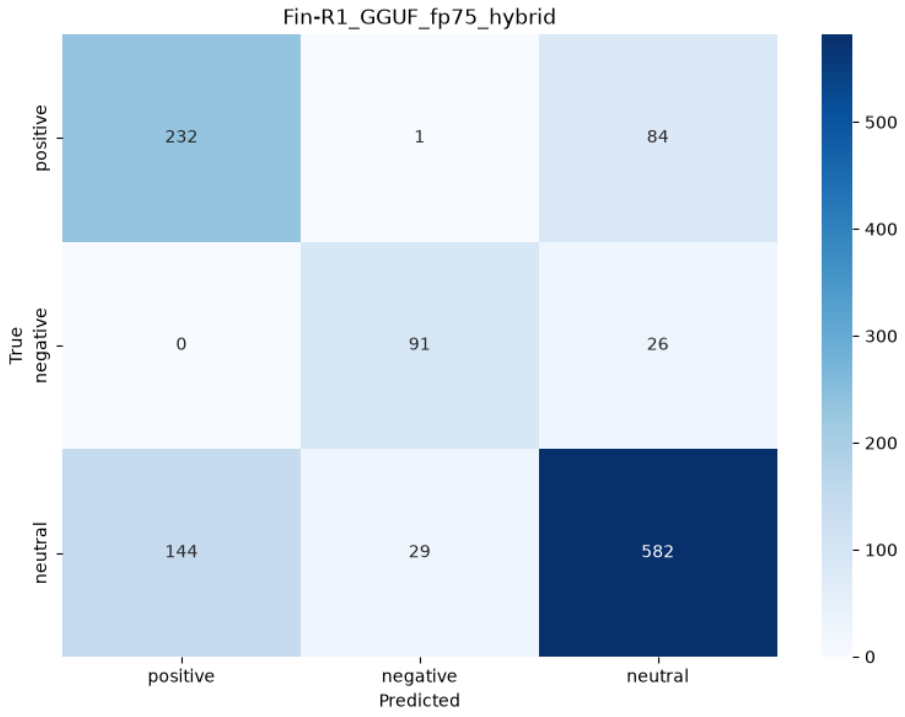

Fin-R1_GGUF_fp50_few_shot
positive
negative
neutral
True
134
2
59
7
62
21
192
29
121
positive
negative
neutral
Predicted
175
150
125
100
75
50
25

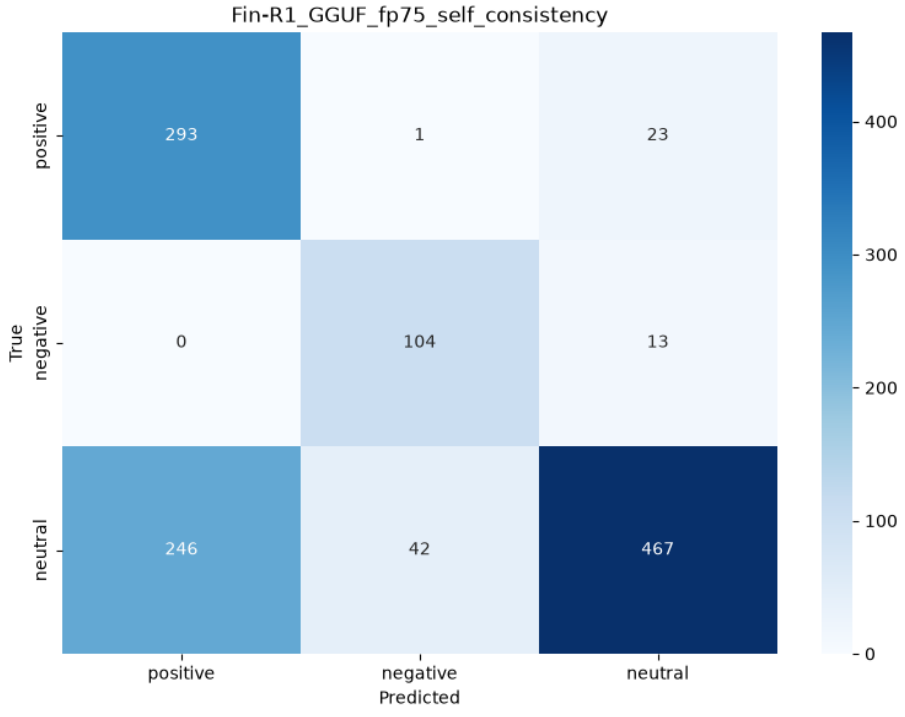

Fin-R1_GGUF_fp50_hybrid
positive
negative
neutral
True
101
2
92
2
49
39
151
17
174
positive
negative
neutral
Predicted
160
140
120
100
80
60
40
20

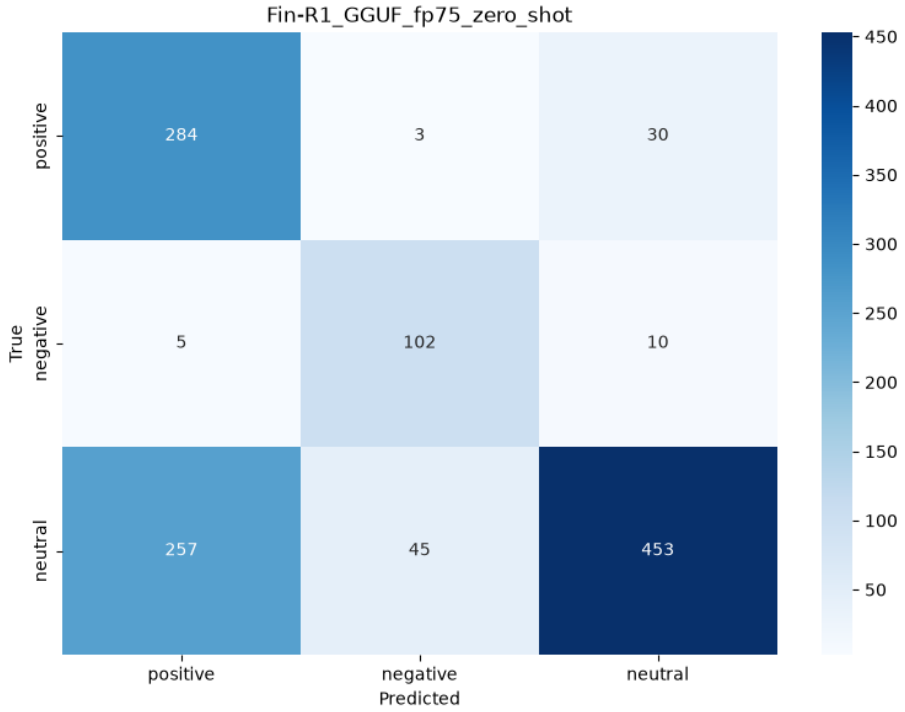

Fin-R1_GGUF_fp50_self_consistency
positive
negative
neutral
True
141
1
53
7
68
15
196
22
124
positive
negative
neutral
Predicted
175
150
125
100
75
50
25

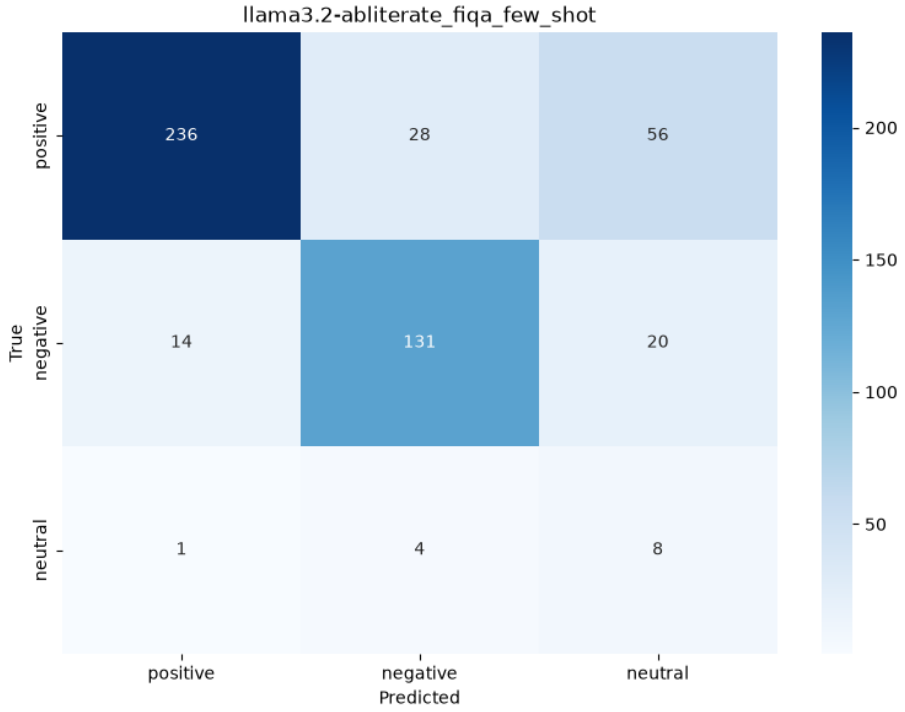

Fin-R1_GGUF_fp50_zero_shot
positive
negative
neutral
True
142
1
52
11
61
18
195
20
127
positive
negative
neutral
Predicted
175
150
125
100
75
50
25

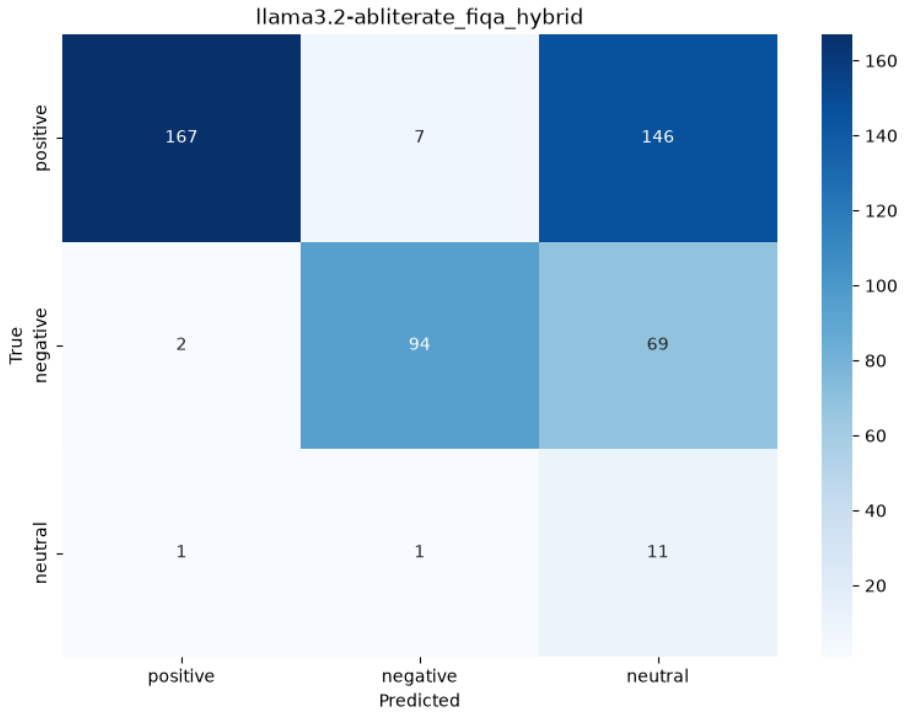

Fin-R1_GGUF_fp75_few_shot
positive
negative
neutral
True
288
2
27
2
102
13
244
41
470
positive
negative
neutral
Predicted
400
300
200
100

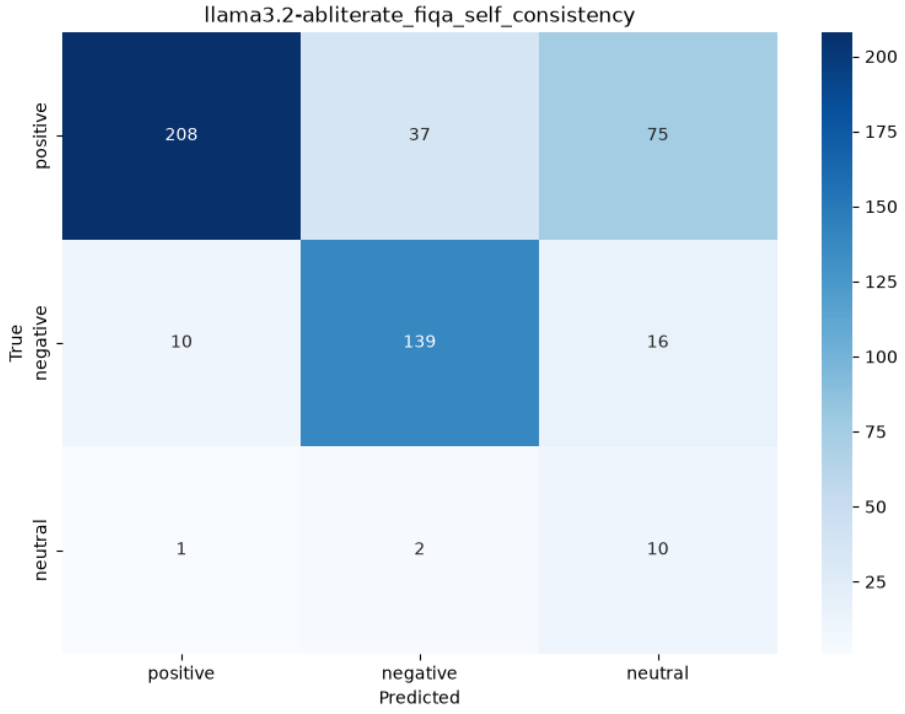

Fin-R1_GGUF_fp75_hybrid
positive
negative
neutral
True
232
1
84
0
91
26
144
29
582
positive
negative
neutral
Predicted
500
400
300
200
100
0

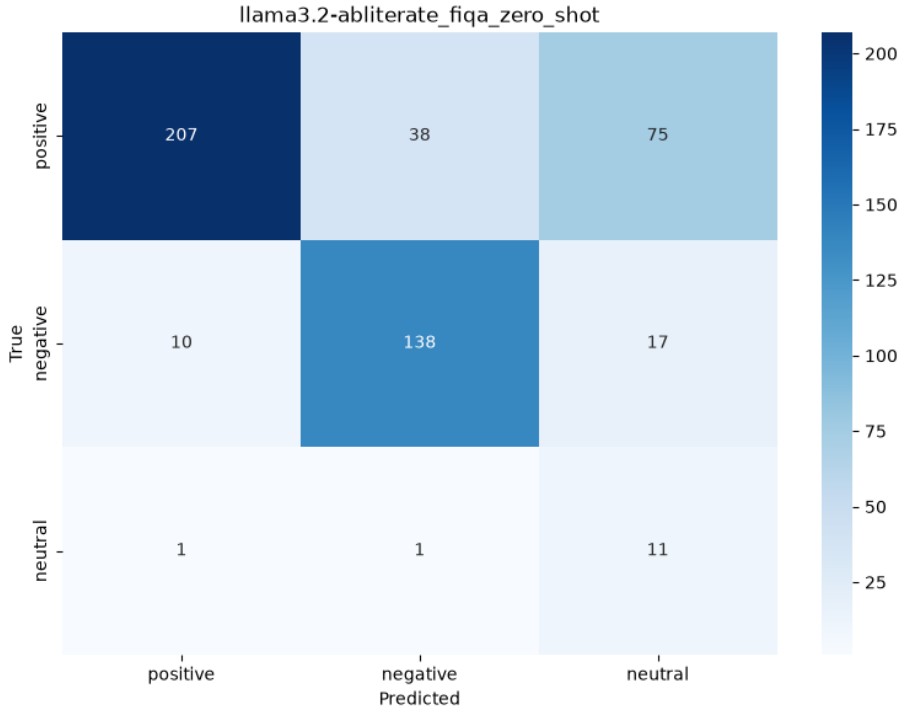

Fin-R1_GGUF_fp75_self_consistency
positive
negative
neutral
True
293
1
23
0
104
13
246
42
467
positive
negative
neutral
Predicted
400
300
200
100
0

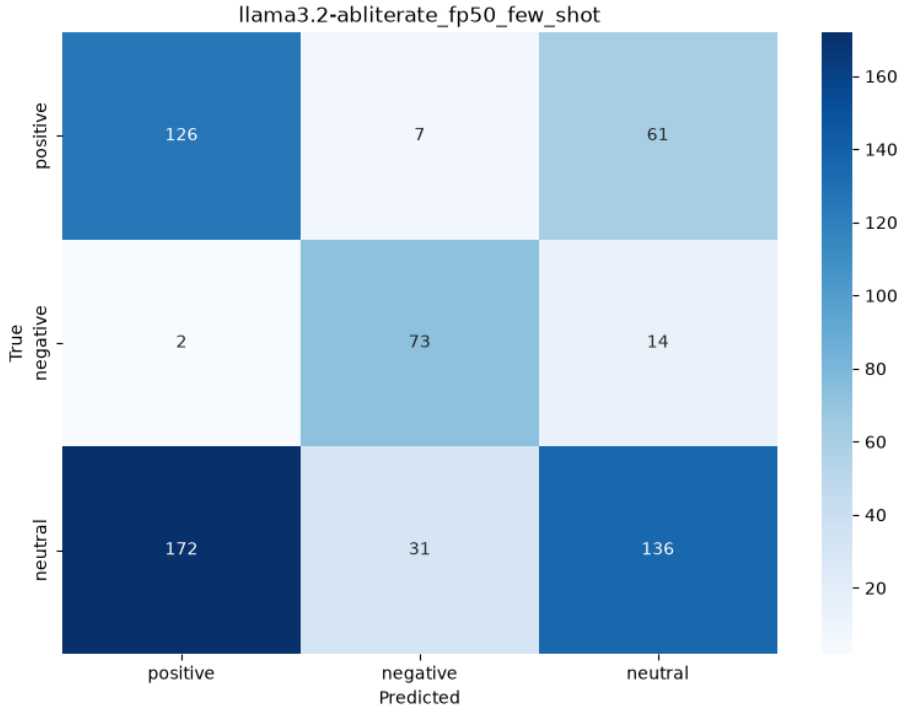

Fin-R1_GGUF_fp75_zero_shot
positive
negative
neutral
True
284
3
30
5
102
10
257
45
453
positive
negative
neutral
Predicted
450
400
350
300
250
200
150
100
50

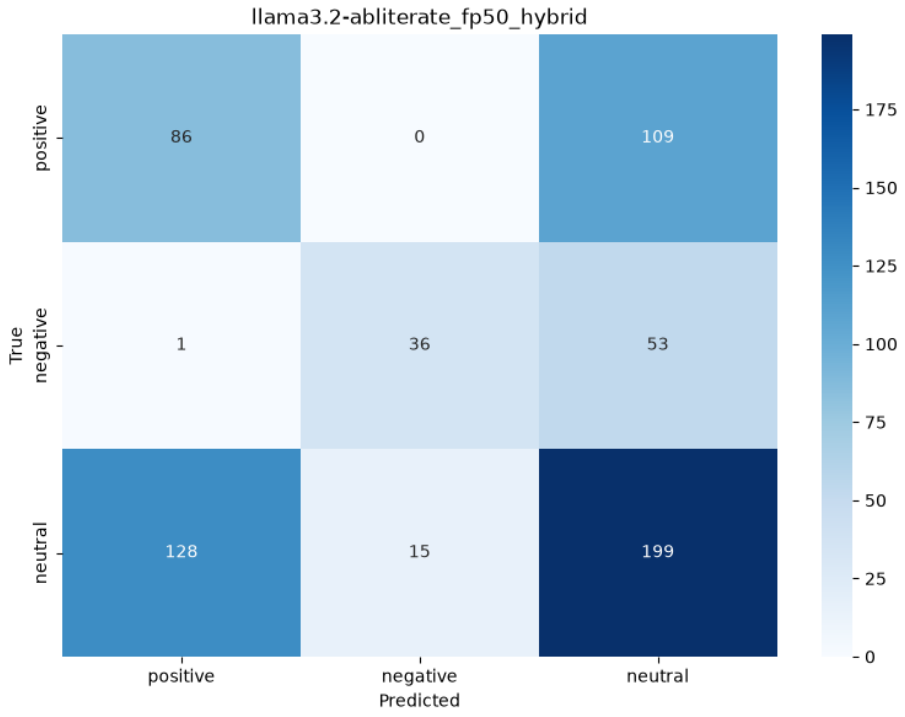

llama3.2-abliterate_fiqa_few_shot
positive
negative
neutral
True
236
28
56
14
131
20
1
4
8
positive
negative
neutral
Predicted
200
150
100
50

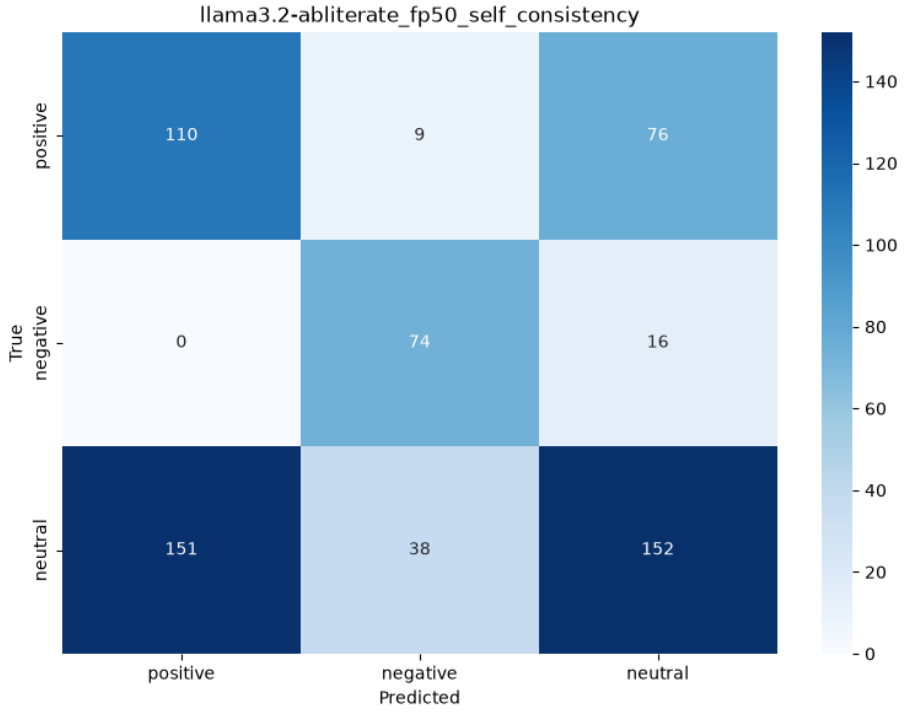

llama3.2-abliterate_fiqa_hybrid
positive
negative
neutral
True
167
7
146
2
94
69
1
1
11
positive
negative
neutral
Predicted
160
140
120
100
80
60
40
20

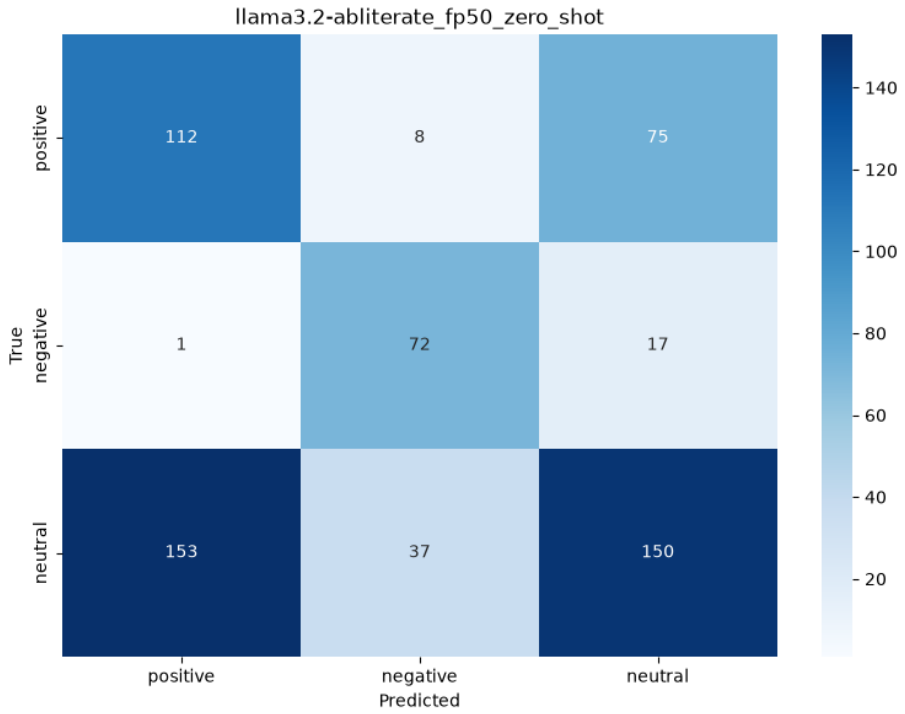

llama3.2-abliterate_fiqa_self_consistency
positive
negative
neutral
True
208
37
75
10
139
16
1
2
10
positive
negative
neutral
Predicted
200
175
150
125
100
75
50
25

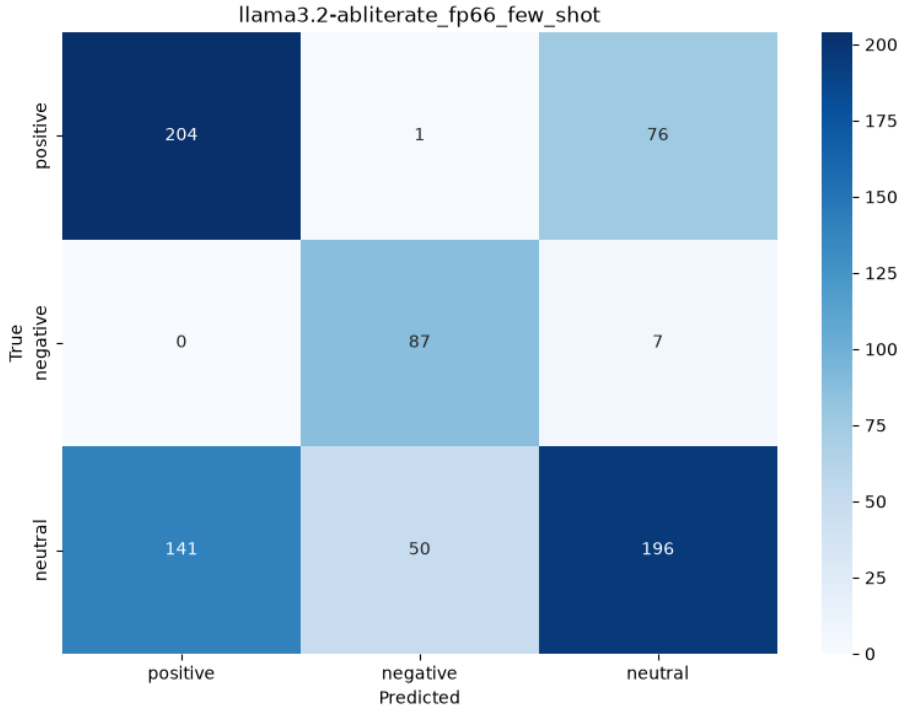

llama3.2-abliterate_fiqa_zero_shot
positive
negative
neutral
True
207
38
75
10
138
17
1
1
11
positive
negative
neutral
Predicted
200
175
150
125
100
75
50
25

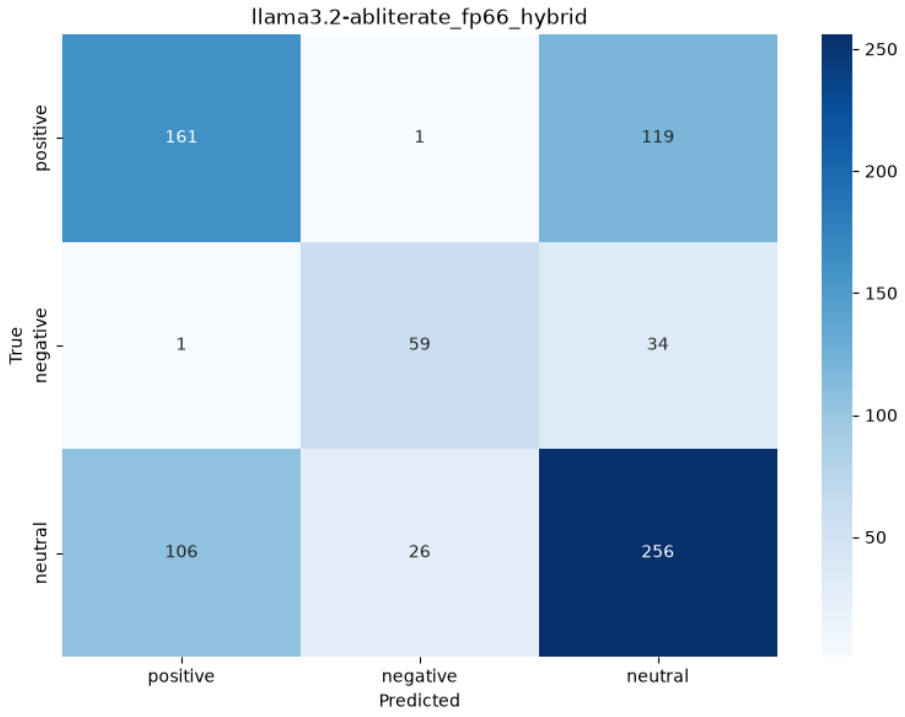

llama3.2-abliterate_fp50_few_shot
positive
negative
neutral
True
126
7
61
2
73
14
172
31
136
positive
negative
neutral
Predicted
160
140
120
100
80
60
40
20

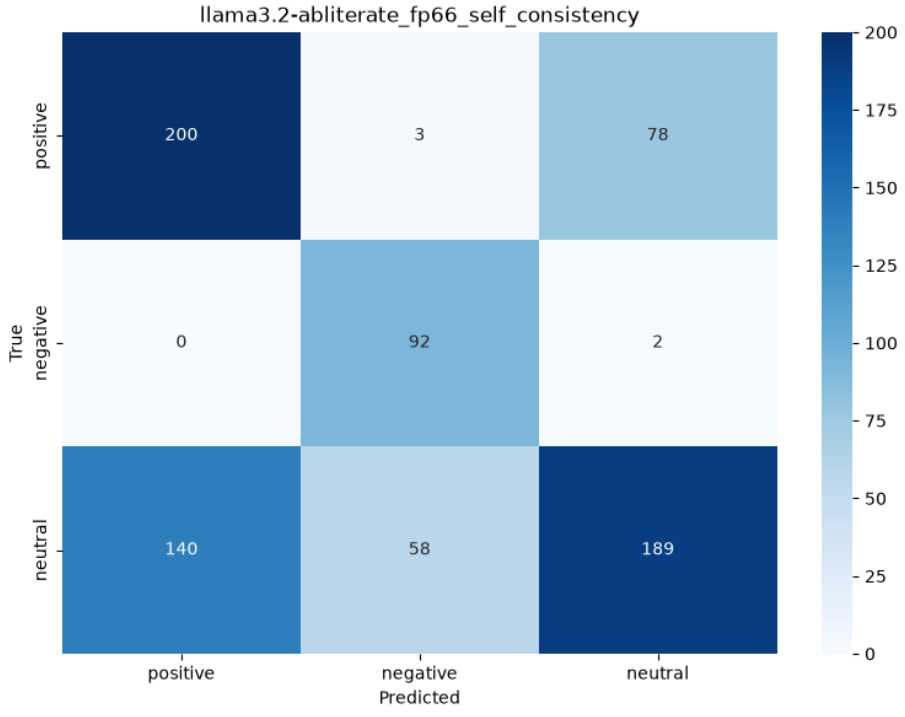

llama3.2-abliterate_fp50_hybrid
positive
negative
neutral
True
86
0
109
1
36
53
128
15
199
positive
negative
neutral
Predicted
175
150
125
100
75
50
25
0

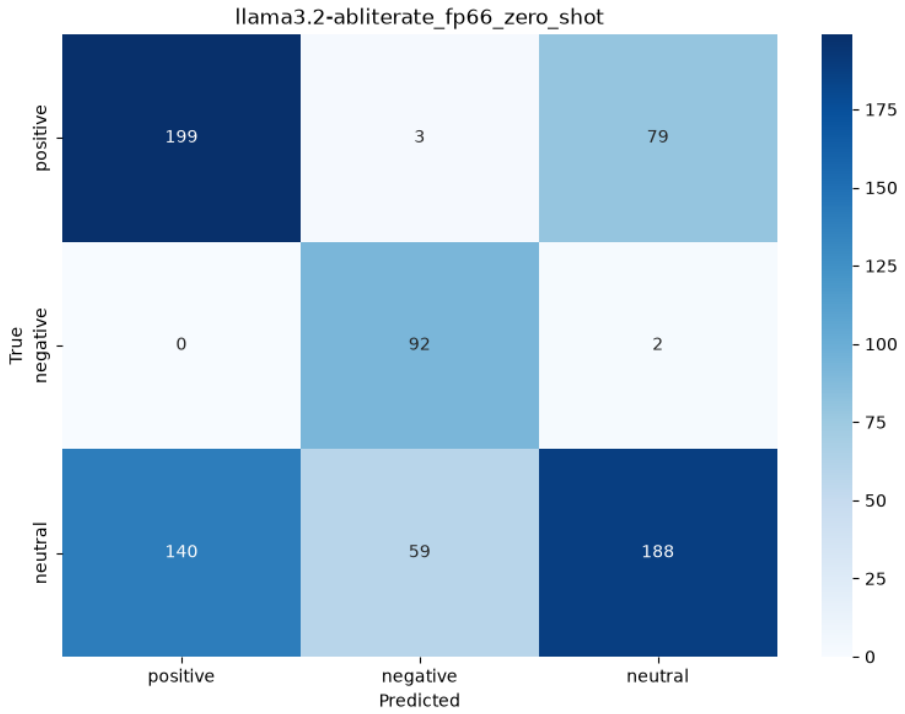

llama3.2-abliterate_fp50_self_consistency
positive
negative
neutral
True
110
9
76
0
74
16
151
38
152
positive
negative
neutral
Predicted
140
120
100
80
60
40
20
0

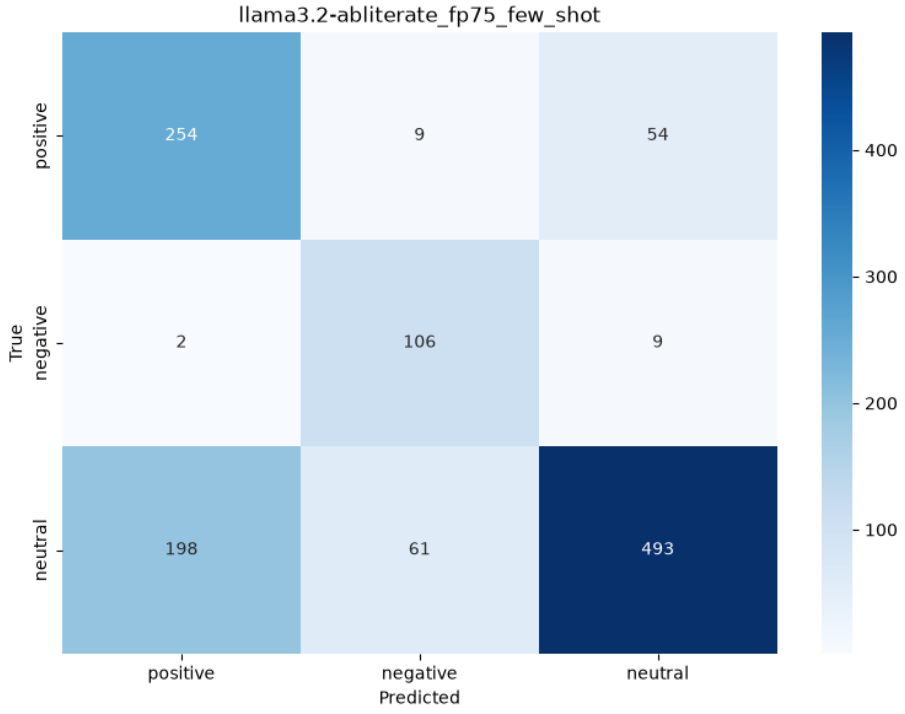

llama3.2-abliterate_fp50_zero_shot
positive
negative
neutral
True
112
8
75
1
72
17
153
37
150
positive
negative
neutral
Predicted
140
120
100
80
60
40
20

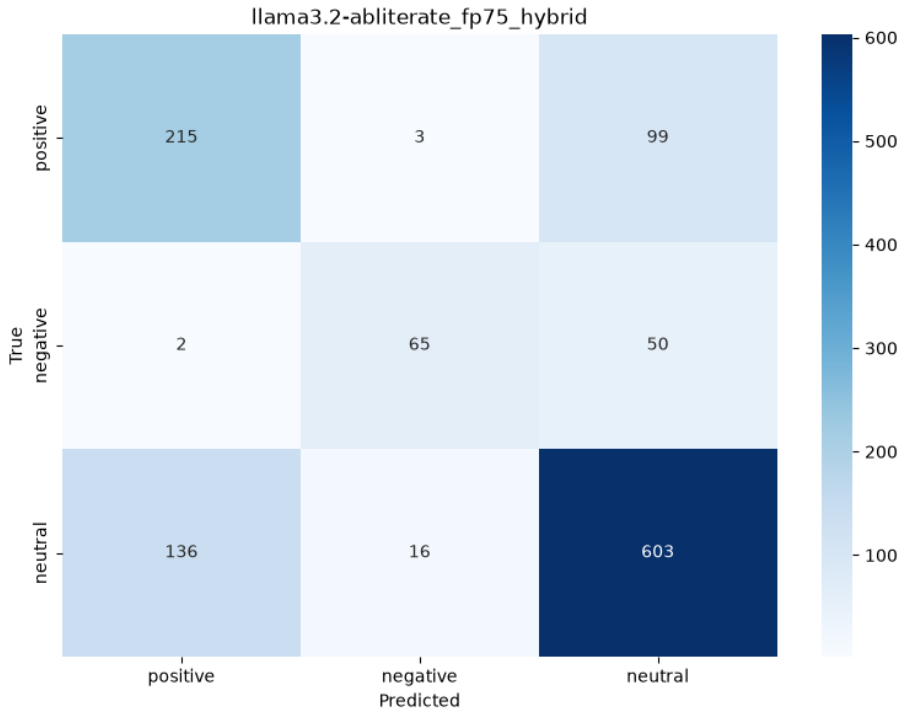

llama3.2-abliterate_fp66_few_shot
positive
negative
neutral
True
204
1
76
0
87
7
141
50
196
positive
negative
neutral
Predicted
200
175
150
125
100
75
50
25
0

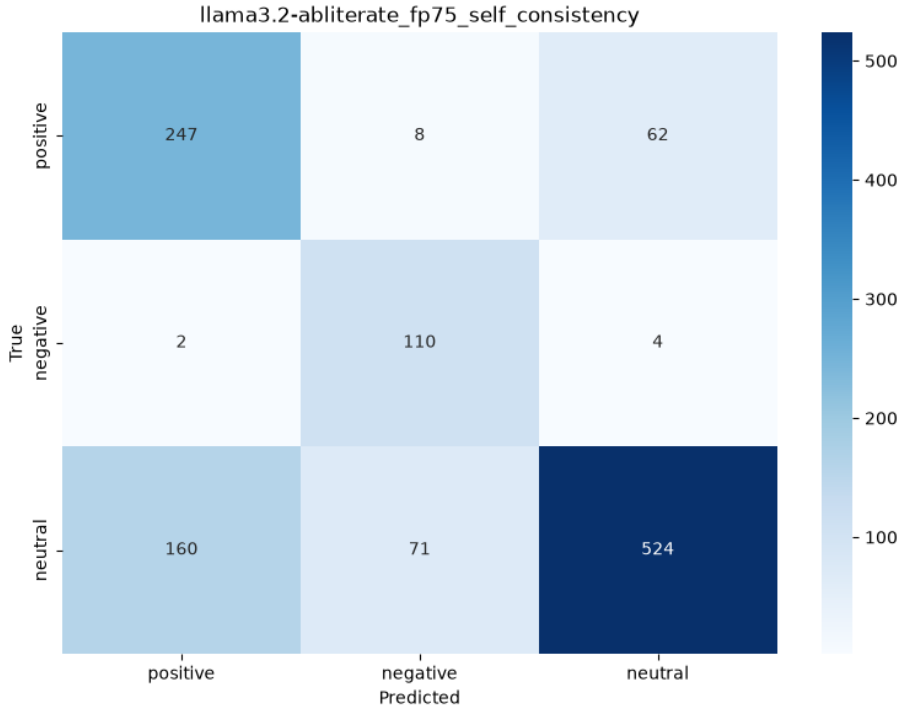

llama3.2-abliterate_fp66_hybrid
positive
negative
neutral
True
161
1
119
1
59
34
106
26
256
positive
negative
neutral
Predicted
250
200
150
100
50

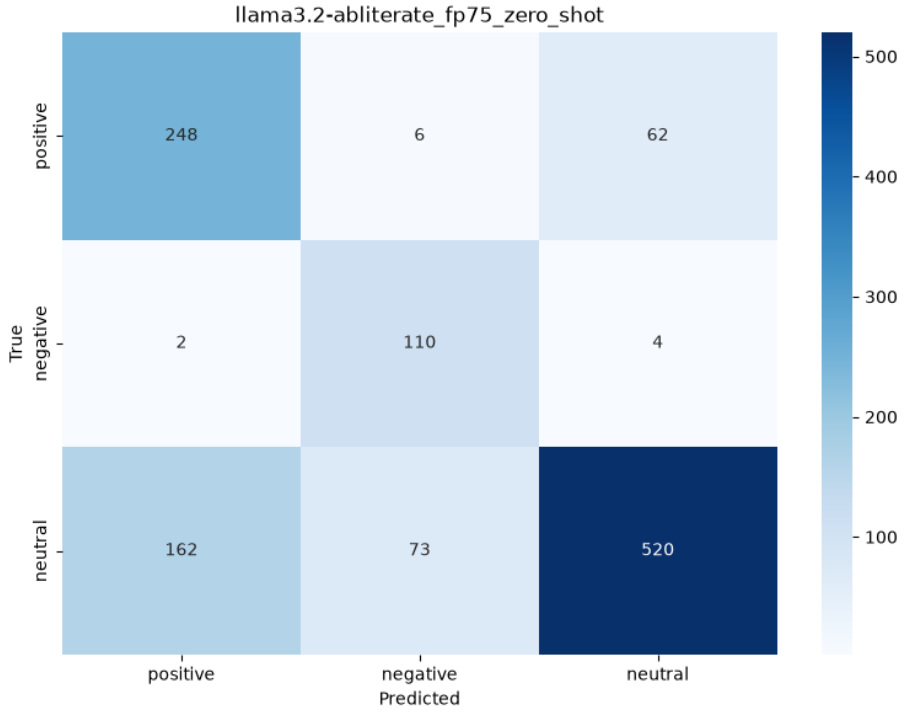

llama3.2-abliterate_fp66_self_consistency
positive
negative
neutral
True
200
3
78
0
92
2
140
58
189
positive
negative
neutral
Predicted
200
175
150
125
100
75
50
25
0

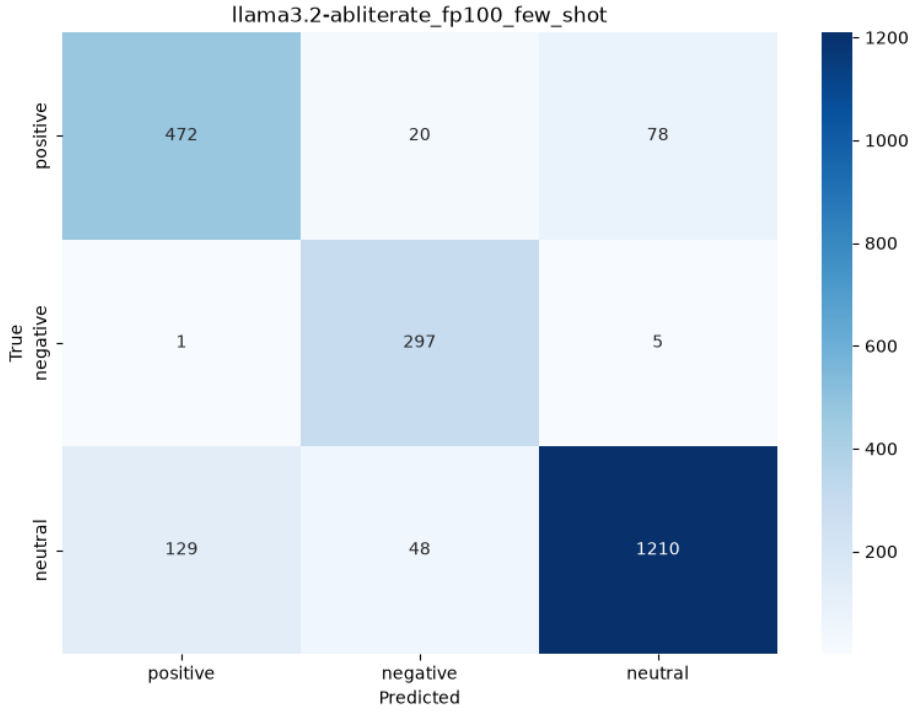

llama3.2-abliterate_fp66_zero_shot
positive
negative
neutral
True
199
3
79
0
92
2
140
59
188
positive
negative
neutral
Predicted
175
150
125
100
75
50
25
0

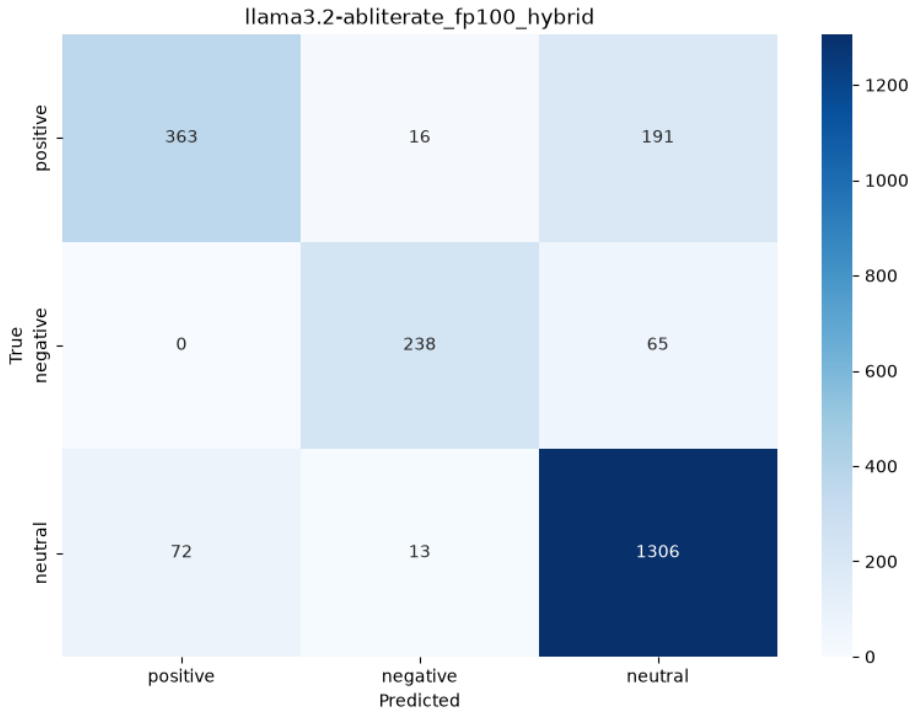

llama3.2-abliterate_fp75_few_shot
positive
negative
neutral
True
254
9
54
2
106
9
198
61
493
positive
negative
neutral
Predicted
400
300
200
100

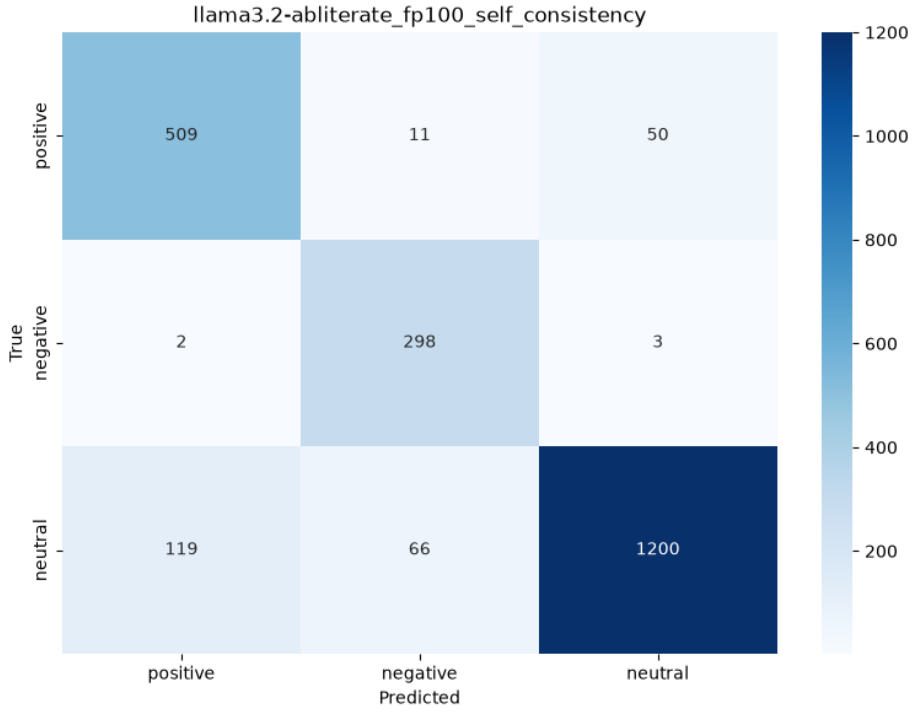

llama3.2-abliterate_fp75_hybrid
True
positive
negative
neutral
215
3
99
2
65
50
136
16
603
positive
negative
neutral
Predicted
600
500
400
300
200
100

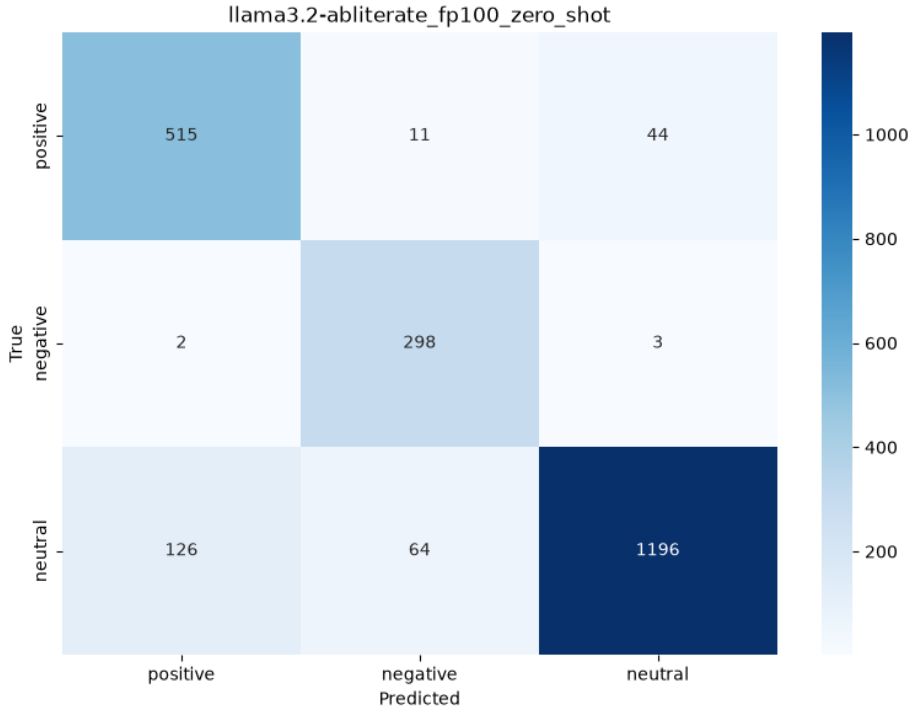

llama3.2-abliterate_fp75_self_consistency
True
positive
negative
neutral
247
8
62
2
110
4
160
71
524
positive
negative
neutral
Predicted
500
400
300
200
100

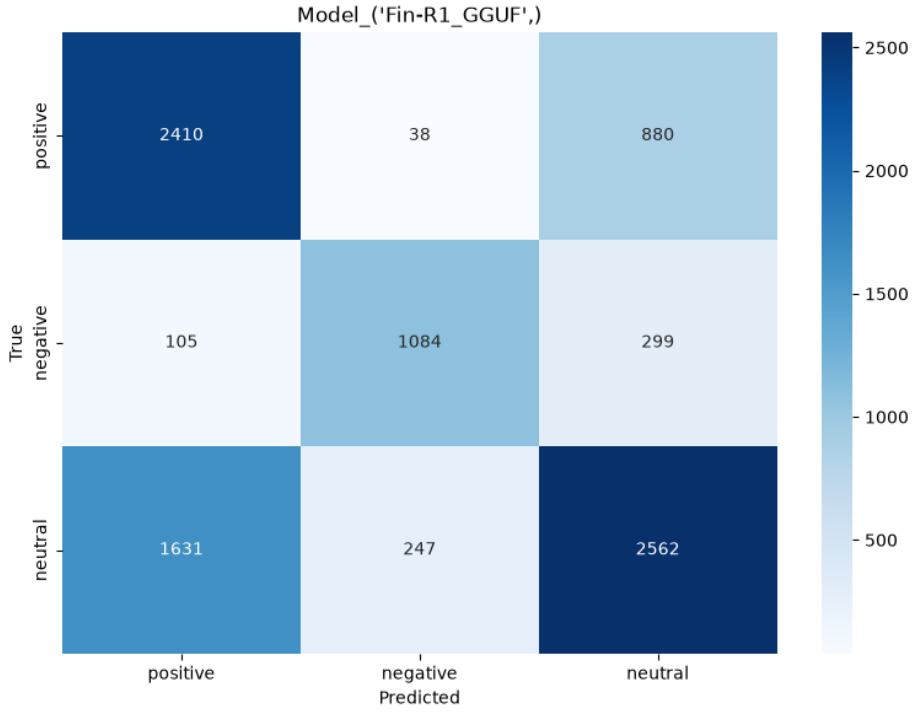

llama3.2-abliterate_fp75_zero_shot
positive
negative
neutral
True
248
6
62
2
110
4
162
73
520
positive
negative
neutral
Predicted
500
400
300
200
100

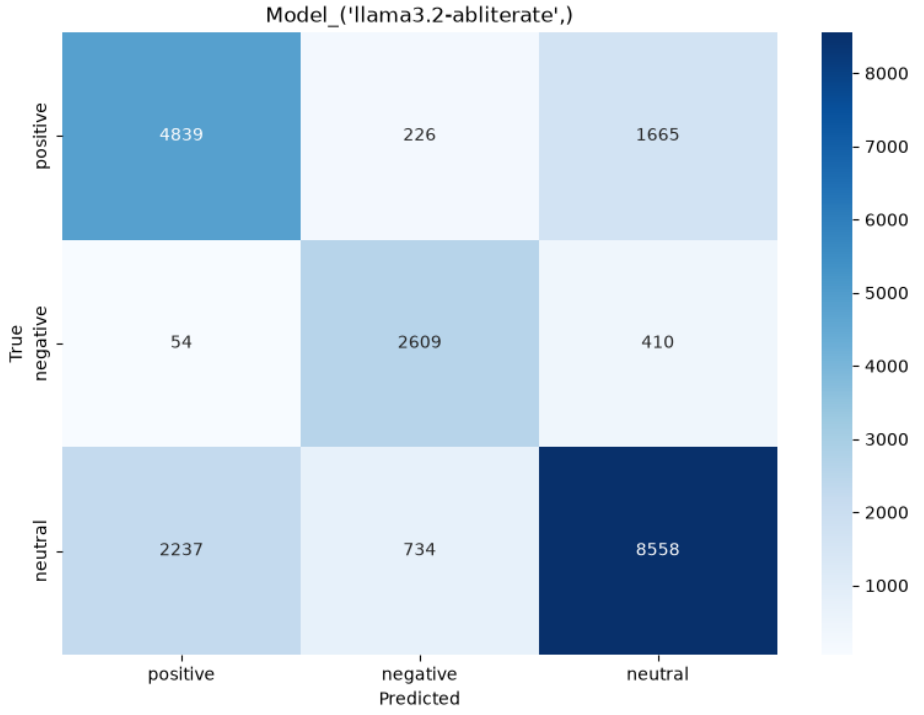

llama3.2-abliterate_fp100_few_shot
positive
negative
neutral
True
472
20
78
1
297
5
129
48
1210
positive
negative
neutral
Predicted
1200
1000
800
600
400
200

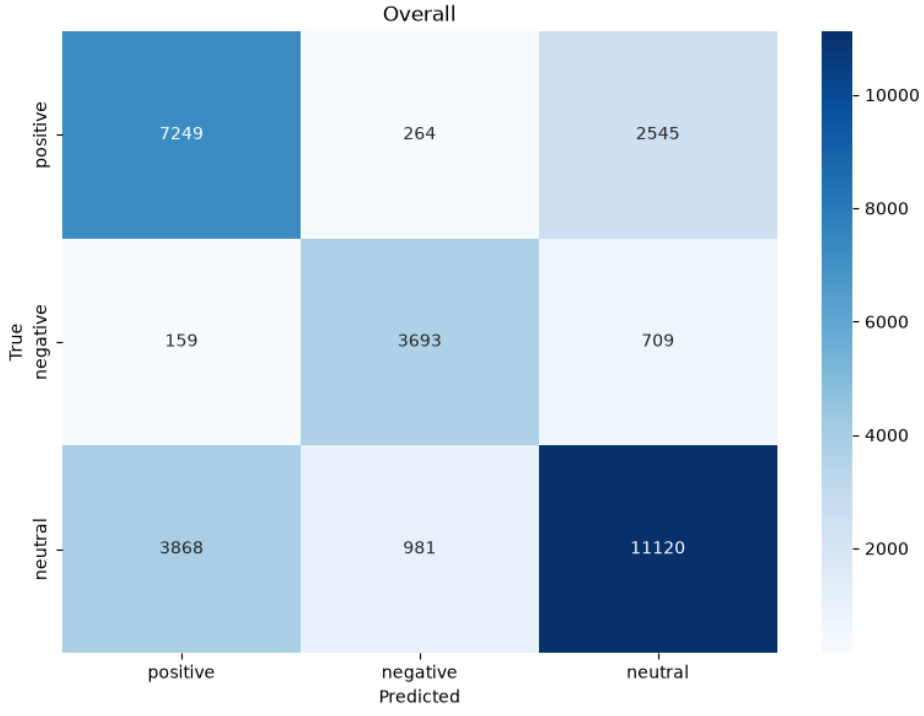

llama3.2-abliterate_fp100_hybrid
positive
negative
neutral
True
363
16
191
0
238
65
72
13
1306
positive
negative
neutral
Predicted
1200
1000
800
600
400
200
0

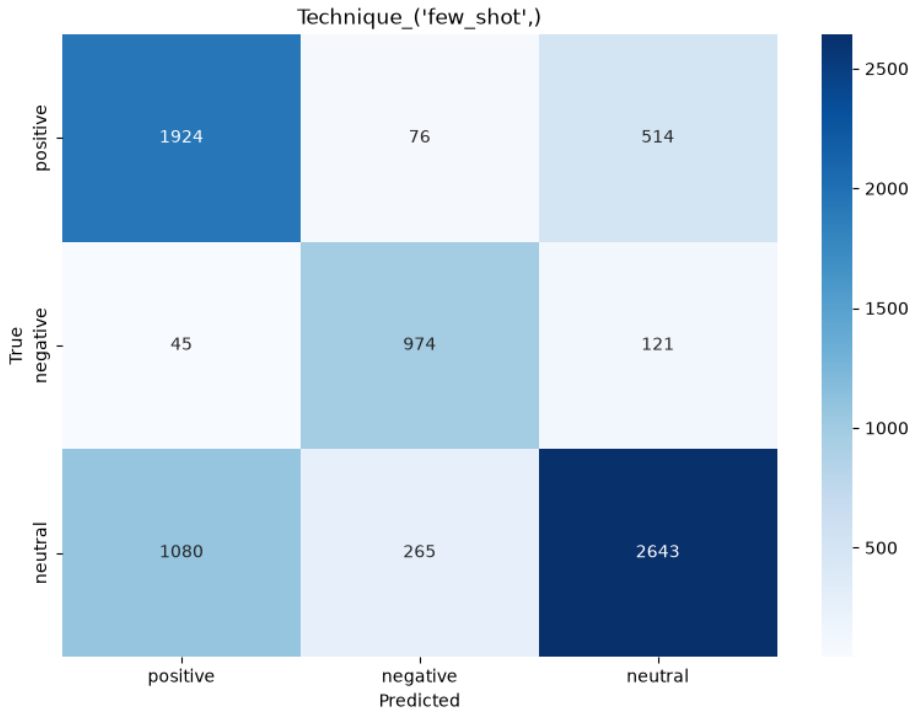

llama3.2-abliterate_fp100_self_consistency
positive
negative
neutral
True
509
11
50
2
298
3
119
66
1200
positive
negative
neutral
Predicted
1200
1000
800
600
400
200

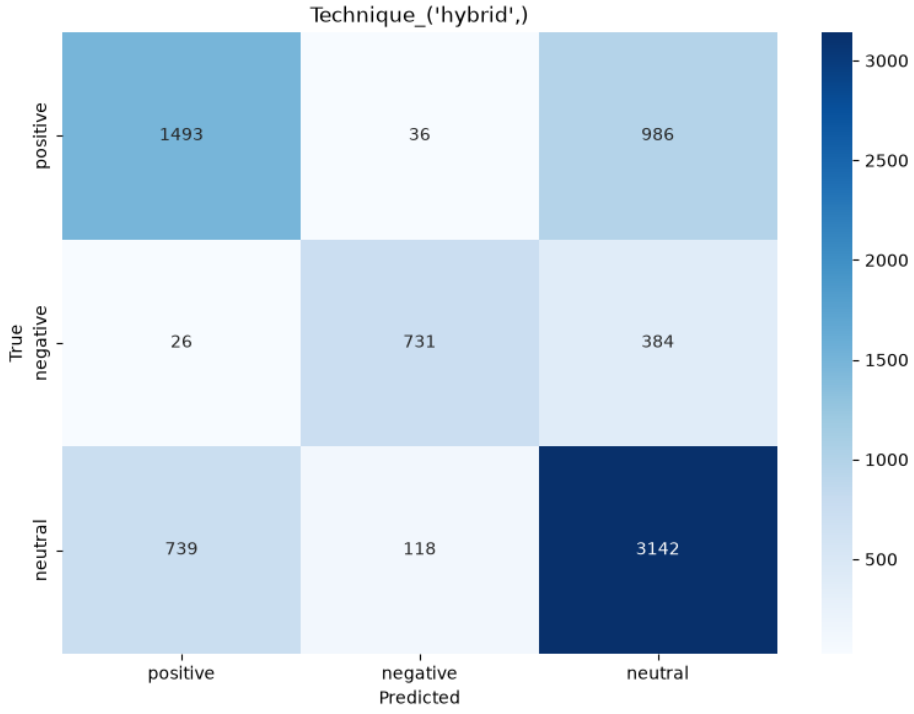

llama3.2-abliterate_fp100_zero_shot
positive
negative
neutral
True
515
11
44
2
298
3
126
64
1196
positive
negative
neutral
Predicted
1000
800
600
400
200

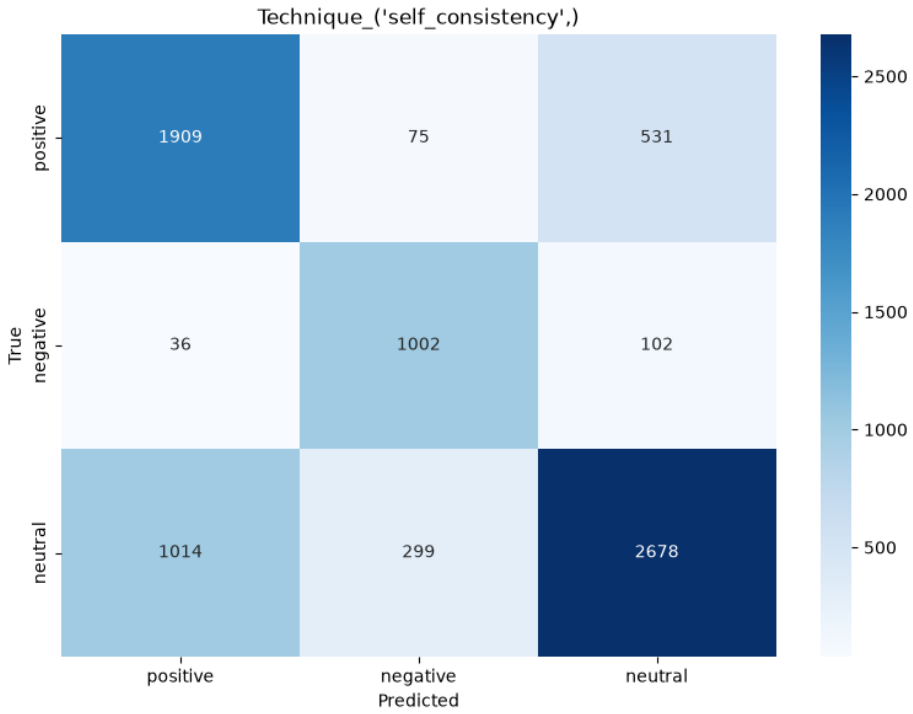

Model_('Fin-R1_GGUF',)
positive
negative
neutral
True
2410
38
880
105
1084
299
1631
247
2562
positive
negative
neutral
Predicted
2500
2000
1500
1000
500

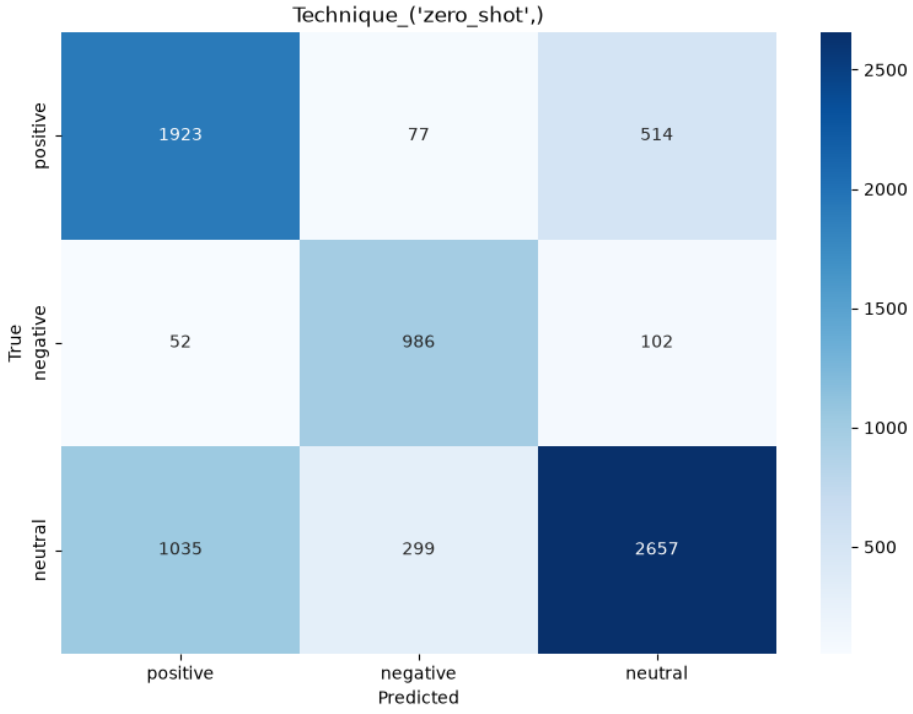

Model_('llama3.2-abliterate',)
positive
negative
neutral
True
4839
226
1665
54
2609
410
2237
734
8558
positive
negative
neutral
Predicted
8000
7000
6000
5000
4000
3000
2000
1000


Overall
positive
negative
neutral
True
7249
264
2545
159
3693
709
3868
981
11120
positive
negative
neutral
Predicted
10000
8000
6000
4000
2000


Technique_('few_shot',)
positive
negative
neutral
True
1924
76
514
45
974
121
1080
265
2643
positive
negative
neutral
Predicted
2500
2000
1500
1000
500


Technique_('hybrid',)
positive
negative
neutral
True
1493
36
986
26
731
384
739
118
3142
positive
negative
neutral
Predicted
3000
2500
2000
1500
1000
500


Technique_('self_consistency',)
positive
negative
neutral
True
1909
75
531
36
1002
102
1014
299
2678
positive
negative
neutral
Predicted
2500
2000
1500
1000
500


Technique_('zero_shot',)
positive
negative
neutral
True
1923
77
514
52
986
102
1035
299
2657
positive
negative
neutral
Predicted
2500
2000
1500
1000
500